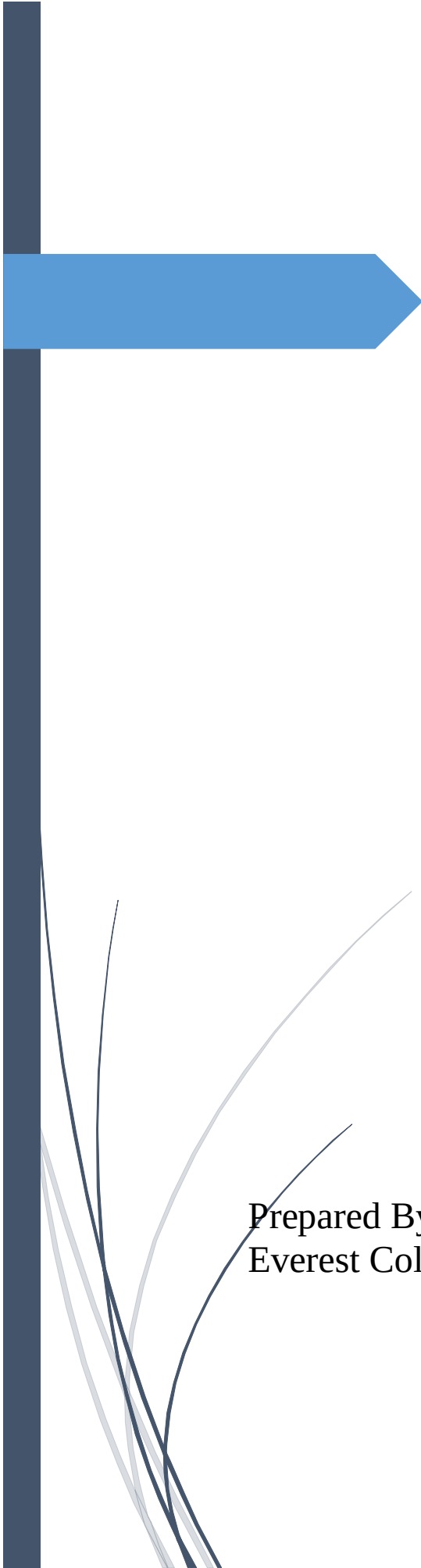




Embedded System

BE, Computer Sixth Semester

Prepared By: Er. Loknath Regmi(lnregmi046@gmail.com)
Everest College of Engineering and Management

Chapter -1

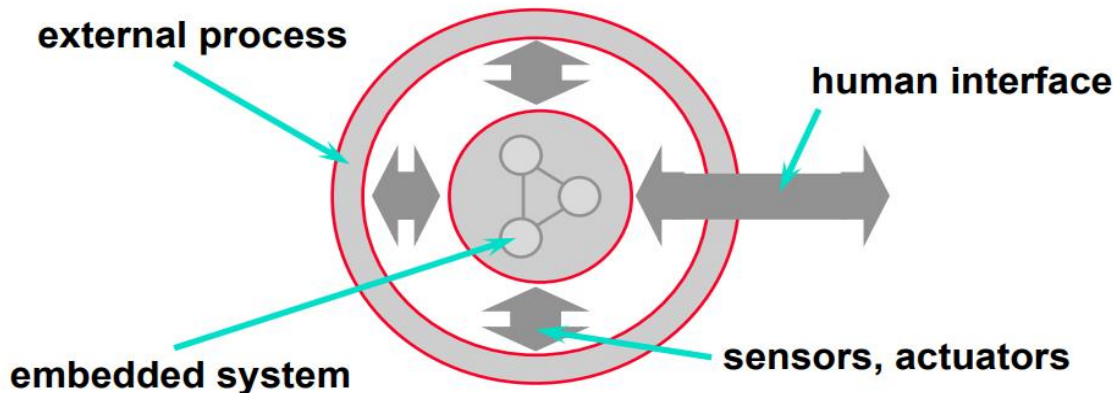
Embedded System

Embedded System Overview:

An embedded system is a special-purpose system in which the computer is completely encapsulated by the device it controls. Unlike a general-purpose computer, such as a personal computer, an embedded system performs pre-defined tasks, usually with very specific requirements. Since the system is dedicated to a specific task, design engineers can optimize it, reducing the size and cost of the product. Embedded systems are often mass-produced, so the cost savings may be multiplied by millions of items.

Handheld computers or PDAs are generally considered embedded devices because of the nature of their hardware design, even though they are more expandable in software terms. This line of definition continues to blur as devices expand.

Hence, it is an information processing systems embedded into a larger product. In general it is the core computational part of any automated system shown as below.



Embedded system are founded in verity of electronics devices as below.

1. Consumer Electronics ⇒ for example MP3 Audio, digital camera, home electronics, ...
2. Information Systems ⇒ for example wireless communication (mobile phone, Wireless LAN, ...), end-user equipment, router, ...
3. Home Appliances ⇒ for example micro-oven, Home Security System, washing machines, lighting system...
4. Office Automation ⇒ for example fax machines, printer, attendance system, pager.....
5. Business Equipment ⇒ for example Cash registers, Card Reader, alarm system, Product Scanner, Automated telling machine
6. Automobiles: for example cruise control, driver assistance system, parking assistance systems, anti-lock brakes.....

Characteristics of Embedded System:

The main characteristics of embedded system are:

Single-Functioned --An embedded system usually executes a specific program reputedly. For example: pager is always pager. A desktop system executes verities of program like spread sheet, word processors

and video games, with new programs added frequently. One case is here the embedded system software be updated to newer versions over a periods of time. For example smart phone software be updated to new versions.

Second case is where several programs are swapped in and out of system due to limitation of size. For example, some missiles run one program while cruise mode, then load a second program for locking into the target.

Tightly Constraint --- All computing system have constraint on design metrics but those on embedded system can be especially tight. A design metric is a measures of implementations feature such as cost, size, power and performance. Embedded system often must cost just a few dollars, must be sized to fit on single size, must perform enough to process the real time data and must consume the minimum power to extend the battery life.

Real Time and Reactive -- Many embedded system most continuously react in many changes in system environments and must compute the certain result in real time without delay. For example a car cruise controller continuously monitors and react to speed and break sensor. It must compute the acceleration and deceleration amount repeatedly with in limited time. A delay computation cause to fail to control the car.

Microprocessors based – It must be microprocessor or microcontroller based.

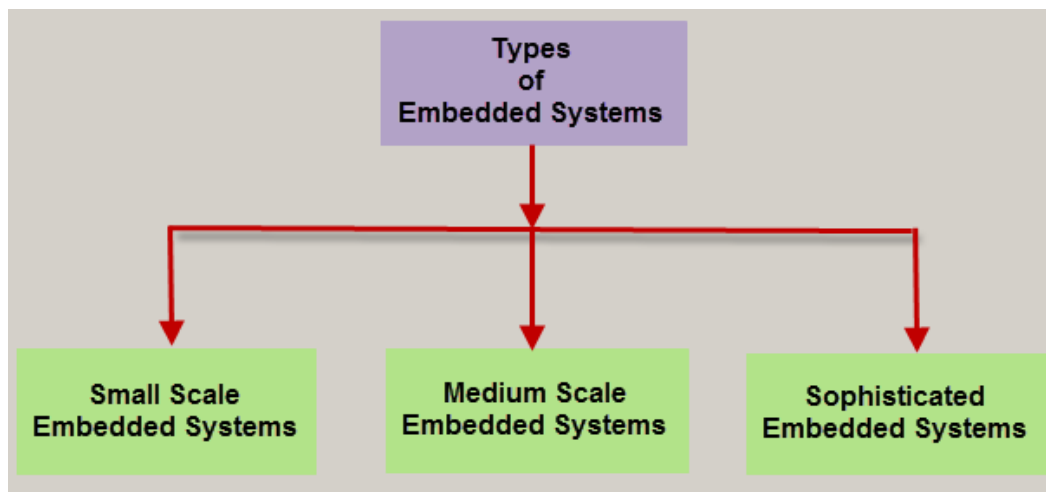
Memory – It must have a memory, as its software usually embeds in ROM. It does not need any secondary memories in the computer.

Connected – It must have connected peripherals to connect input and output devices.

HW-SW systems – Software is used for more features and flexibility. Hardware is used for performance and security.

Classification of Embedded System:

Embedded systems are classified into three:



Small Scale Embedded Systems:

Small scale embedded systems are designed with a single 8 or 16-bit microcontroller which may even be operated with a battery. For developing embedded software for these types of systems, an editor, assembler, (IDE) integrated development environment, and cross assembler are the main programming tools.

Medium Scale Embedded Systems:

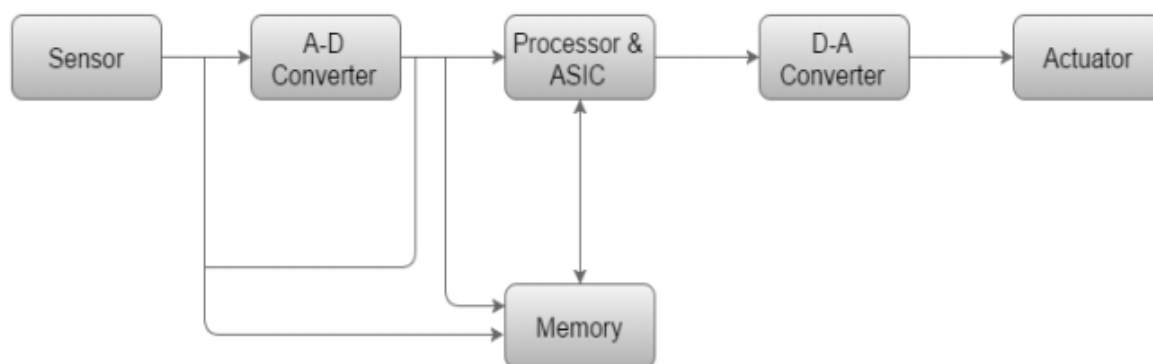
Medium scale embedded systems are designed with a single or few 16 or 32 bit microcontrollers, DSPs or RISCs. These systems have both hardware and software complexities. When developing embedded software for these types of systems, the following programming tools are available. They are C, C++, Visual C++, Java, and RTOS, source code engineering tool, debugger, simulator and integrated development environment.

Sophisticated Embedded Systems:

Sophisticated embedded systems have huge hardware and software complexities and may need PLAs, IPs, ASIPs, scalable processors or configurable processors. They are used for cutting-edge applications that need hardware and software co-design & components which have to combine in the final system.

Basic Structure of an Embedded System:

The following illustration shows the basic structure of an embedded system:



Sensor – It measures the physical quantity and converts it to an electrical signal which can be read by an observer or by any electronic instrument like an A2D converter. A sensor stores the measured quantity to the memory.

A-D Converter – An analog-to-digital converter converts the analog signal sent by the sensor into a digital signal.

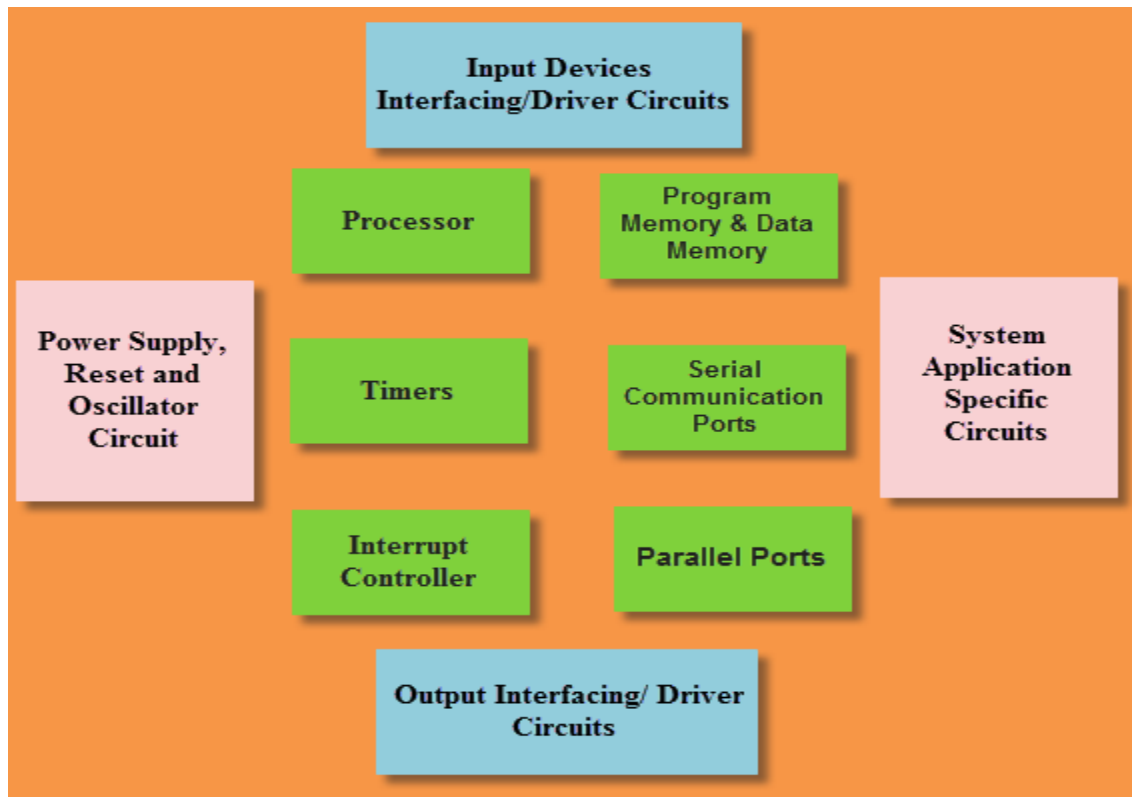
Processor & ASICs – Processors process the data to measure the output and store it to the memory.

D-A Converter – A digital-to-analog converter converts the digital data fed by the processor to analog data.

Actuator – An actuator compares the output given by the D-A Converter to the actual (expected) output stored in it and stores the approved output.

Components of Embedded System:

The embedded systems basics include the components of embedded system hardware, embedded system types and several characteristics. An embedded system has three main components: Embedded system hardware, embedded system software and Operating system.



Embedded system block diagram

Embedded System Hardware:

As with any electronic system, an embedded system requires a hardware platform on which it performs the operation. Embedded system hardware is built with a microprocessor or microcontroller. The embedded system hardware has elements like input output (I/O) interfaces, user interface, memory and the display. Usually, an embedded system consists of:

- Power Supply
- Processor
- Memory
- Timers
- Serial communication ports
- Output/Output circuits
- System application specific circuits

Embedded System Software:

The embedded system software is written to perform a specific function. It is typically written in a high level format and then compiled down to provide code that can be lodged within a non-volatile memory within the hardware. An embedded system software is designed to keep in view of the three limits:

- Availability of system memory
- Availability of processor's speed
- When the system runs continuously, there is a need to limit power dissipation for events like stop, run and wake up.

Real Time Operating System

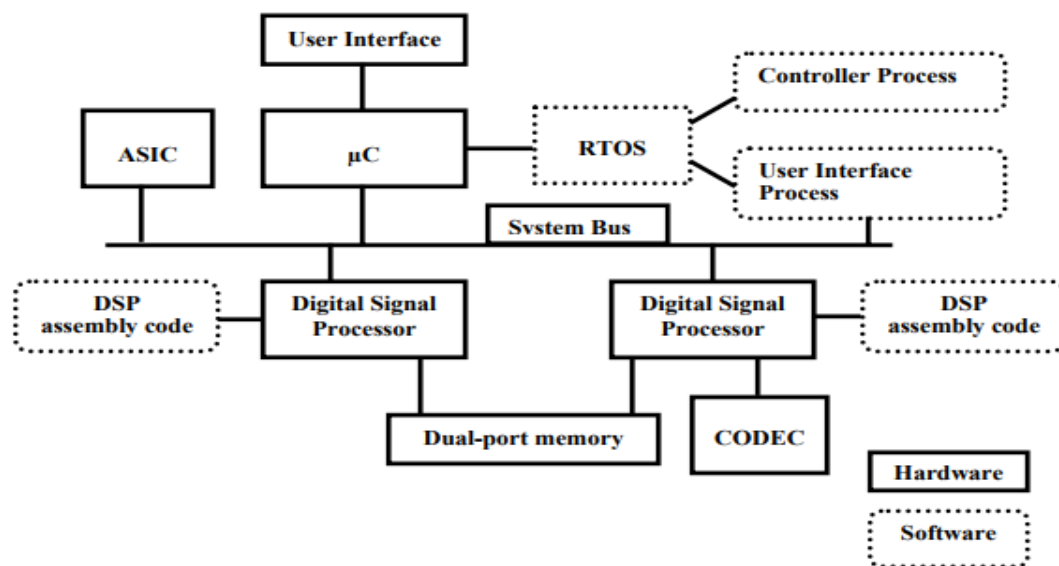
A system is said to be real time, if it is essential to complete its work and deliver its service on time. Real time operating system manages the application software and affords a mechanism to let the processor run. The Real Time operating system is responsible for handling the hardware resources of a computer and host applications which run on the computer.

An RTOS is specially designed to run applications with very precise timing and a high amount of reliability. Especially, this can be important in measurement and industrial automation systems wherein downtime is costly or a program delay could cause a safety hazard.

Architecture of Embedded System:

A Desktop Computer will have more open standards than an Embedded System. This is because of the level of integration in the later. Many of the components of the embedded systems are integrated on to a single chip. This concept is known as **System on Chip (SOC)** design. Thus there are only few subsystems left to be connected.

Analyzing the assembling process of a Desktop let us comparatively assess the possible subsystems of the typical RTES.



Architecture of Embedded System

User Interface: for interacting with users. May consists of keyboard, touch pad etc

ASIC: Application Specific Integrated Circuit: for specific functions like motor control, data modulation etc.

Microcontroller (μC): A family of microprocessors.

Real Time Operating System (RTOS): contains all the software for the system control and user interface
Controller Process: The overall control algorithm for the external process. It also provides timing and control for the various units inside the embedded system.

Digital Signal Processor (DSP) a typical family of microprocessors

DSP assembly code: code for DSP stored in program memory

Dual Ported Memory: Data Memory accessible by two processors at the same time

CODEC: Compressor/Decompressor of the data.

User Interface Process: The part of the RTOS that runs the software for User Interface activities.

Controller Process: The part of the RTOS that runs the software for Timing and Control amongst the various units of the embedded system.

Embedded Systems – Processors:

Processor is the heart of an embedded system. It is the basic unit that takes inputs and produces an output after processing the data. For an embedded system designer, it is necessary to have the knowledge of both microprocessors and microcontrollers.

A processor has two essential units –

- Program Flow Control Unit (CU)
- Execution Unit (EU)

The CU includes a fetch unit for fetching instructions from the memory. The EU has circuits that implement the instructions pertaining to data transfer operation and data conversion from one form to another.

The EU includes the Arithmetic and Logical Unit (ALU) and also the circuits that execute instructions for a program control task such as interrupt, or jump to another set of instructions.

A processor runs the cycles of fetch and executes the instructions in the same sequence as they are fetched from memory.

Types of Processors:

Processors can be of the following categories –

General Purpose Processor (GPP): GPP is used for processing signal from input to output by controlling the operation of system bus, address bus and data bus inside an embedded system. It provides the hardware circuit for memory management i.e. supports the on-chip DMA and Cache. It consists the common circuitry for computations of arithmetic as well as logical operations used in daily life i.e. it includes the powerful ALU. It use the large instruction set and use the pipeline structure for instruction execution to speed up computer. Types of general purpose processor are:

- ✓ Microprocessor
- ✓ Microcontroller
- ✓ Embedded Processor
- ✓ Digital Signal Processor

Microprocessor

A microprocessor is a single VLSI chip having a CPU. In addition, it may also have other units such as caches, floating point processing arithmetic unit, and pipelining units that help in faster processing of instructions.

Earlier generation microprocessors' fetch-and-execute cycle was guided by a clock frequency of order of ~1 MHz Processors now operate at a clock frequency of 2GHz. Some of the examples are: Intel 8085/8086, 80186, 80286, Motorola 6800, 6809, G3, G4, G5 etc.

Microcontroller

A microcontroller is a single-chip VLSI unit (also called microcomputer) which, although having limited computational capabilities, possesses enhanced input/output capability and a number of on-chip functional units.

CPU	RAM	ROM
I/O Port	Timer	Serial COM Port

Microcontrollers are particularly used in embedded systems for real-time control applications with on-chip program memory and devices. Some of the examples are: Intel 8032, 8051, 8052, AVR ATMEGA 328 etc.

Embedded Processor

An Embedded Processor is a microprocessor that is used in an embedded system. These processors are usually smaller, use a surface mount form factor and consume less power. Embedded processors can be divided into two categories: ordinary microprocessors and microcontrollers. Microcontrollers have more peripherals on the chip. In essence, an embedded processor is a CPU chip used in a system which is not a general-purpose workstation, laptop or desktop computer. For example: ARM 7/9/11, CORETX-M Intel i960 etc.

Digital Signal Processor

A digital signal processor (DSP) is an integrated circuit designed for high-speed data manipulations, and is used in audio, communications, image manipulation, and other data acquisition and data-control applications. For example: PAC, TMS320XX series, Zed-broad etc.

Application Specific System Processor (ASSP): ASSP is application dependent system processor used for processing signal of embedded system. Therefore for different application performing task a unique set of system processors is required.

Application Specific Instruction Processors (ASIPs): ASIP is application dependent instruction processors. It is used for processing the various instruction set inside a combinational circuit of an embedded system.

Design Issues on Embedded System:

The constraints in the embedded systems design are imposed by external as well as internal specifications. Design metrics are introduced to measure the cost function taking into account the technical as well as economic considerations.

Design Metrics on Embedded System:

A Design Metric is a measurable feature of the system's performance, cost, time for implementation and safety etc. Most of these are conflicting requirements i.e. optimizing one shall not optimize the other: e.g. a cheaper processor may have a lousy performance as far as speed and throughput is concerned. Following metrics are generally taken into account while designing embedded systems

NRE cost (nonrecurring engineering cost):

It is one-time cost of designing the system. Once the system is designed, any number of units can be manufactured without incurring any additional design cost; hence the term nonrecurring.

Suppose three technologies are available for use in a particular product. Assume that implementing the product using technology 'A' would result in an NRE cost of \$2,000 and unit cost of \$100, that technology B would have an NRE cost of \$30,000 and unit cost of \$30, and that technology C would have an NRE cost of \$100,000 and unit cost of \$2. Ignoring all other design metrics, like time-to-market, the best technology choice will depend on the number of units we plan to produce.

Unit Cost:

The monetary cost of manufacturing each copy of the system, excluding NRE cost.

Size: The physical space required by the system, often measured in bytes for software, and gates or transistors for hardware.

Performance:

The execution time of the system

Power Consumption:

It is the amount of power consumed by the system, which may determine the lifetime of a battery, or the cooling requirements of the IC, since more power means more heat.

Flexibility:

The ability to change the functionality of the system without incurring heavy NRE cost. Software is typically considered very flexible.

Time-to-prototype:

The time needed to build a working version of the system, which may be bigger or more expensive than the final system implementation, but it can be used to verify the system's usefulness and correctness and to refine the system's functionality.

Time-to-market:

The time required to develop a system to the point that it can be released and sold to customers. The main contributors are design time, manufacturing time, and testing time. This metric has become especially demanding in recent years. Introducing an embedded system to the marketplace early can make a big difference in the system's profitability.

Maintainability:

It is the ability to modify the system after its initial release, especially by designers who did not originally design the system.

Correctness:

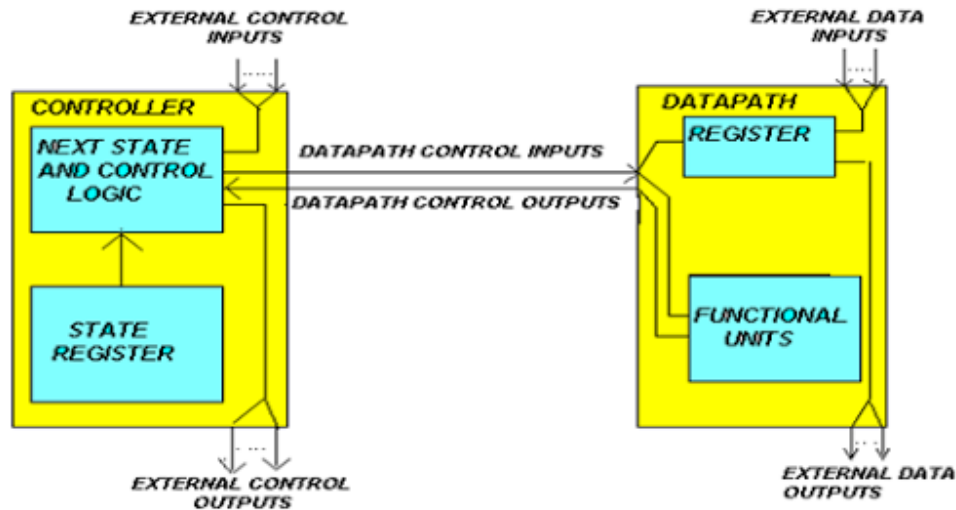
This is the measure of the confidence that we have implemented the system's functionality correctly. We can check the functionality throughout the process of designing the system, and we can insert test circuitry to check that manufacturing was correct.

Single Purpose Processor:-

A single purpose processor is a digital; circuit designed to execute exactly one program. An embedded system designer may obtain several benefits by choosing to use a custom single purpose processor to implement a computation task.

A basic processor consists of a controller and a data path. The data path stores and manipulates a system's data. The data path contains registers units, functional units and connection like wires and multiplexers. The data path can be configured to read data from particular registers feed that data through functional units configured to carry out particular operations like add or shift and store the operation results back in to the particular registers. Controller carries out such configuration of the data path. It sets the data path control inputs, like register load and multiplexer select signals, of the registers units, functional units and connection units to obtain the desired configuration at a particular time.

It monitors external control inputs as well as data path control outputs, known as status signals, coming from functional units, and it sets external control outputs as well. The digital systems design techniques such as combinational and sequential logic design including those of synchronous and asynchronous design can be applied to build a CONTROLLER and a DATA PATH.



Benefits of Custom Single Purpose Processor:

- Performance may be faster, due to fewer clock cycles resulting from a customized data path and due to shorter clock cycles resulting from the simpler controller logic.
- Size may be smaller due to simplest data path and no program memory.
- Power consumption may be less due to more efficient computation.

However, cost could be higher because of high NRE cost. Also time to market may be longer.

Embedded Systems Applications:

Embedded systems have different applications. A few select applications of embedded systems are smart cards, telecommunications, satellites, missiles, digital consumer electronics, computer networking, etc.

- ✓ Embedded Systems in Automobiles
 - Motor Control System
 - Cruise Control System
 - Engine or Body Safety
 - Robotics in Assembly Line
 - Car Entertainment
 - Car multimedia
 - Mobile and E-Com Access
- ✓ Embedded systems in Telecommunications
 - Mobile computing
 - Networking
 - Wireless Communications
- ✓ Embedded Systems in Smart Cards
 - Banking

- Telephone
- Security Systems
- ✓ Embedded Systems in Missiles and Satellites
 - Defense
 - Aerospace
 - Communication
- ✓ Embedded Systems in Computer Networking & Peripherals
 - Networking Systems
 - Image Processing
 - Printers
 - Networks Cards
 - Monitors and Displays
- ✓ Embedded Systems in Digital Consumer Electronics
 - DVDs
 - Set top Boxes
 - High Definition TVs
 - Digital Camera

Chapter -2

Hardware and Software Design Issues

A processor is a digital circuit designed to perform computation task. A processor consists of data path capable of storing and manipulating data and a controller is capable of moving data through data path. A general purpose processor is designed such that it can carry out a wide variety of computation task, which are described by a set of instructions.

In contrast, a single purpose processor is designed specifically to carry out a part task. While some tasks are so common that we can purchase standard single implement those tasks, other is best implemented using custom single processor to implement those task.

It is possible that certain subsystems in hardware (microcontroller), IO memory accesses, real-time clock, system clock, pulse width modulation, timer and serial communication are also implemented by the software. A serial communication real-time clock and timers featuring microcontroller may cost more than the microprocessor with external memory and a software implementation. Hardware implementations provide advantage of processing speed. Two approaches for the embedded two approaches for the embedded system design are follows as below.

- When the software development cycle ends then the cycle begins for the process of integrating the software into the hardware at the time when a system is designed.
- Both cycles concurrently proceed when co-designing a time critical sophisticated system.

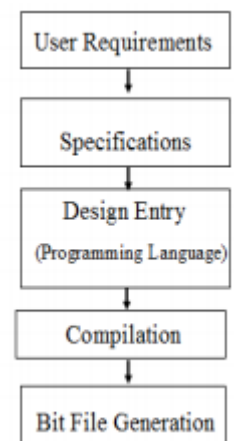
Software Design Methodology:

In general the design of a system with software based needs a design entry as programming language like c, c++ etc. The design flow is shown in Fig... In this methodology the constraints are with respect to area but not speed. The system design information can be specified by using user requirements to the system designer. The user requirements are verified according to the target board (like processor), area, speed and other nonfunctional constraints. If the user requirements satisfy existing elements then the design will move forward, otherwise user has to refine the requirements. General programming languages like c, c++ etc. can be used as the design entry and compilation will be done to check the functionality is with respect to specifications or not. Finally hex file or bit file is generated according to the target device.

The advantage of this design process is easy to design and implement with less area but timing issues and speed cannot be improved. The timing issues and speed can be considered by using the hardware based design methodology.

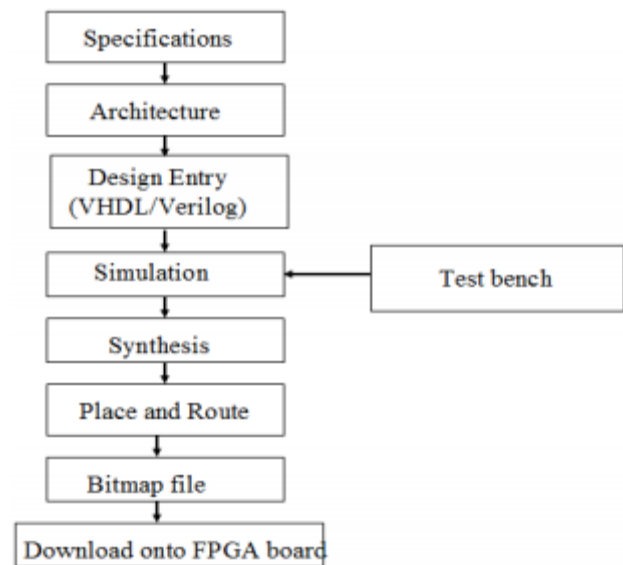
Hardware Design Methodology:

The systems with hardware components considering timing can be designed by using a programmable devices like PLA, PAL, PGA, FPGA or a non-programmable device like ASIC. The systems behavior can be expressed by using a Hardware Description Language like Verilog HDL, VHDL etc. instead of a programming language. These HDL languages describe the timing behavior of the hardware elements. The design flow for an FPGA is shown in Fig.



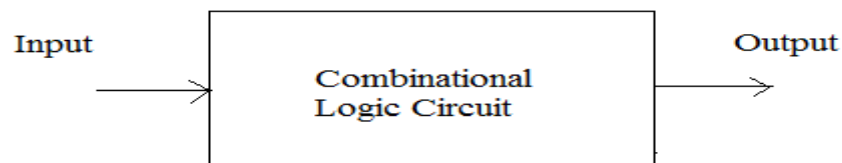
The system design starts with the refined specifications. The specifications are converted into architecture. The architecture functionality is expressed by using Hardware Description Language (HDL) like VHDL or Verilog HDL. The functionality can be verified by using simulation with the help of test bench. After the functional verification the design can be converted into gate level structural interconnected form called as net list with the help of synthesis. The synthesized net list will be placed and routed on the selected target FPGA board. A bit file will be generated to load the designed system on to the target FPGA.

The advantage of hardware based design methodology is the timing information can be specified by using HDL and high speed systems can be designed compared to software based methods.

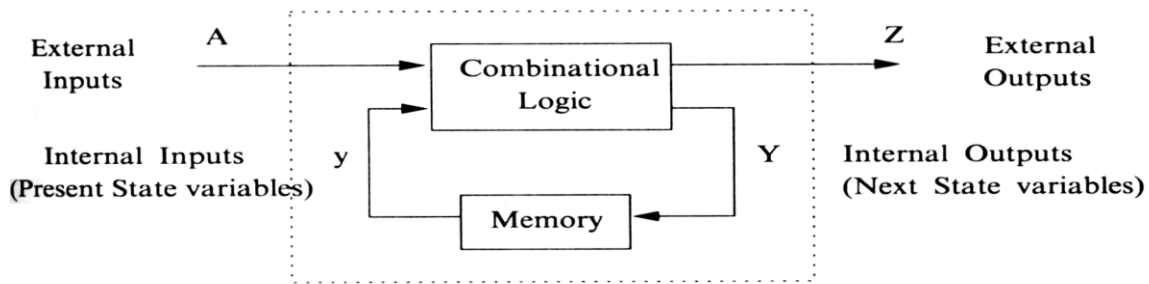


Combinational and Sequential Logic:

Combinational logic refers to circuits whose output is a function of the present value of the inputs only. As soon as inputs are changed, the information about the previous inputs is lost, that is, combinational logic circuits have no memory.



Sequential logic circuits are those whose outputs are also dependent upon past inputs, and hence outputs. In other words the output of a sequential circuit may depend upon its previous outputs and so in effect has some form of "memory". The mathematical model of a sequential circuit is usually referred to as a sequential machine. The general block diagram of a sequential switching circuit is shown below:



Sequential circuits are basically combinational circuits with the additional properties of storage (to remember past inputs) and feedback.

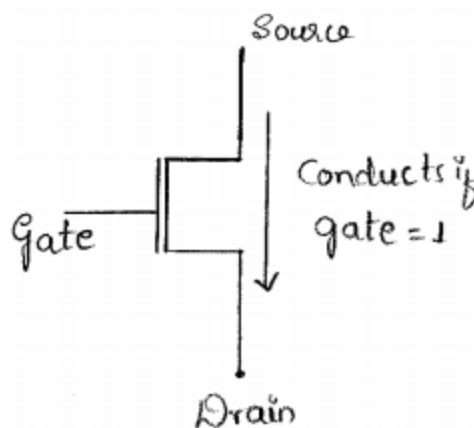
Transistor:

A transistor is a basic electrical component in digital system. It acts as a simple ON/OFF switch. Transistor are abstracted to construct the logic gates in higher level. One type of transistor is complementary metal oxide (CMOS) which is more popular in combinational circuit design and corresponding technology is called CMOS technology. CMOS transistor are two types:

1. nMOS Transistor
2. pMOS Transistor

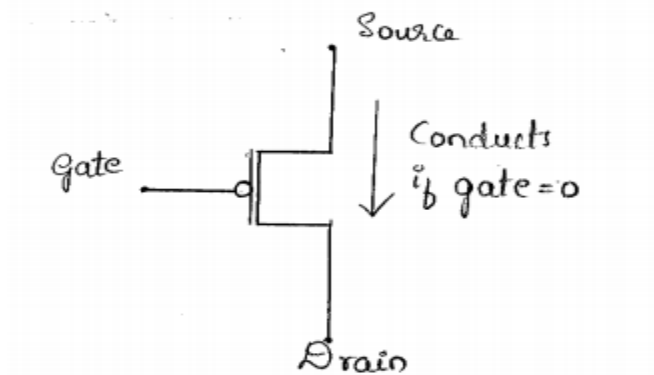
We can apply low or high levels to the gate of the CMOS transistor. We refers logical levels i.e. logic 0 is 0V and logic 1 is 5V.

nMOS Transistor:



- When logic 1 is applied to gate, the transistor conducts and current flows from source to drain.
- When logic 0 is applied to gate, the transistor do not conducts and high resistance is developed about $10\text{ M}\Omega$ along source to drain.

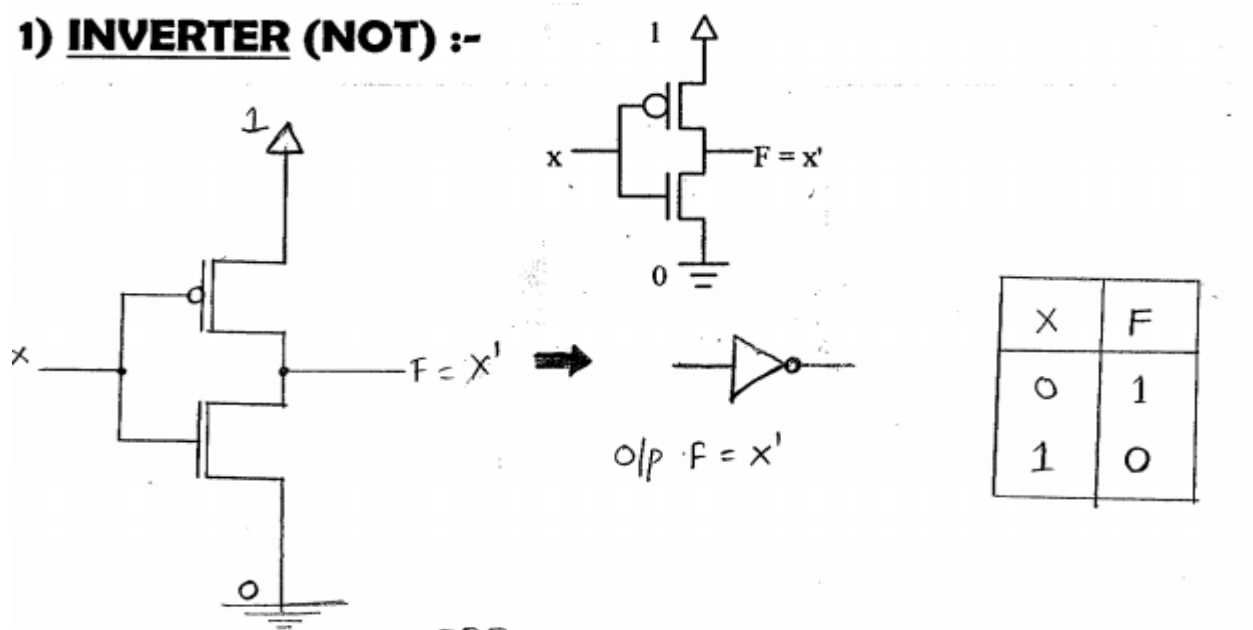
pMOS Transistor:



- When logic 0 is applied to gate, the transistor conducts and current flows from source to drain. When logic 1 is applied to gate, the transistor does not conduct and high resistance is developed about $10\text{ M}\Omega$ along source to drain

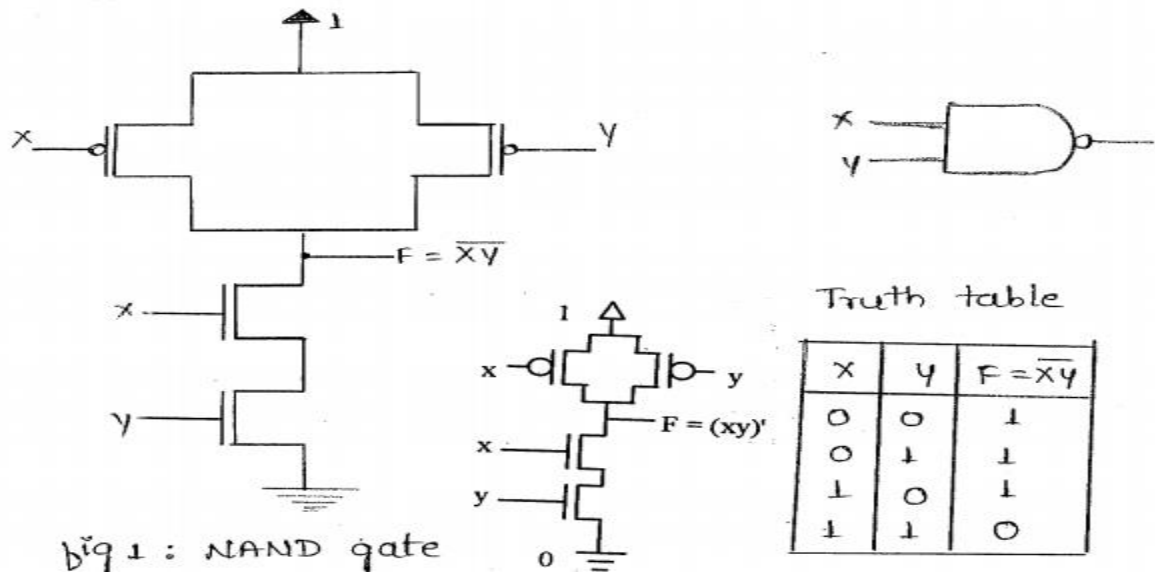
Implementations of Logic gates using CMOS

1) INVERTER (NOT) :-



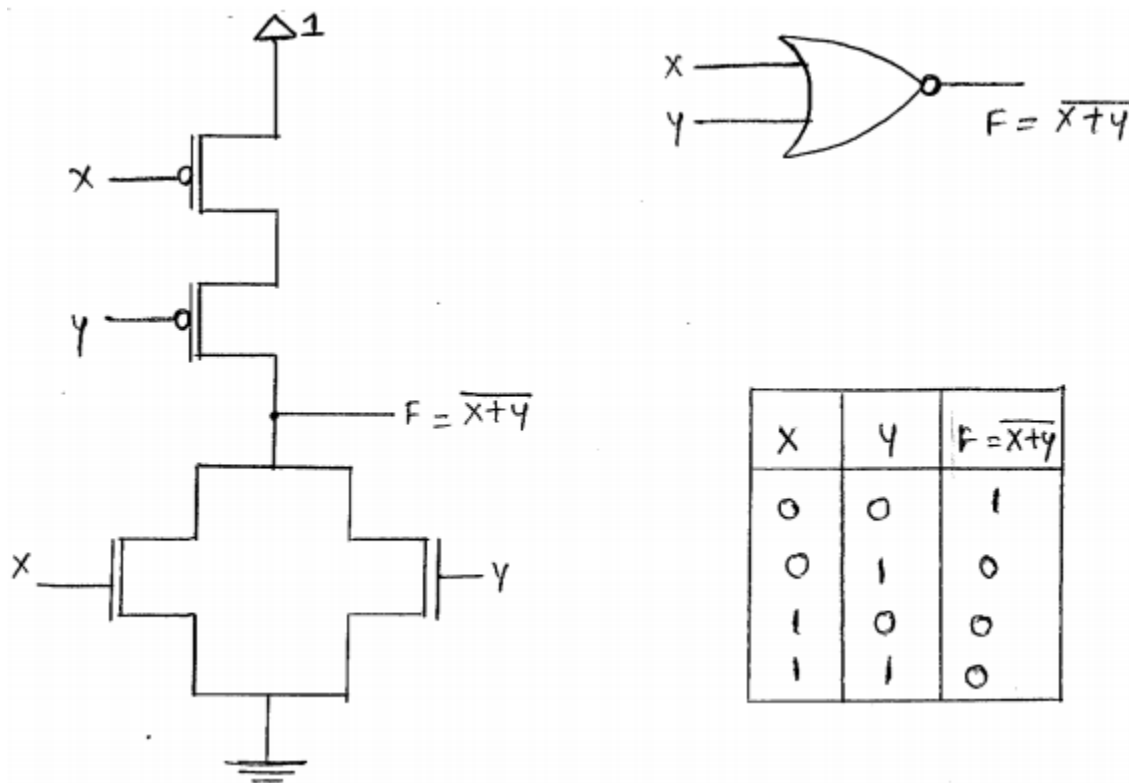
- When the input "x" is logic 0, the upper transistor conducts and lower transistor does not conduct. Thus logic 1 appears as output "F".
- Similarly when the input "x" is logic 1, the upper transistor does not conduct and lower transistor conducts. Thus logic 0 appears as output "F".

NAND Gate:



- When one of the input “x” or “y” is logic 0 then at least one of the upper transistor and lower conducts. Thus logic 1 is appears as output “F”.
- When both input “x” or “y” is logic 1 then neither of the upper transistor conducts but both lower conducts. Thus logic 0 is appears as output “F”.

NOR Gate

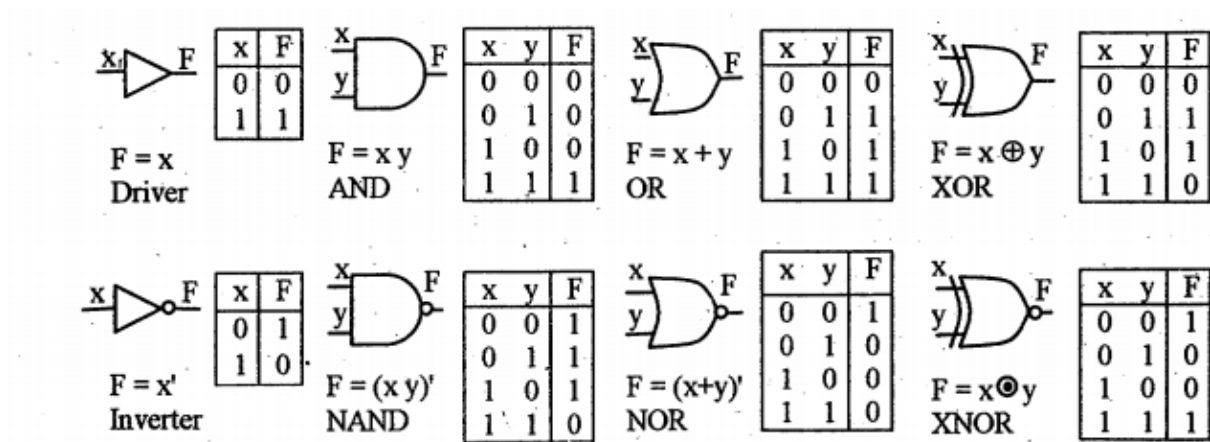


- When both input “x” or “y” is logic 0 then both of the upper transistor conducts but both lower does not conducts .Thus logic 1 is appears as output “F”.
- When both input “x” or “y” is logic 1 then both of the upper transistor does conducts but both lower does conducts .Thus logic 0 is appears as output “F”.
- When one of the input “x” or “y” is logic 0 then at least one of the upper transistor and lower conducts .Thus logic 0 is appears as output “F”.

Logic Gates:

The Digital Logic Gate is the basic building block from which all digital electronic circuits and microprocessor based systems are constructed from. Basic digital logic gates perform logical operations of AND, OR and NOT on binary numbers.

Digital logic gates may have more than one input, (X, Y, Z, etc.) but generally only have one digital output, (F). Individual logic gates can be connected together to form combinational or sequential circuits, or larger logic gate functions.

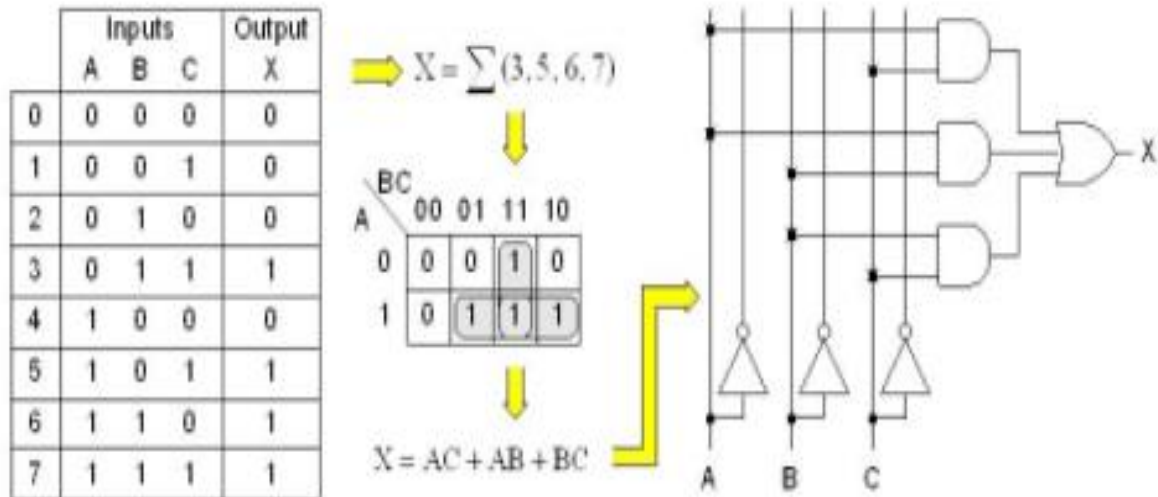


Basic Combinational Logic Design:

Combinational logic refers to circuits whose output is a function of the present value of the inputs only. Combinational logic circuits have no memory. The design procedure includes following steps:

- Determine the no of input and output of system from the problem specifications.
- Derive truth table for each output from their relationship with number of inputs.
- Simplify the Boolean expression using K-maps & obtain logic equations.
- Draw logic diagram (sharing common gates).
- Simulate circuit for design verification.
- Optimize the circuit for area and/or performance using different optimization metric.
- Re-simulate & verify optimized design.

For example: a combinational circuit that have three input say A, B and C and will give a single output X as logic 1 if the binary number consists more 1's than 0's.



RT-level Combinational Component:

All digital circuit can be designed as the procedure stated as above but it is difficult and complex so higher abstract level of combinational logic devices are used in register transfer level design. Some of the component are:

1. Multiplexer:

A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.

A –a

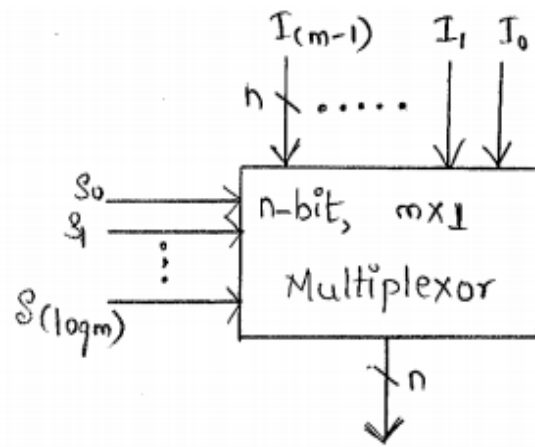


fig 1: Multiplexor

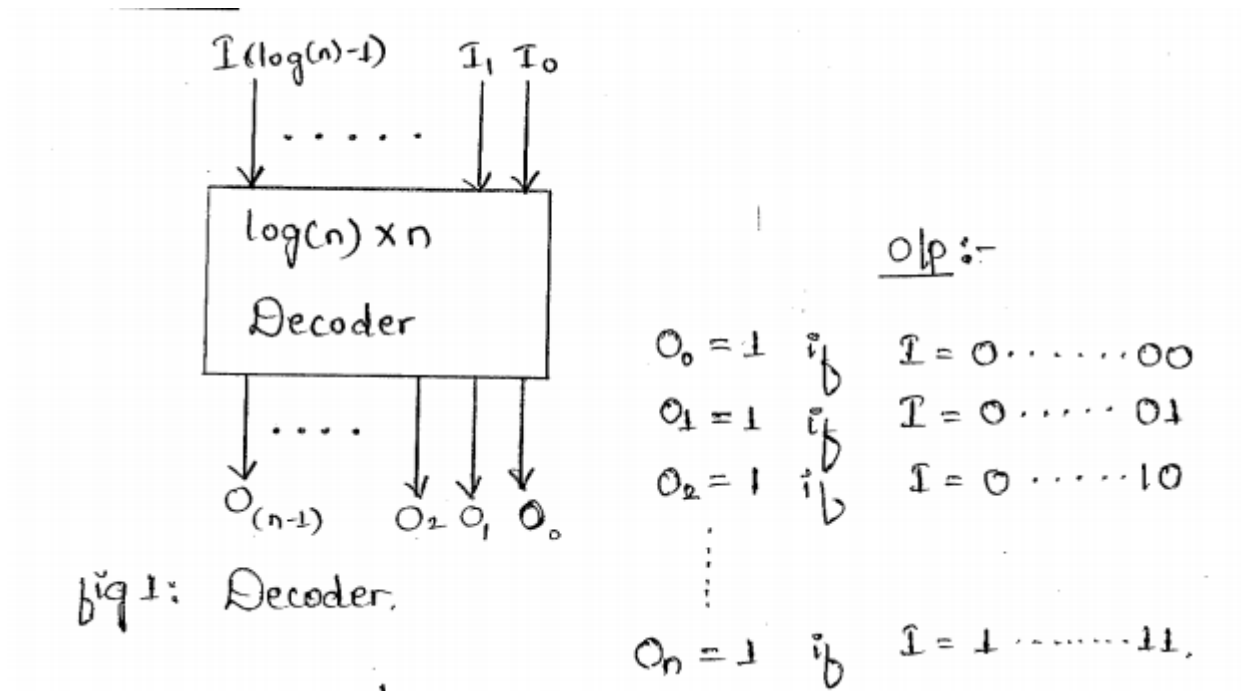
o/p :

$$\begin{aligned}
 0 &= I_0 \text{ if } S = 0 \dots 00 \\
 &= I_1 \text{ if } S = 0 \dots 01 \\
 &\vdots \\
 &= I_{(m-1)} \text{ if } S = 1 \dots 11
 \end{aligned}$$

- A multiplexer sometimes called as a selector, always only one its data input I_m passed through the o/p. Thus multiplexer acts much like a railroad switch, allowing multiple input tracks to a single track as output.
- If there are “m” data inputs then it consists $\log_2(m)$ selection inputs lines “S”. We call this an m-by-1 MUX.
- The binary inputs to “S” determines which input line is connected to O/P.
 - 0000.....0000 means the input I_0 passed through
 - 0000.....0001 means the input I_1 passed through
 - 0000.....0010 means the input I_2 passed through
 - And so on.....

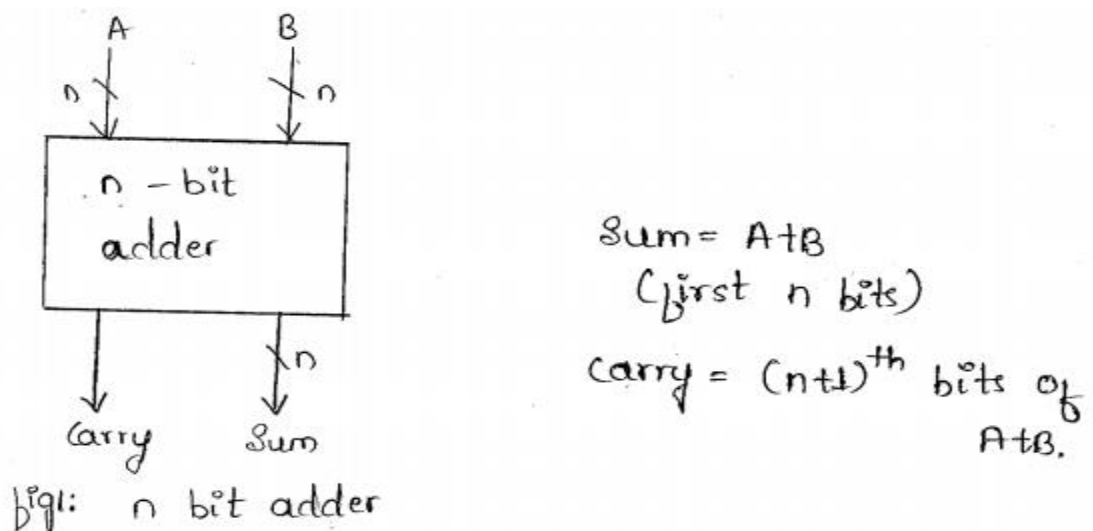
2. Decoder

Discrete quantities of information are represented in digital systems with binary codes. A binary code of n bits is capable of representing up to 2^n distinct elements of the coded information. Decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.



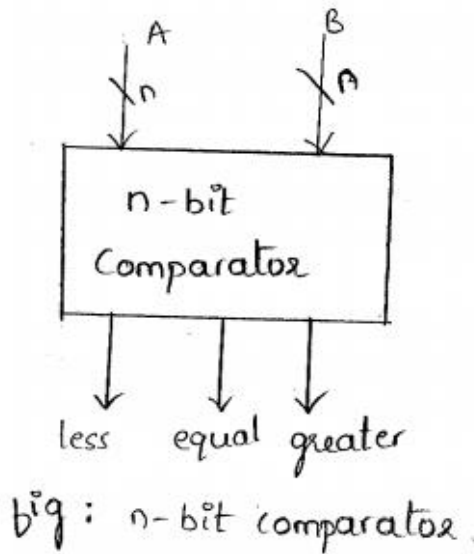
- A decoder converts the binary input combinations "I" into a one-hot output. One-hot means that exactly one output line can be 1 at a given time.
- Thus there are n -outputs, then there must be $\log_2(n)$ then it is called as $\log_2(n)$ by n decoder.

3. Adder:



- An n -bit adder adds two n -bit data say A and B at a time and generates carry and sum as an output.

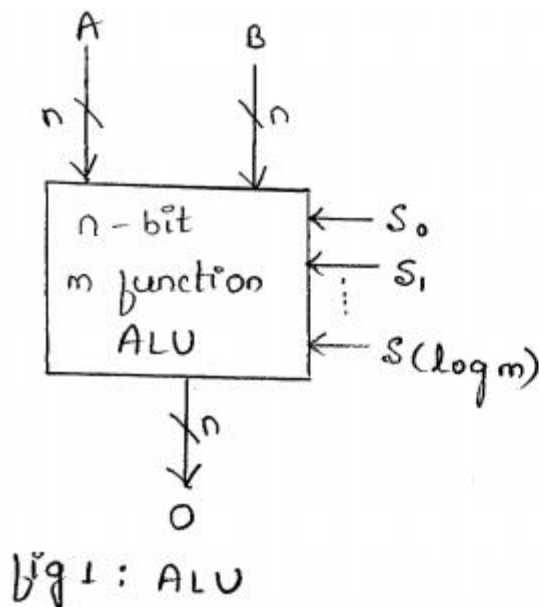
4. Comparator:



$I_b \ A < B$, then $less = 1$
 $I_b \ A = B$, then $equal = 1$
 $I_b \ A > B$ then $greater = 1$.

- An n-bit comparator compares n-bit data say A and B and generates the output as $A < B$ or $A > B$ or $A = B$.

5. Arithmetic-Logic Unit (ALU)



$O = A \text{ op } B$
 op (operation) is determined by S.

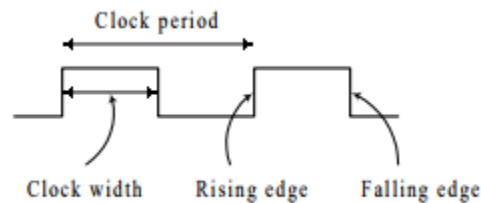
- An n-bit ALU can perform arithmetic and logical calculations on n-bit data inputs A and B.
- The selection input "S" determines the operation that is going to perform.

6. Shifter

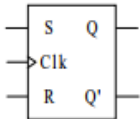
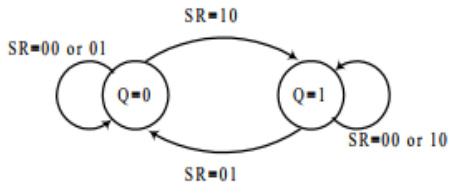
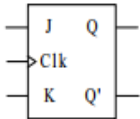
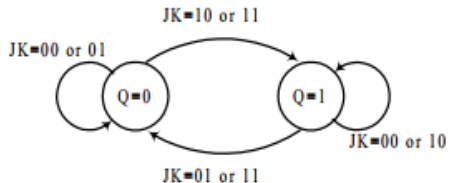
- An n-bit shifter can shift the n bit data stored on n-bit register towards right or left direction.

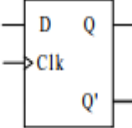
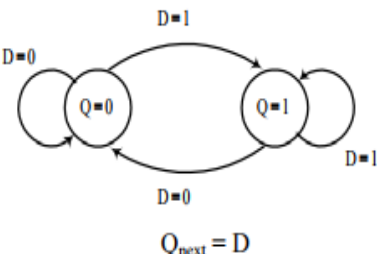
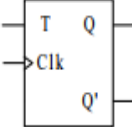
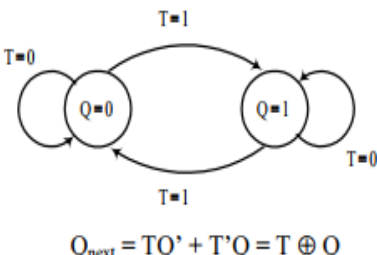
Sequential Logic Components:

Their output values are computed using both the present and past input values. In other words, their outputs depend on the sequence of input values that have occurred over a period of time. This dependence on the past input values requires the presence of memory elements. The values stored in memory elements define the state of a sequential component. Since memory is finite, therefore, the sequence size must always be finite, which means that the sequential logic can contain only a finite number of states. So sequential circuits are sometimes called finite-state machines. Sequential circuits can be asynchronous or synchronous.



Asynchronous sequential circuits change their state and output values whenever a change in input values occurs. Synchronous sequential circuits change their states and output values at fixed points of time, which are specified by the rising or falling edge of a free-running clock signal. Clock period is the time between successive transitions in the same direction, i.e., between two rising or two falling edges. Clock frequency = $1/\text{clock period}$. Clock width is the time during which the value of the clock signal is equal to 1. Duty cycle is the ratio of clock width and clock period. Active high if the state changes occur at the clock's rising edge or during the clock width. Active low if the state changes occur at the clock's falling edge. Latches and flip flops are the basic storage elements that can store one bit of information. The basic components with their properties are shown below.

Name	FF Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table																																			
SR	 The triangle indicates that the ff is triggered by the rising edge.	<table><tr><th>S</th><th>R</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>NA</td></tr></table>	S	R	Q_{next}	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q_{next} = S + R'Q$$SR = 0$	<table><tr><th>Q</th><th>Q_{next}</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q_{next}	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q_{next}																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q_{next}	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table><tr><th>J</th><th>K</th><th>Q_{next}</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Q'</td></tr></table>	J	K	Q_{next}	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q_{next} = J'K'Q + JK' + JKQ'$$= J'K'Q + JK'Q + JK'Q' + JKQ'$$= K'Q(J' + J) + JQ'(K' + K)$$= K'Q + JQ'$	<table><tr><th>Q</th><th>Q_{next}</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>X</td></tr><tr><td>1</td><td>0</td><td>X</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q_{next}	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q_{next}																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q_{next}	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				

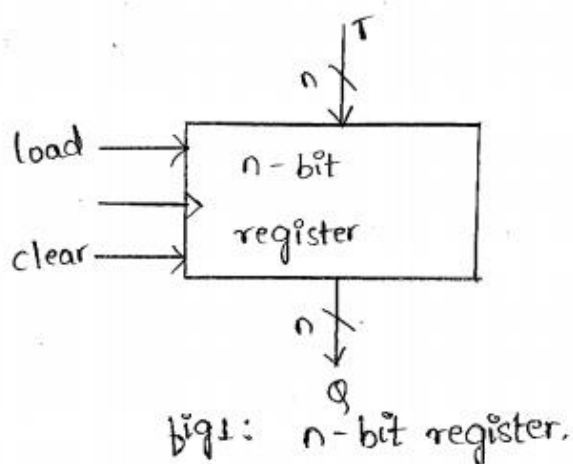
D		$\begin{array}{c c} D & Q_{next} \\ \hline 0 & 0 \\ 1 & 1 \end{array}$		$\begin{array}{c c c} Q & Q_{next} & D \\ \hline 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{array}$
T		$\begin{array}{c c} T & Q_{next} \\ \hline 0 & Q \\ 1 & Q' \end{array}$		$\begin{array}{c c c} Q & Q_{next} & T \\ \hline 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{array}$

The **characteristic table** is a shorter version of the truth table that gives for every set of input values and the state of the flip-flop before the rising edge, the corresponding state of the flip-flop after the rising edge of the clock. It is used during the analysis of sequential circuits.

The **characteristic equation** is just the functional expressions derived from the characteristic (truth) table. It formally describes the functional behavior of a latch or flip-flop. They specify the flip-flop's next state as a function of its current state and inputs.

The **excitation table** gives the value of the flip-flop inputs that are necessary to change the flip-flop's present state to the desired next state after the rising edge of the clock signal. It is obtained from the characteristic table by transposing input and output columns. It is used during the synthesis of sequential circuits.

RT-level Sequential Component:

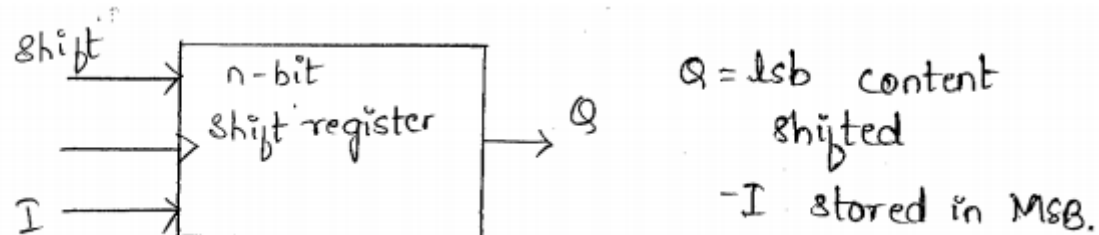


$Q = 0$ if $clear = 1$
 $Q = 1$ if $load = 1$ & $clock = 1$
 $Q = \text{otherwise}$

- A register stores the n-bits from its input "I" with those n bits appearing at output line "Q".

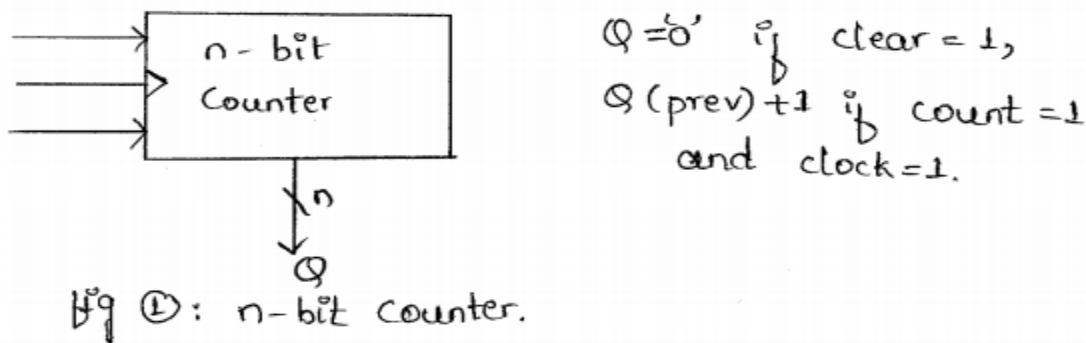
- A register usually has at least two control input, clock and load. For a raising edge triggered register, input is usually drawn as a small triangle as shown in fig.
- Another common control unit is clear, which resets all bits to value "0" regardless the value of "I".
- All input bits are loaded in parallel so it is called as parallel load register.

Shift Register:



- A shift register stores n bit but they cannot be stored in parallel. Instead they must be shifted into the register i.e. one bit at a single clock.
- The register have at least a data input "I" that holds a single bit at a time and control input shift that is used to insert a data.
- When clock is raising mode the shift equals 1 such that the data bits in "I" is inserted into "n" bit position of register while n^{th} bit is inserted into $(n-1)^{\text{th}}$ bits and $(n-1)$ bit into $(n-2)$ and so on.
- The first bit is usually shifted to the output end "Q".

Counter:



- A counter is a register that can be also increment, meaning add the binary value 1, to its stored binary value. A counter has a control input clear that resets all bits of register to vale "0" and count input that increments the value of stored number by 1 in each raising edge of clock.
- A counter has also a load input to load the n-bit data in parallel.
- Commonly a counter operates in both mode as down and up. The up counter increments the value stored in register by 1 and down counter decrements the contents of register by value 1 up to define level. Mode M counter counts from 0 to M-1 or M-1 to 0. For this it requires another control input as Count UP/DOWN.
- These control input may be

1. Synchronous
 2. Asynchronous
- A synchronous input value only have effect during in a clock edge.
 - An asynchronous value effects the circuit independent of clock.

Design Procedure to Sequential Circuit Design:

The design of a clocked sequential circuit starts from a set of specifications (state table) and ends in a logic diagram or a list of Boolean functions from which the logic diagram can be obtained. The procedure can be summarized by a list of consecutive recommended steps:

1. State the word description of the circuit behavior. It may be a state diagram, a timing diagram, or other pertinent information.
2. From the given information about the circuit, obtain the state table.
3. Apply state-reduction methods if the sequential circuit can be characterized by input-output relationships independent of the number of states.
4. Assign binary values to each state if the state table obtained in step 2 or 3 contains letter symbols.
5. Determine the number of flip-flops needed and assign a letter symbol to each.
6. Choose the type of flip-flop to be used.
7. From the state table, derive the circuit excitation and output tables.
8. Using the map or any other simplification method, derive the circuit output functions and the flip-flop input functions.
9. Draw the logic diagram.

Design Examples:

1. Design a combinational circuit for output y and z where y is 1 when a is 1 or b and c are 1 and z is 1 if b or c is 1 but not all i.e. a and b and c are 1.

Truth Table:

I/p's			O/p's	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

K-map Reduction:

K-map

y a \ bc	00	01	11	10
0	0	0	1	0
1	1	1	1	1

$$\begin{aligned}
 y &= bc + a\bar{b}\bar{c} + a\bar{b}c + abc + ab\bar{c} \\
 &= bc + a\bar{b}[\bar{c} + c] + ab[c + \bar{c}] \\
 &= bc + a\bar{b} + ab \\
 &= bc + a[\bar{b} + b] \\
 &= bc + a
 \end{aligned}$$

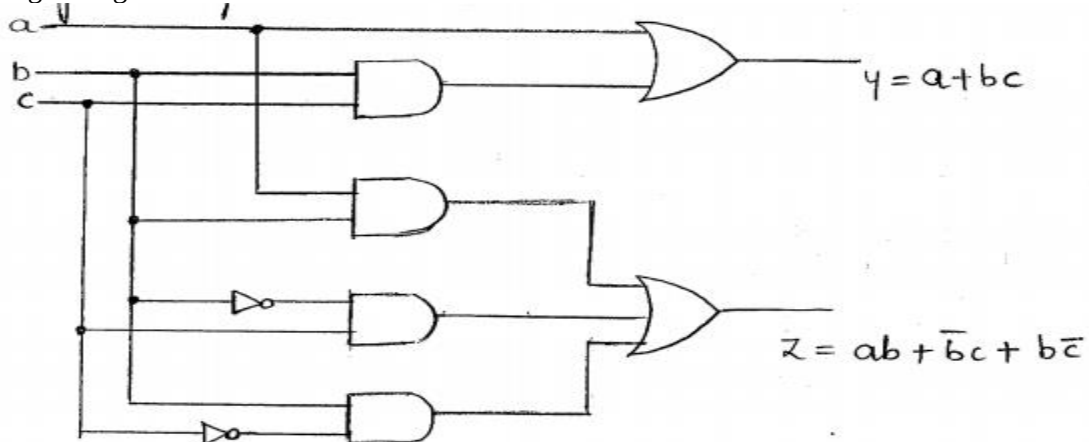
$$y = a + bc$$

K-map

z a \ bc	00	01	11	10
0	0	1	0	1
1	0	1	1	1

$$\begin{aligned}
 z &= \bar{b}c + ab + b\bar{c} \\
 z &= ab + \bar{b}c + b\bar{c}
 \end{aligned}$$

Logic Diagram:

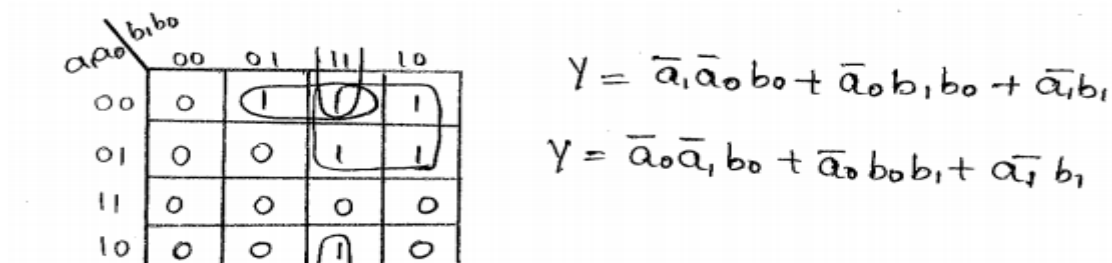


2. Design a 2 bit comparator circuit that have one single output “less than” using the approach of combinational design technique.

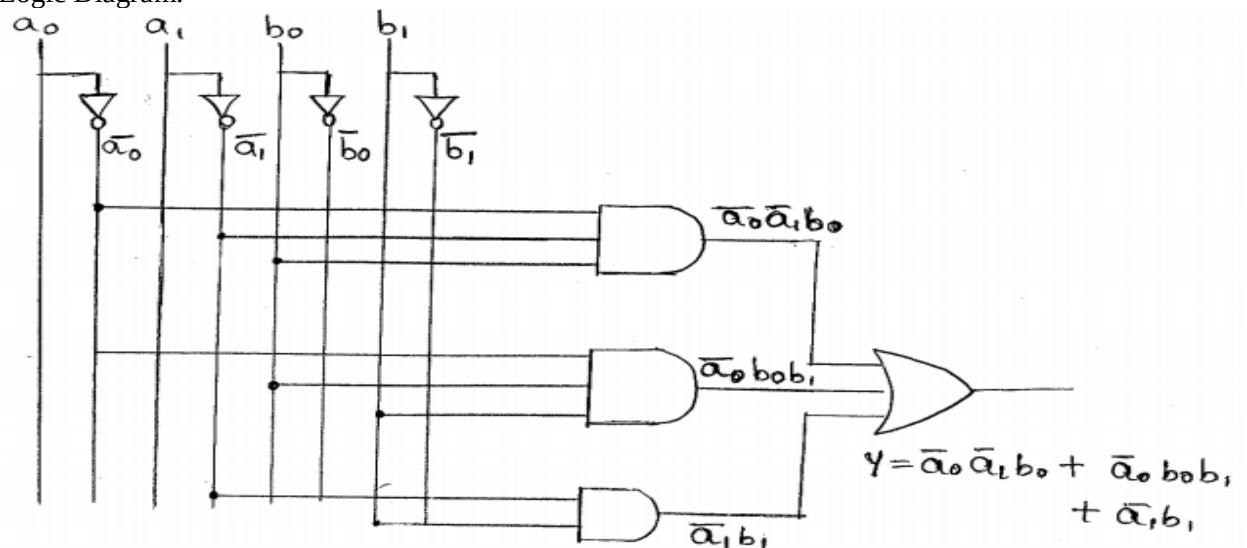
Truth Table:

a1	a0	b1	b0	lt (a < b)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

K-map Reduction:



Logic Diagram:



3. Design a 3X8 decoder. Start from truth table, use k-map to minimize the logic gates and draw the final circuit.

Soln

I/p's			O/p's							
a_2	a_1	a_0	d_7	d_6	d_5	d_4	d_3	d_2	d_1	d_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

d_0

a_2	a_1	a_0	00	01	11	10
0	1	0	1	0	0	0
1	0	0	0	0	0	0

$$d_0 = \bar{a}_2 \bar{a}_1 \bar{a}_0$$

d_1

a_2	a_1	a_0	00	01	11	10
0	0	1	0	1	0	0
1	0	0	0	0	0	0

$$d_1 = a_0 \bar{a}_1 \bar{a}_2$$

d_2

a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	0	1
1	0	0	0	0	0	0

$$d_2 = \bar{a}_0 a_1 \bar{a}_2$$

d_3

a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	1	0
1	0	0	0	0	0	0

$$d_3 = a_0 a_1 \bar{a}_2$$

d_4

a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	0	0
1	1	0	0	0	0	0

$$d_4 = \bar{a}_0 \bar{a}_1 a_2$$

d_5

a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	0	0
1	0	1	0	1	0	0

$$d_5 = a_0 \bar{a}_1 a_2$$

d_6

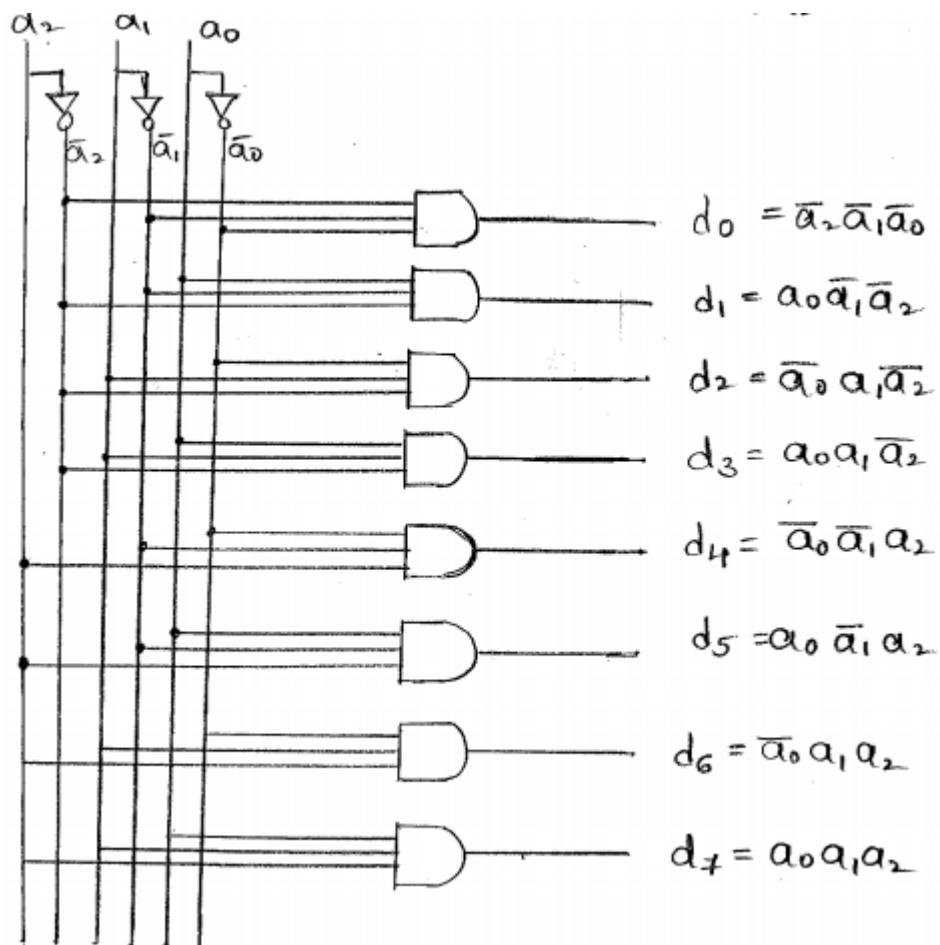
a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	0	0
1	0	0	0	0	0	1

$$d_6 = \bar{a}_0 a_1 a_2$$

d_7

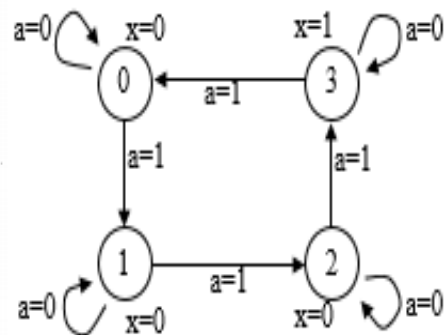
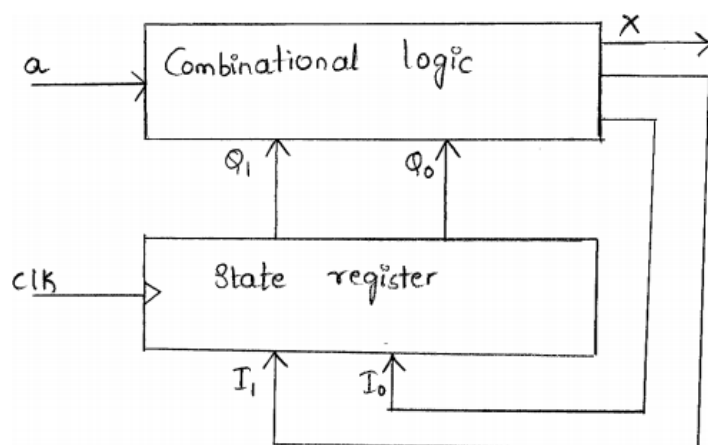
a_2	a_1	a_0	00	01	11	10
0	0	0	0	0	0	0
1	0	0	0	1	0	0

$$d_7 = a_0 a_1 a_2$$



4. Construct a clock divider. Slow down your pre-existing clock so that you output a 1 for every four clock cycles.

State diagram



State table

Inputs			Outputs		
Q1	Q0	a	I1	I0	x
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	1	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	1	1	1
1	1	1	0	0	

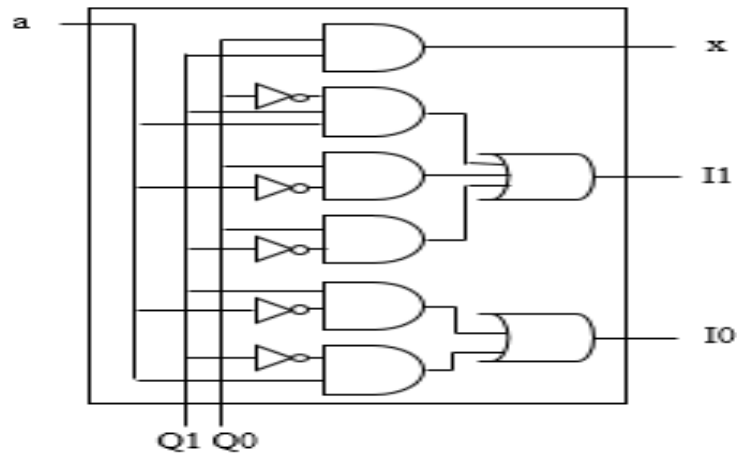
Minimized state equation:

a	Q1Q0				
	00	01	11	10	
0	0	0	1	1	$I1 = Q1'Q0a + Q1a' + Q1Q0'$
1	0	1	0	1	

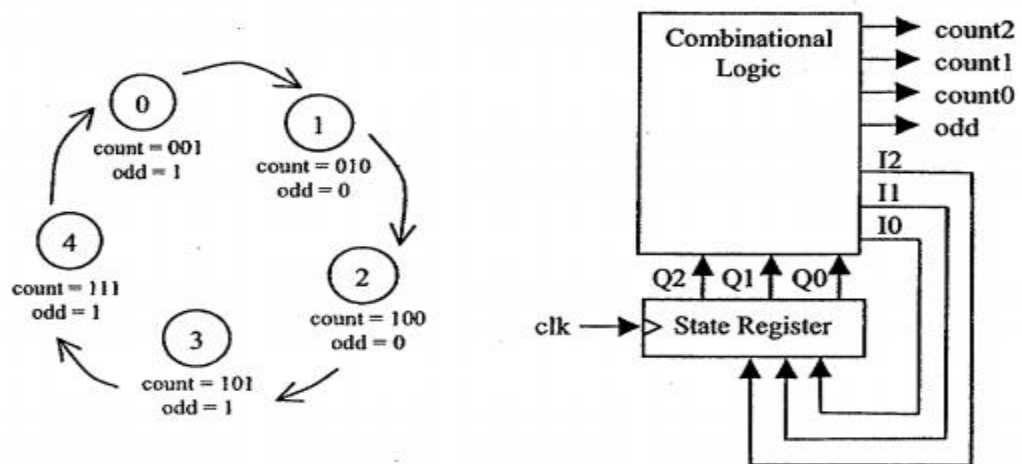
a	Q1Q0				
	00	01	11	10	
0	0	1	1	0	$I0 = Q0a' + Q0'a$
1	1	0	0	1	

a	Q1Q0				
	00	01	11	10	
0	0	0	1	0	$x = Q1Q0$
1	0	0	1	0	

Logic diagram



5. Design the 3- bit counter that counts the sequence as 1,2,3,4,5,6,7,1,2,3,...etc. . This counter has output “odd” whose value is 1 when it counts odd. Design the circuit by using sequential circuit design technique.



Inputs			Outputs						
Q2	Q1	Q0	I2	I1	I0	count2	count1	count0	add
0	0	0	0	0	1	0	0	1	1
0	0	1	0	1	0	0	1	0	0
0	1	0	0	1	1	1	0	0	0
0	1	1	1	0	0	1	0	1	1
1	0	0	0	0	0	1	1	1	1
1	0	1	x	x	x	x	x	x	x
1	1	0	x	x	x	x	x	x	x
1	1	1	x	x	x	x	x	x	x

		Q1Q0			
Q2		00	01	11	10
	0	0	0	1	0
	1	0	X	X	X

$$I2 = Q1Q0$$

		Q1Q0			
Q2		00	01	11	10
	0	0	1	0	1
	1	0	X	X	X

$$I1 = Q1'Q0 + Q1Q0'$$

		Q1Q0			
Q2		00	01	11	10
	0	1	0	0	1
	1	0	X	X	X

$$I0 = Q2'Q0'$$

		Q1Q0			
Q2		00	01	11	10
	0	0	0	1	1
	1	1	X	X	X

$$\text{count2} = Q2 + Q1$$

		Q1Q0			
Q2		00	01	11	10
	0	0	1	0	0
	1	1	X	X	X

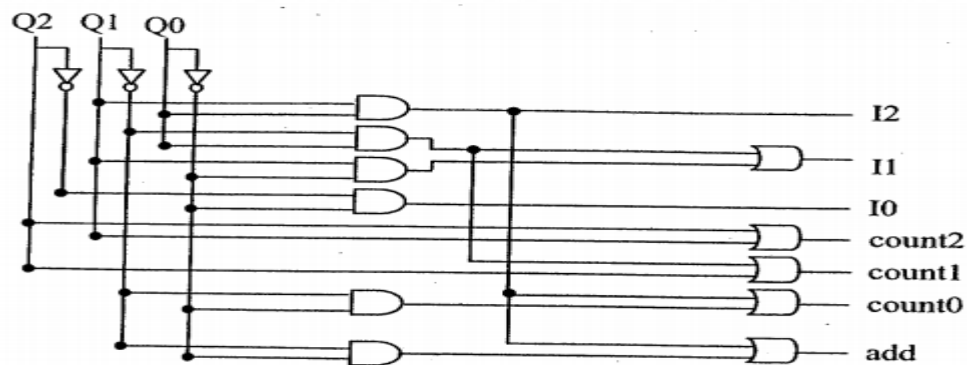
$$\text{count1} = Q2 + Q1'Q0$$

		Q1Q0			
Q2		00	01	11	10
	0	1	0	1	0
	1	1	X	X	X

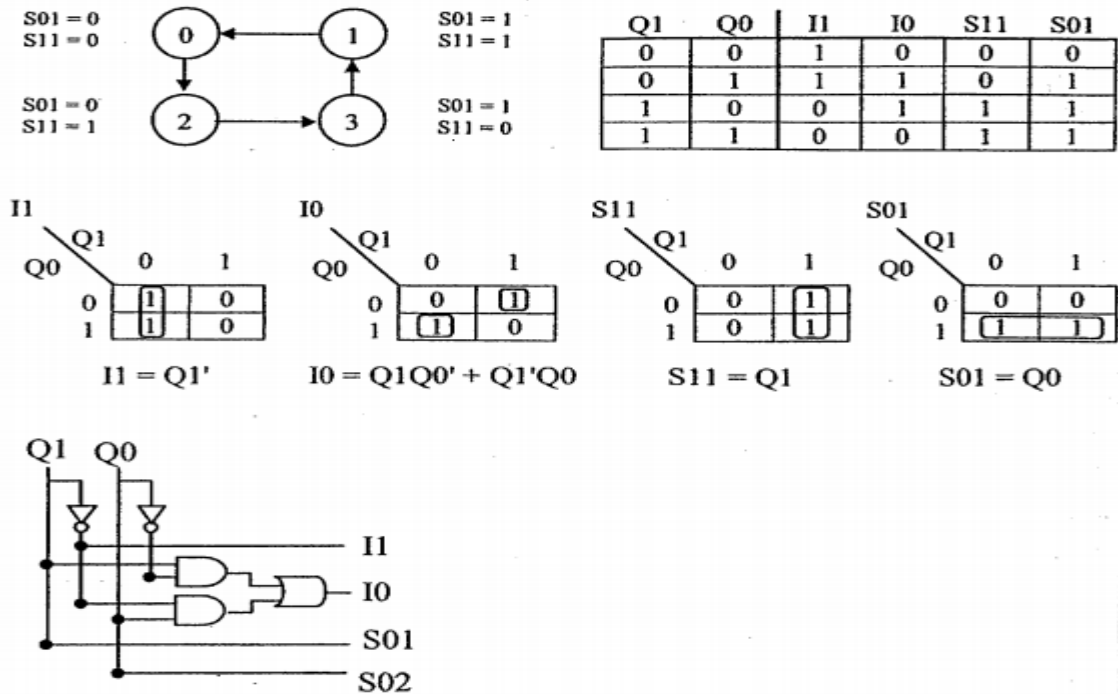
$$\text{count0} = Q1'Q0' + Q1Q0$$

		Q1Q0			
Q2		00	01	11	10
	0	1	0	1	0
	1	1	X	X	X

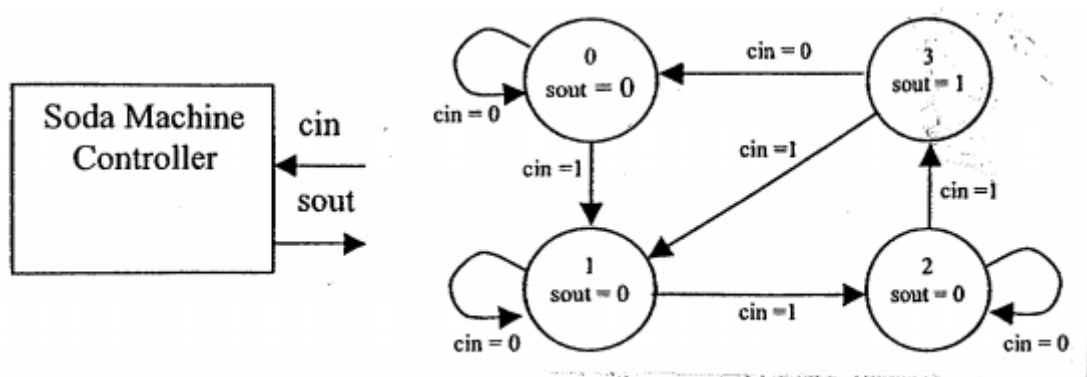
$$\text{add} = Q1'Q0' + Q1Q0$$

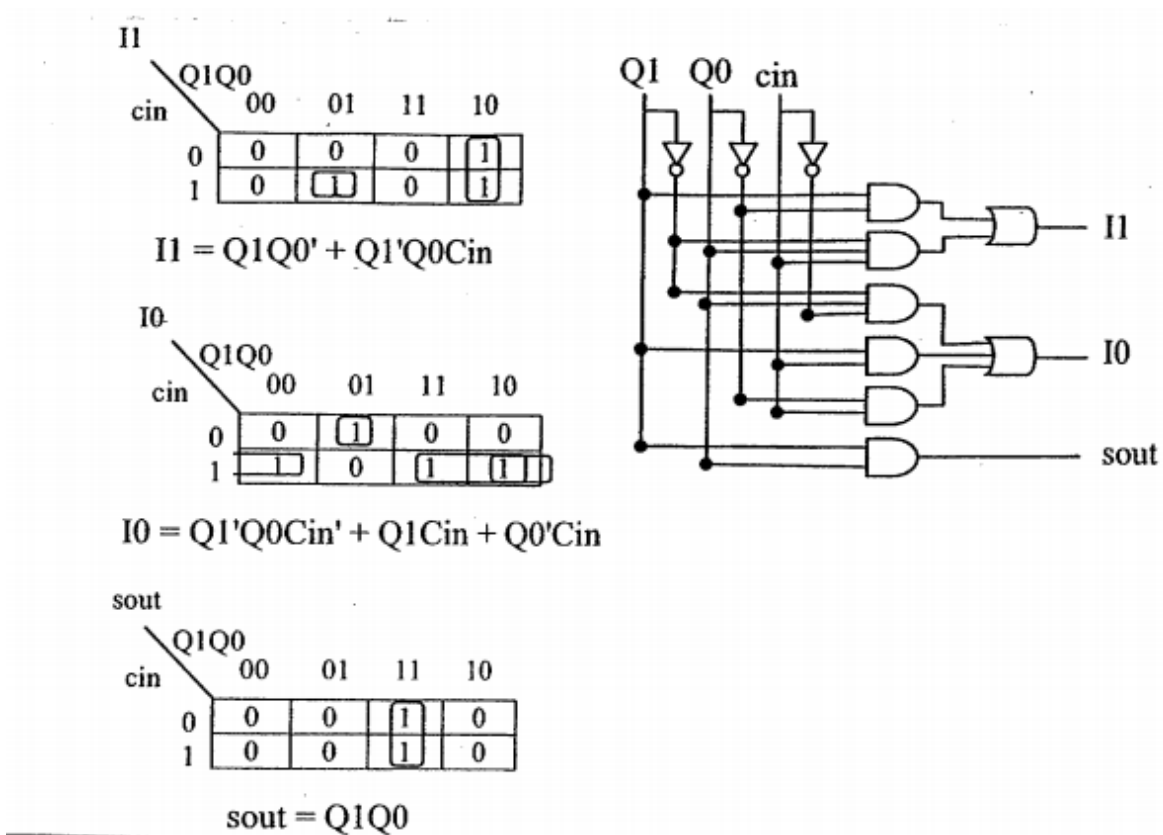


6. Four lights are connected to decoder. Build a circuit that will blink the lights in the order 0, 2, 1, 3, 0, 2..... Start from state diagram draw the final circuit.



7. Design a soda controller machine given that a soda costs 75 cents and your machine accepts quarters only. Draw the black box view, come up with state diagram, and draw the final circuit.





Sequence Recognizer (Moore)

A sequence recognizer is a special kind of sequential circuit that looks for a special bit pattern in some input. The recognizer circuit has only one input, X

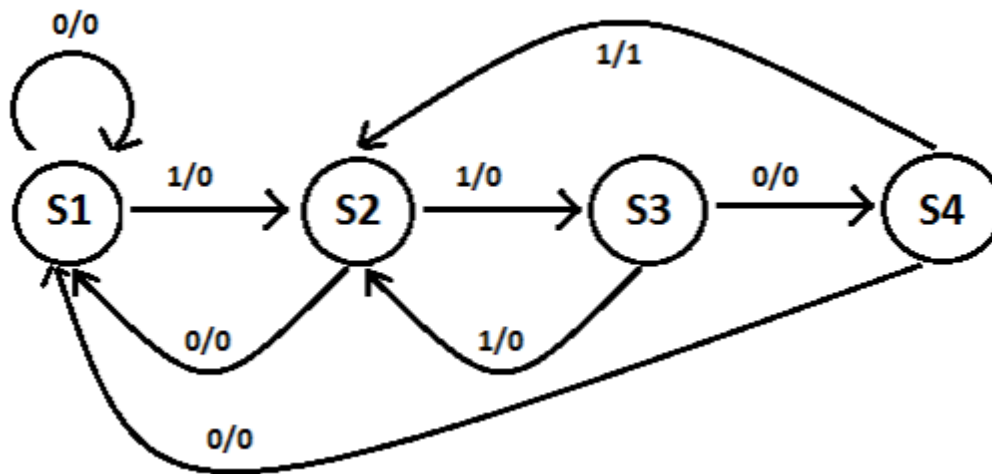
- One bit of input is supplied on every clock cycle
- This is an easy way to permit arbitrarily long input sequences

There is one output, Z, which is 1 when the desired pattern is found. Our example will detect the bit pattern —10011:

Inputs: 1 1 1 101 1 0 1 10 1 101 1 0...

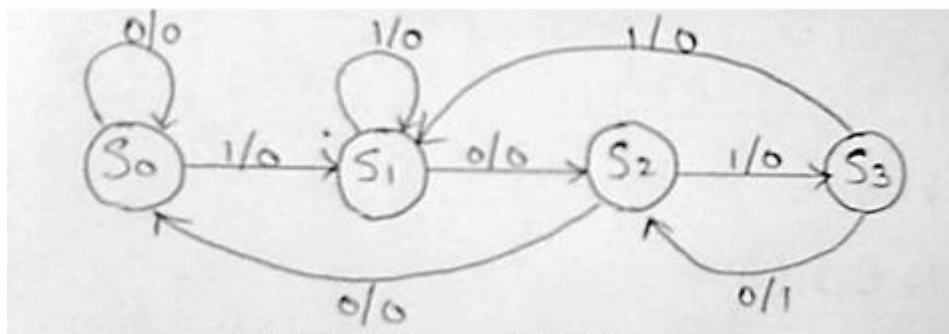
Outputs: 0 00 001 00 0 00 1 001 0 0...

A sequential circuit is required because the circuit has to —remember the inputs from previous clock cycles, in order to determine whether or not a match was found. The corresponding state diagram is:



1. Design a sequence detector to detect a sequence ----1010---- using D flip-flops and logic gates.

State Diagram



The state table for the above diagram:

Present state	Next state		Output	
	X=0	X=1	X=0	X=1
S ₀	S ₀	S ₁	0	0
S ₁	S ₂	S ₁	0	0
S ₂	S ₀	S ₃	0	0
S ₃	S ₂	S ₁	1	1

State assignments: Let S₀S₀ = 00, S₁S₁ = 01, S₂S₂ = 10, S₃S₃ = 11

The above state table becomes:

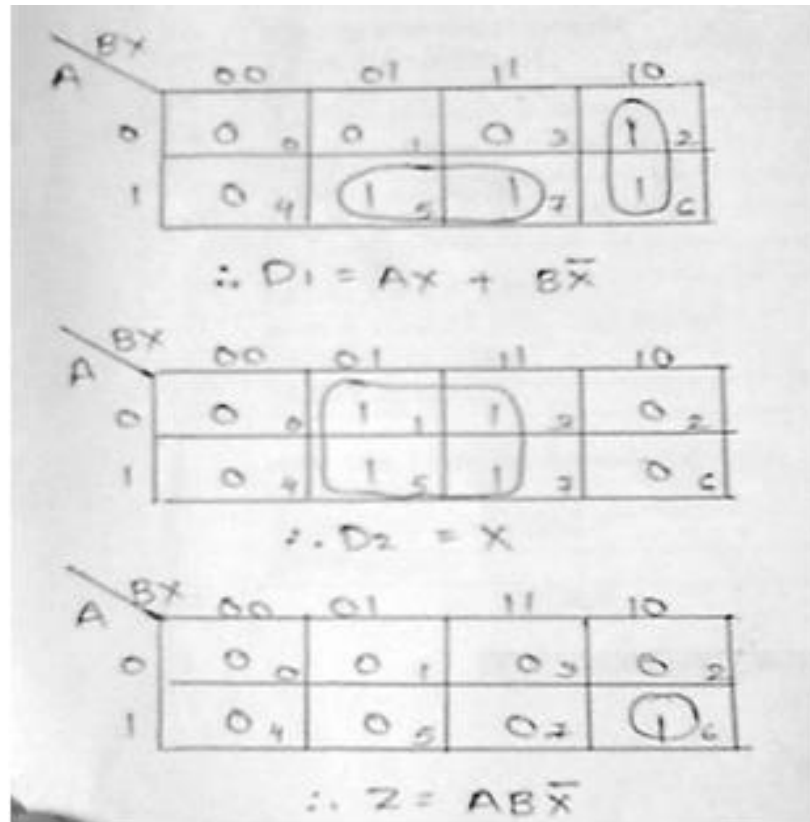
Present state	Next state		Output	
	X=0	X=1	X=0	X=1
00	00	01	0	0
01	10	01	0	0
10	00	11	0	0
11	10	11	1	0

Four states will require two flip flops. Consider two D flip flops. Their excitation table is shown below.

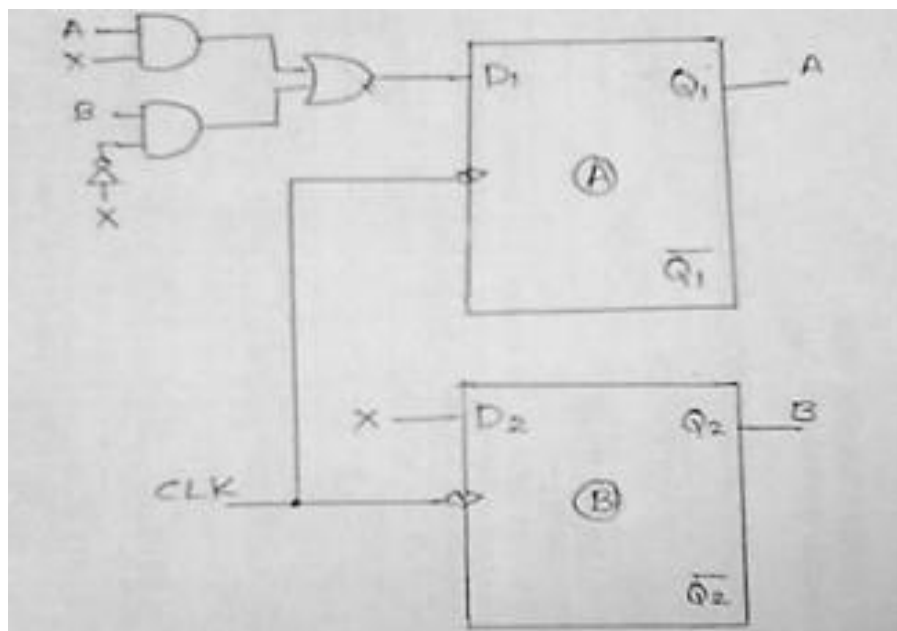
Excitation table:

Present state		Input	Next state		F/F Inputs		Output
A	B	X	A ⁺	B ⁺	D ₁	D ₂	Z
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	0
1	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	1	0	1	0	1
1	1	1	1	1	1	1	0

K-maps to determine inputs to D Flip flop:

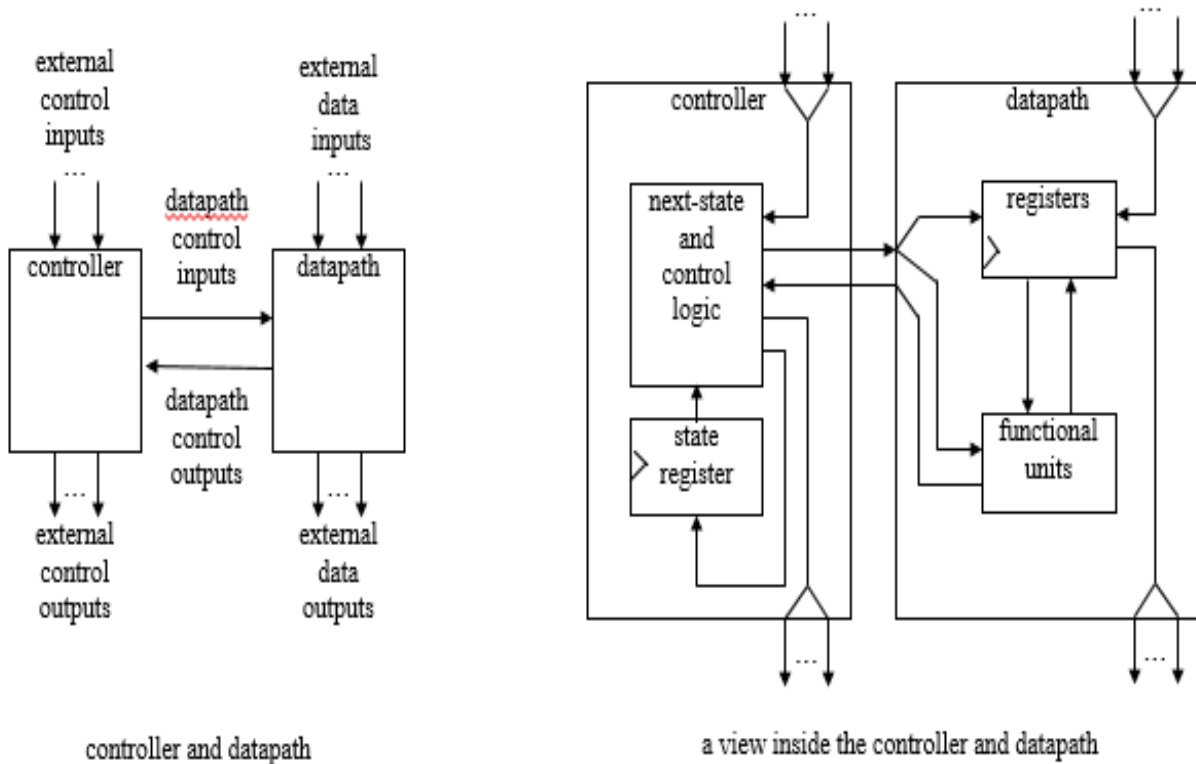


Circuit diagram for the sequence detector:



- Design a sequence detector that produces a true output whenever it detects the sequence 010 at its input.

Custom Single-Purpose Processor Basic Model:



- The data path stores and manipulates the system data. Examples of data in embedded system includes binary numbers representing external conditions like temperature or speed, the character to be displayed on the screen or digitized photographic image to be stored and compressed.
- The data path consists register unit, functional unit, and connection unit like wires and multiplexer.
- The data path can be configured to read the data from register, fed that data into the functional unit configured to carry the operations like add, subtract, shift etc. and stores the result back to the designated register.
- A controller carries out the configuration of data path. It sets the data path control signals like load input to register, select inputs for selecting the register, operation selection for functional unit and connection unit to obtain the desired configuration at particular instant of time.
- It monitors the external control input as well as data path control outputs known as status signal, coming from the functional units and it sets the external control output as well.
- The combinational or sequential logic can be applied to design controller and data path.

Statements:

Statements are used to implement the logical operation, data flow and control flow during the custom single processor design. Some of the useful statements are:

1. Assignment statements
2. Loop statements

3. Branch statements

Assignment Statements:

Assignment statements are used to initialize the fixed data into a variable or transfer of data from one variable to another. For example $a = 5$, $b = 10$, $d = a$ etc. It is used to create a single state and available for next statement.

Assignment statement

a = b
next statement

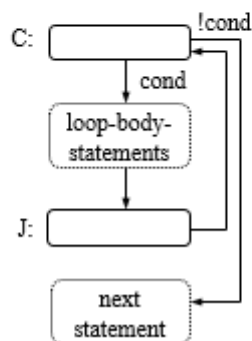
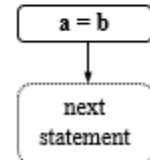
Loop Statements:

Loop statement

```
while (cond) {  
    loop-body-  
    statements  
}  
next statement
```

The control statement are represented by the loop statements. For loop statement:

- We create condition statement C and a join state J, both have no action.
- We add an arc with the loop's condition state to in first statement in loop body.
- We create the second arc with the complement of the loop's condition from the condition state to next statement after the loop body.



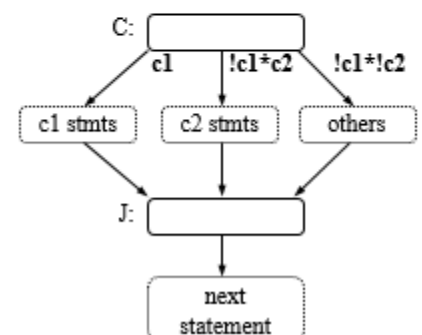
Branch Statements:

Branch statements are used to alter the execution order of statements with or without condition. For branch statements:

- We create the condition statement C with a join statements J both with no action.
- We create the arc from the first branch statement from the condition statement.
- We add another arc with the complement of first branch condition ANDed with second branch condition from the first statement of first branching condition. We repeat this for all branching conditions.
- Finally we connect the arc leaving the last statement of each branch to join state and we add an arc from this state to next statement state.

Branch statement

```
if (c1)  
    c1 stmts  
else if c2  
    c2 stmts  
else  
    other stmts  
next statement
```



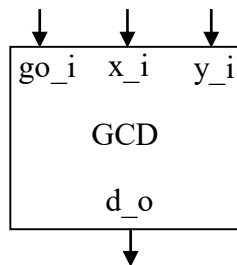
Steps to Design Single Custom Processor

Steps:

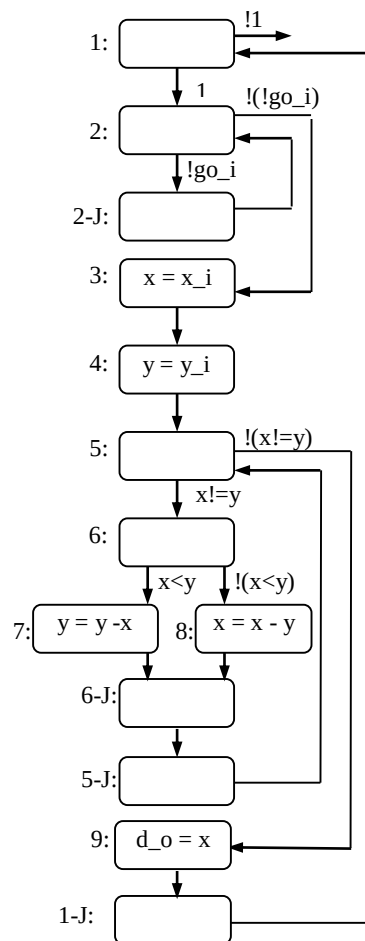
1. Draw a black box that shows the abstract view of the implementation logic.
2. Derive the algorithms to implement the functionality of system.
3. Derive the state diagram to implement the operational logic in terms of control flow, dataflow and applicable logic using different statements.
4. Design the data path, functional unit as well as controller to implement the complex logic specified in step 2.

For example: design of custom single processor to find the GCD of given two numbers

Suppose X_i and Y_i be the two input number, go be the control input and d_o is the GCD of X_i and Y_i such that the black box, functionality and state diagram be designed as below be represented as:

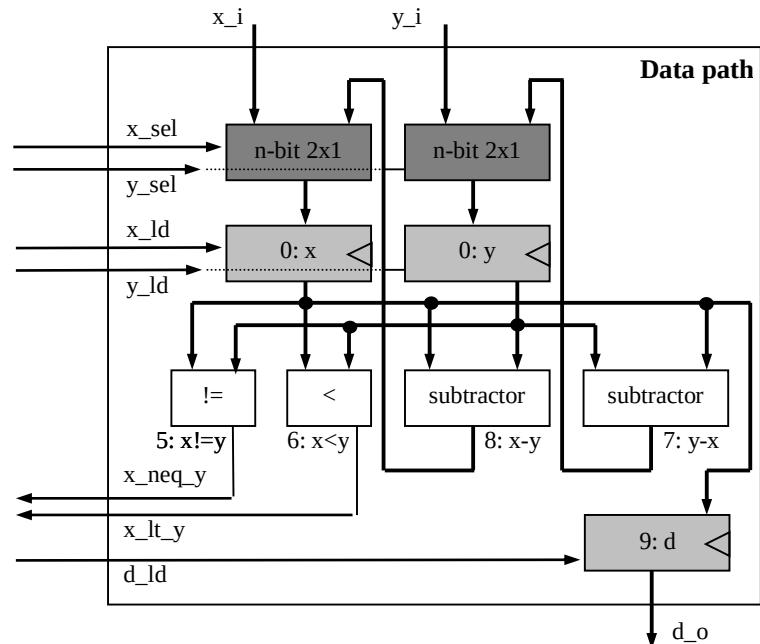


```
0: int x, y;  
1: while (1) {  
2:   while (!go_i);  
3:   x = x_i;  
4:   y = y_i;  
5:   while (x != y) {  
6:     if (x < y)  
7:       y = y - x;  
8:     else  
9:       x = x - y;  
10:  }  
11:  d_o = x;  
12: }
```



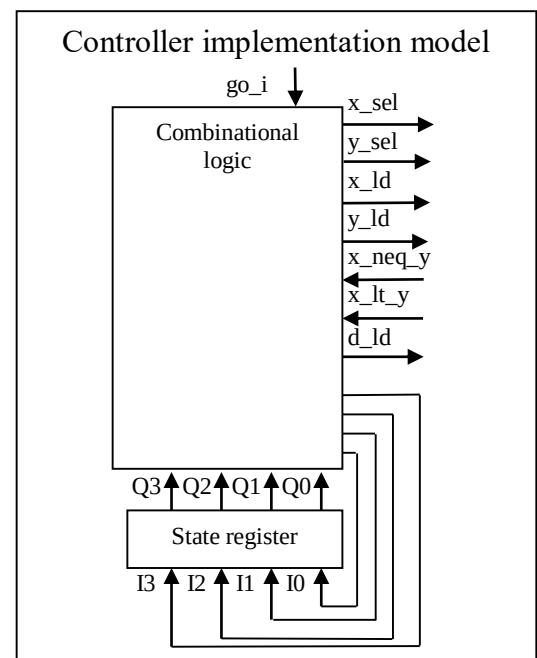
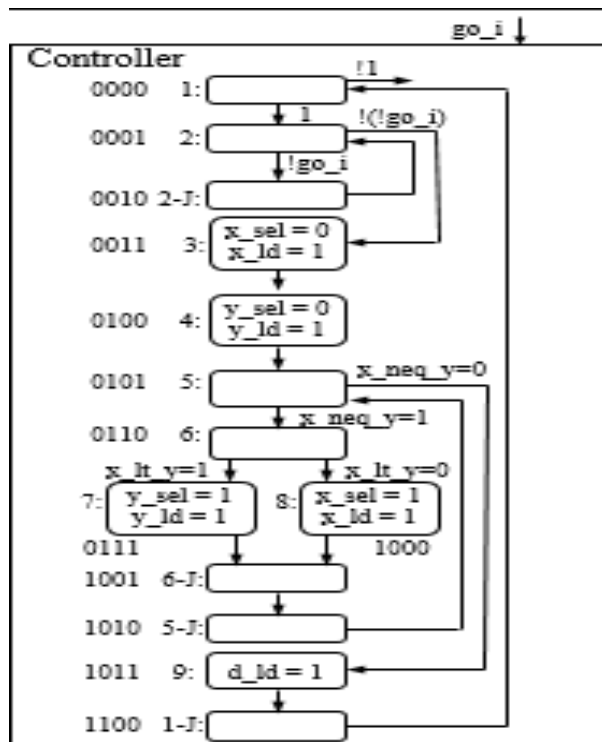
The data path can be designed as below:

- Create a register for any declared variable
- Create a functional unit for each arithmetic operation
- Connect the ports, registers and functional units
 - Based on reads and writes
 - Use multiplexors for multiple sources
- Create unique identifier
 - for each datapath component control input and output



The controller of the above functionality is:

- Same structure as FSMD and convert it to FSM.
- Replace complex actions/conditions with data path configurations



Controller state table for the GCD

Inputs							Outputs								
Q3	Q2	Q1	Q0	x_neq_y	x_lt_y	go_i	I3	I2	I1	I0	x_sel	y_sel	x_ld	y_ld	d_ld
0	0	0	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	0	1	*	*	0	0	0	1	0	X	X	0	0	0
0	0	0	1	*	*	1	0	0	1	1	X	X	0	0	0
0	0	1	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	1	1	*	*	*	0	1	0	0	0	X	1	0	0
0	1	0	0	*	*	*	0	1	0	1	X	0	0	1	0
0	1	0	1	0	*	*	1	0	1	1	X	X	0	0	0
0	1	0	1	1	*	*	0	1	1	0	X	X	0	0	0
0	1	1	0	*	0	*	1	0	0	0	X	X	0	0	0
0	1	1	0	*	1	*	0	1	1	1	X	X	0	0	0
0	1	1	1	*	*	*	1	0	0	1	X	1	0	1	0
1	0	0	0	*	*	*	1	0	0	1	1	X	1	0	0
1	0	0	1	*	*	*	1	0	1	0	X	X	0	0	0
1	0	1	0	*	*	*	0	1	0	1	X	X	0	0	0
1	0	1	1	*	*	*	1	1	0	0	X	X	0	0	1
1	1	0	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	0	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0

Optimization of Single Processor Design

The finite state machine uses the number of states so thaht they can be reduced without halting the operation. The machine can be optimized by optimizing different parameter as below:

- Original program
- FSMD
- Data path
- FSM for controller

1. Optimization of Original Program:

The program can be optimize by optimizing the no of computation, size of variable, time and space complexity, operation used like multiplication and division may have the higher cost etc. for example:

Original program

```
0:   int x,y;
1:   while (1){
2:       while(!go_i);
3:       x=x_i;
4:       y=y_i;
5:       while(x!=y){
6:           if(x<y)
7:               y=y-x;
8:           else
9:               x=x-y;
           }
           d_o=x;
       }
}
```

Let us see, how many iteration does it takes to find GCD of (42,8)

(42,8)→(34,8)→(26,8)→(18,8)→(10,8)→(2,8)→(2,6)→(2,4)→(2,2)

Thus, we can see it takes 9 iteration to complete. Now, let us refine our algorithm by using the next approach as shown below.

Optimized Program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
   // x must be the larger number
3:   if (x_i >= y_i) {
4:     x=x_i;
5:     y=y_i;
   }
6:   else {
7:     x=y_i;
8:     y=x_i;
   }
9:   while (y != 0) {
10:    r = x % y;
11:    x = y;
12:    y = r;
   }
13:  d_o = x;
}
```

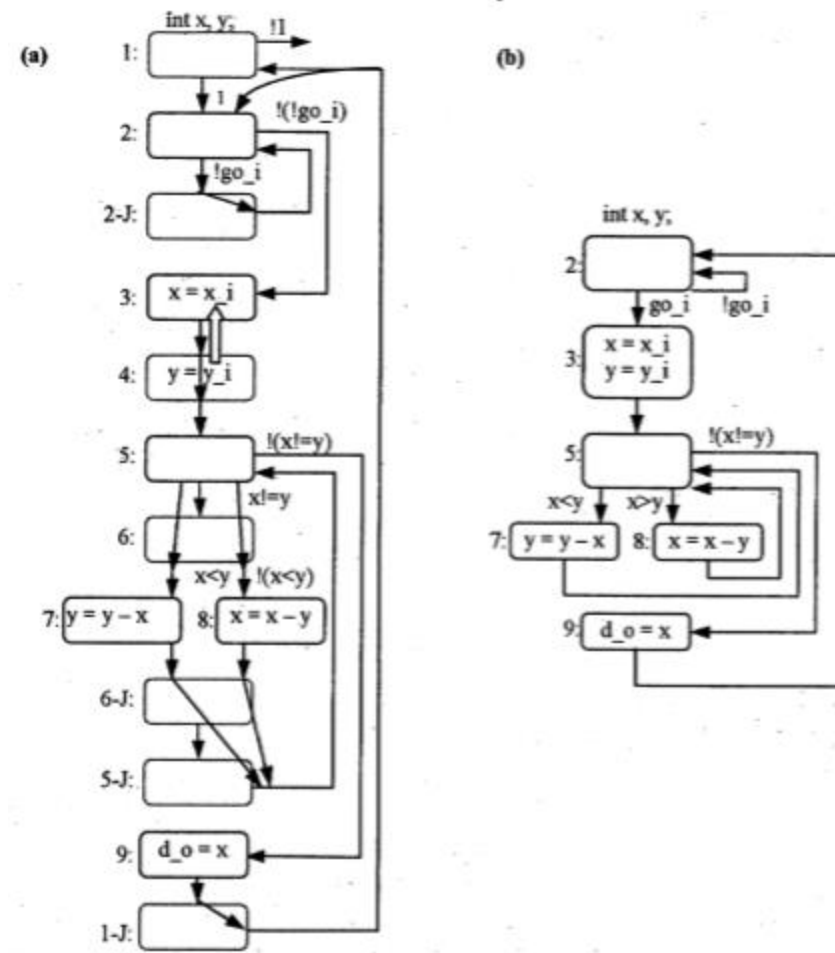
Again let's see how many iteration does it takes to find GCD of (42,8)

(42,8)→(8,2)→(2,0)

Here subtract operation is replaced by modulo operation, the program be optimized in terms of number of computation as well as space and time complexity.

2. Optimization of FSMD:

FSMD can be optimized by using different concepts as merge state, separate state and scheduling. Those states they have constant value or independent with change then they can be merged and so called merged state. The state with complex logic can be replaces by no of sub operation with simpler logic that reduce the hardware complexity and so called as separate state. By optimization of scheduling time, we can optimize the FSMD. For example:



- Eliminate the state -1 that consists constant value.
- Merge the state 2 and 2-J since there is no loop operation in between them.
- Merge the state 3 and 4 since they are independent to each other.
- Merge the state 5 and 6 since transition from state 6 can be done from 5.
- Eliminate state 6-J and 5-J since it can be done from state 7 and 8.
- Eliminate the state 1-J since it can be done through state 9.

3. Optimizing Data Path:

To optimize the data path, a sheared functional unit can be used such that there is a optimized datapath and able perform the verity of operation. For this purpose we can use the sheared ALU circuit that supports verity of state equation as well as state operations. For example: for operation $X-Y$ and $Y-X$, instead of using two sub tractor we can generate the result.

4. Optimizing FSM for Controller:

The FSM of controller can be optimize by reducing the states and using efficient state encoding. The n bit encoded binary bits can define 2^n different states and n bits can generate the $n!$ Combinations. The states can be merged into a single state such that it provides the optimize FSM.

Software Design Issues: General Purpose Processor

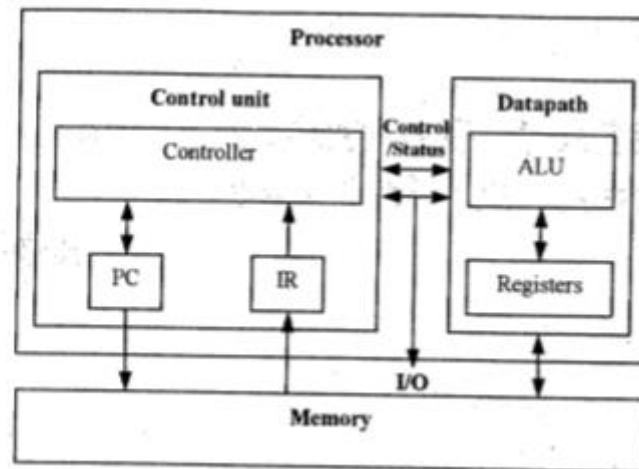


Fig: General Purpose Processor

The general purpose processor sometimes called as central processing unit (CPU) or a microprocessor. It consists of a datapath and a control unit, tightly coupled with memory as shown above fig. the general purpose basic architecture consists of:

- Datapath
- Control unit
- Memory

Data path:

The data path consists the circuitry to transfer the data from one place to another and storing the temporary data. The datapath contains the ALU capable of transferring the data from different operation like additions, subtraction, bitwise OR, AND etc. ALU also generates the status signals often stored in status register. These status bit conditions are known as flags. The flags may be zero, sign carry overflow etc. It also contains register and stores the data temporarily during ALU operation. The temporary data includes:

- Data read from memory but not yet send to ALU.
- Result from ALU operation that is used for next operation or going to write on memory.

The capacity of processor measured by bandwidth of datapath i.e. data carrying capacity. The n-bit size processor consists:

- N bit registers set
- N bit internal and external system bus
- N bit ALU

The processor size may be 4, 8, 16, 32 or 64 bit.

Control Unit

It consists a circuitry that generates the control signal to read the instruction stored in memory, its execution and transferring the data from datapath, storing the result on memory and IO operation. A register called as program counter (PC) that is use to sequence the program i.e. it points the address of next instruction that is going to fetch and execute. Another register called as instruction register (IR) is used to hold the instruction read from memory. The register called as Address Register (AR) is used to store the memory address during memory read/write operation. The control unit has a controller, consisting of a status register plus next state and control logic. It controls the data flow on datapath and such flow includes:

- Inputting the two particular register for ALU operation.
- Storing the ALU results in particular register.
- Moving the data between memory and register.

An m bit sized address memory consists the address space of 2^m and controller goes through the following operation to execute an instructions.

- Fetching the instruction from memory.
- Decode the instruction.
- Fetch the operand from memory
- Perform the operation on memory.
- Store the result.

Memory:

Registers are used as the short term storage whereas memory is used as the mid-term and long –term storage. Memory be classified as:

- Program Memory
- Data memory

The program memory is used to store the sequence of instruction called as program which is use to achieve a given functionality whereas data memory use to store the data information and represents the values of input, output, and transformed by program. We can store the data and program together or separately. The memory architecture follow the following two model:

- Princeton Architecture

- Harvard Architecture

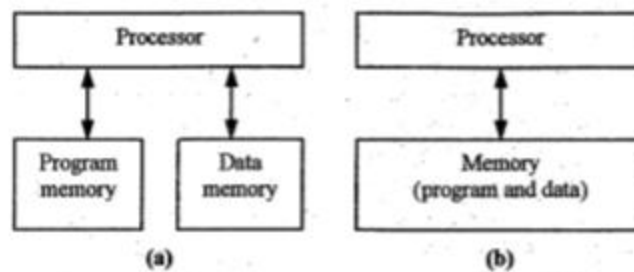


Fig: Harvard architecture (a), Princeton Architecture (b)

- The Princeton architecture shares the common memory space to store the data and program and requires one to one connection with hardware.
- The Harvard architecture uses the separate memory space to store program as well as data and requires different connection. The microcontroller 8051/52 follow this model.

Instruction Execution:

The instruction be executed in microprocessor by taking following steps:

- **Fetch Instruction (FI):** It is a task that reads the instruction from memory pointed by PC and loaded into the instruction register.
- **Decode Instruction (DI):** In this phase, an instruction be decoded to separate the operand reference as well as operation code to represent the particular operation as ADD, MOV, AND, SUB etc.
- **Fetch Operand (FO):** In this stage, the operand be read from the memory represented by the effective address (EA). EA calculation is needed for indirect address.
- **Execute Instruction (EI):** In this phase, the instruction be executed in accordance with its opcode an operand and generates the result.
- **Store Result (SR):** The result is stored on the particular destination that may be register or memory.

Pipeline for Instruction Execution: The pipeline is a common process to increase the instruction throughput of a microprocessor. The pipelining is easily understand by taking the example of washing and drying eight dishes as below.

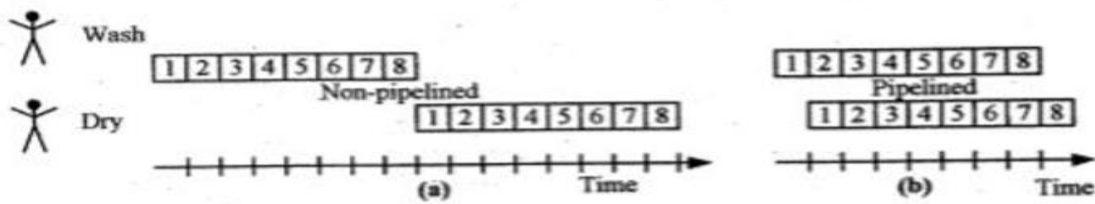
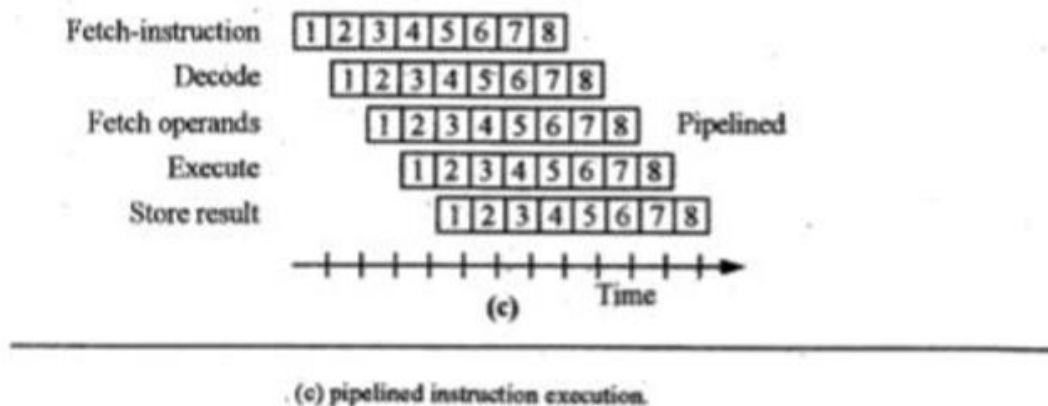


Figure : Pipelining: (a) nonpipelined dish cleaning, (b) pipelined dish cleaning.

In one approach, the first person washes all dishes and second person drying all dishes. If one minutes time is require for each person the total of 16 minutes be required to complete the task. This approach is inefficient since only one person in working other in idle mode. The better approach is that where a person washes and another goes to drying immediately such that this task is finishes at 9 minutes.

The instruction pipeline can be carried to execute the instruction in five independent segments as specified above as FI, DI, FO, EI and SR. The instruction pipeline structure be constructed as:



Programmer View:

A programmer writes the program instruction carryout the desired functionality on GPP. For this purpose, programmer doesn't need to know about the detailed structure of the processor where as he/she need to know how instruction be executed. There are three levels of programming:

1. Assembly Level
2. Structure level
3. Machine level

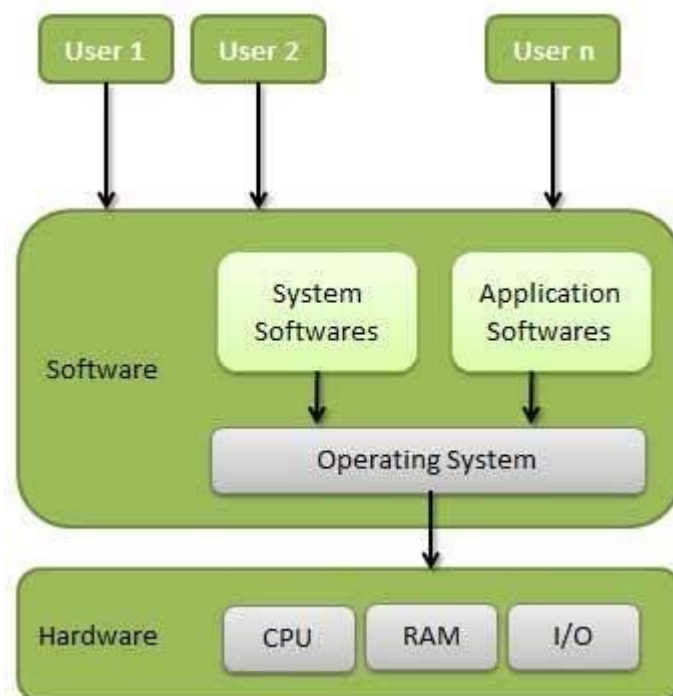
Assembly Level: An assembly language is a low-level programming language for microprocessors and other programmable devices. It is not just a single language, but rather a group of languages. An assembly language implements a symbolic representation of the machine code needed to program a given CPU architecture. Assembly language is also known as assembly code. The term is often also used synonymously with 2GL. The assembler is needed to convert the assembly code into machine code.

Structure Level: Structured programming is a logical programming method that is considered a precursor to object-oriented programming (OOP). Structured programming facilitates program understanding and modification and has a top-down design approach, where a system is divided into compositional subsystems. These language are machine independent and need a compiler to convert them into machine code.

Machine Level: Sometimes referred to as machine code or object code, machine language is a collection of binary digits or bits that the computer reads and interprets. Machine language is the only language a computer is capable of understanding.

Operating System:

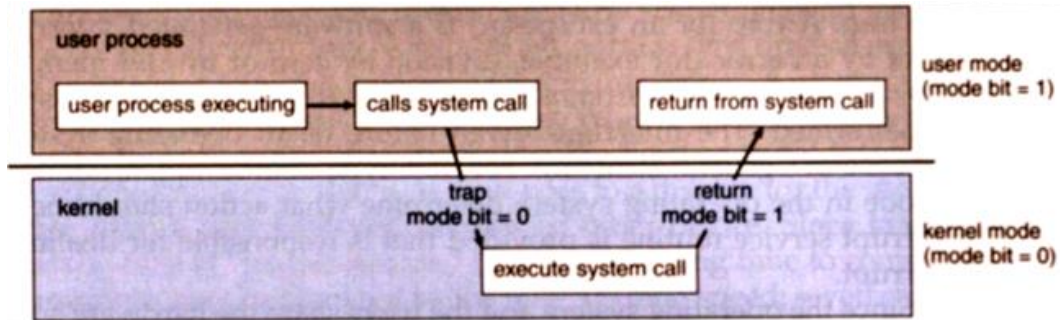
An Operating System (OS) is an interface between a computer user and computer hardware. An operating system is a software which performs all the basic tasks like file management, memory management, process management, handling input and output, and controlling peripheral devices such as disk drives and printers. An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.



The system call provides an interface to the operating system services. Application developers often do not have direct access to the system calls, but can access them through an application programming interface (API). The functions that are included in the API invoke the actual system calls. By using the API, certain benefits can be gained:

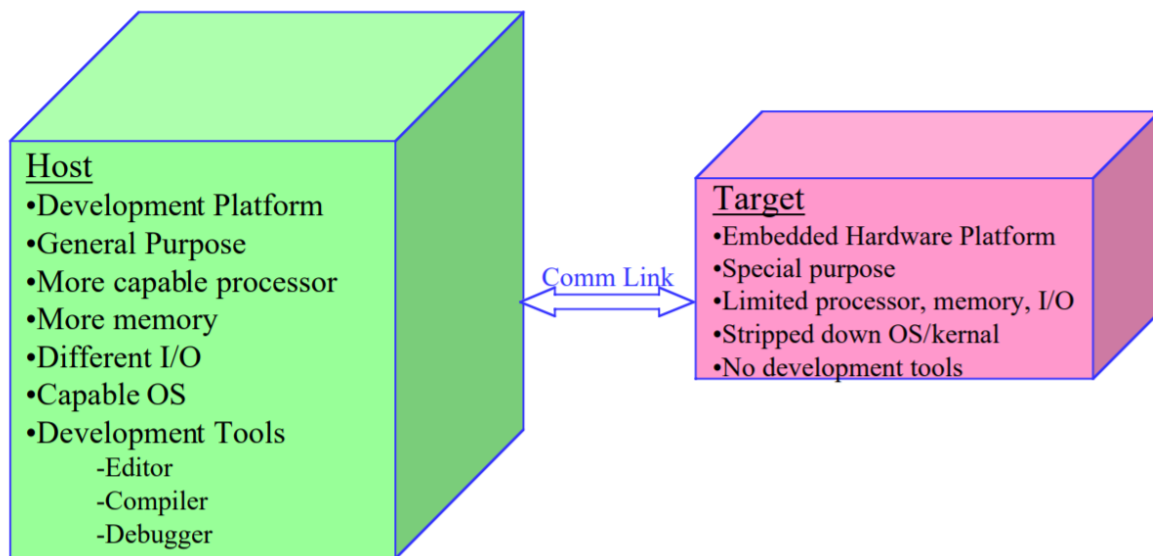
- **Portability:** as long a system supports an API, any program using that API can compile and run.

- Ease of Use: using the API can be significantly easier than using the actual system call.



Development Environment:

The development environment is comprised of the general software tools they are used to design, testing, validation and verification of embedded system software. The software is developed in a general processor called as development processor then it is burned to the target embedded processor.



Embedded Systems Programming requires a more complex software build process:

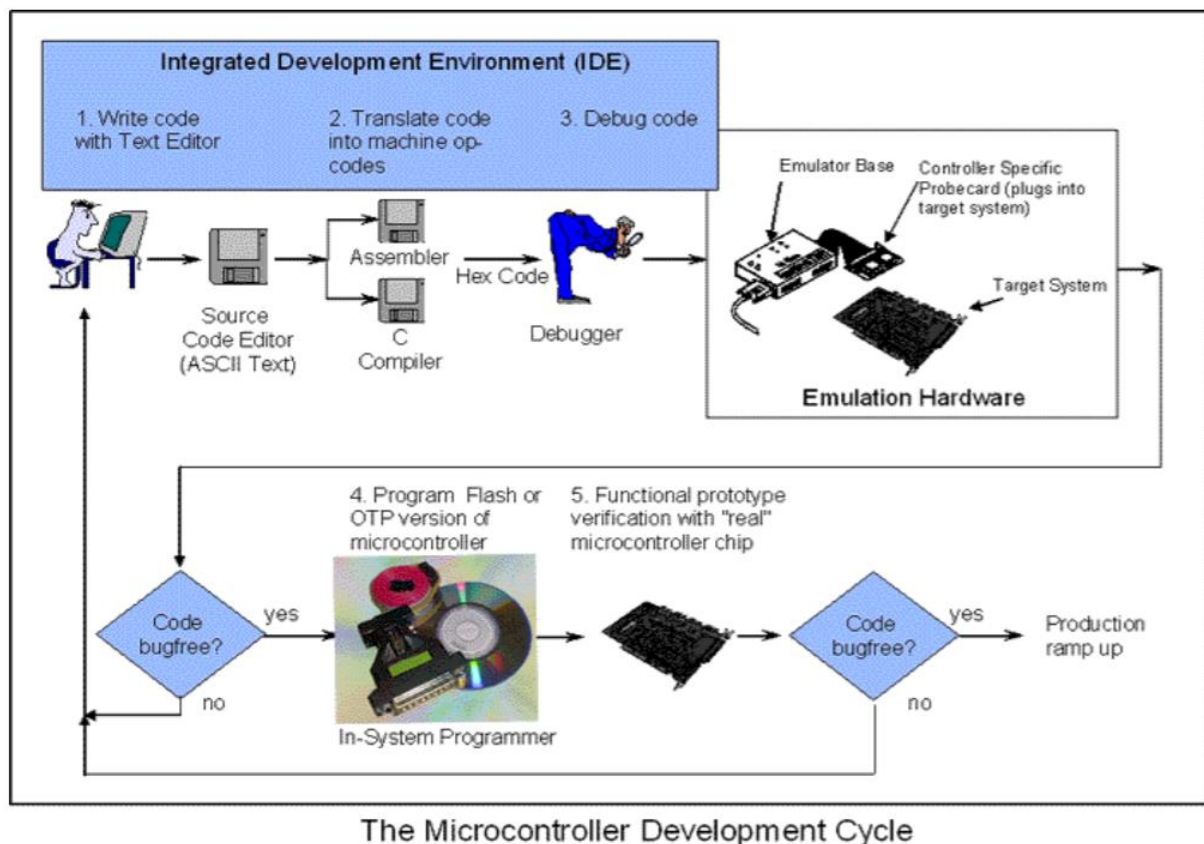
- Target hardware platform consists of target hardware (processor, memory, I/O) and Runtime environment (Operating System/Kernel).
- Target hardware platform contains only what is needed for final deployment.
- Target hardware platform does not contain development tools (editor, compiler, debugger).

Target hardware platform is different from development platform

- Development platform, called the Host Computer, is typically a general purpose computer.
- Host computer runs compiler, assembler, linker, locator to create a binary image that will run on the Target embedded system.

An Integrated Development Environment (IDE)

An Integrated Development Environment (IDE) is software that assists programmers in developing software. IDEs normally consist of a source code editor, a compiler, a linker/locator and usually a debugger. Sometimes, an IDE is devoted to one specific programming language or one (family of) specific processor or hardware but more often the IDEs support multiple languages, processors, etc. Some commonly used IDEs for embedded systems are the GNU compiler collection (gcc), Eclipse, Delphi etc.



Editor

A source code editor is a text editor program designed specifically for editing source code to control embedded systems. It may be a standalone application or it may be built into an integrated development environment (e.g. IDE). Source code editors may have features specifically designed to simplify and speed up input of source code, such as syntax highlighting and auto complete functionality. These features ease the development of code.

Compiler

A compiler is a computer program that translates the source code into computer language (object code). Commonly the output has a form suitable for processing by other programs (e.g., a linker), but it may be a human readable text file. A compiler translates source code from a high level language to a lower level language (e.g., assembly language or machine language). The most common reason for wanting to translate source code is to create a program that can be executed on a computer or on an embedded system. The

compiler is called a cross compiler if the source code is compiled to run on a platform other than the one on which the cross compiler is run. For embedded systems the compiler always runs on another platform, so a cross compiler is needed.

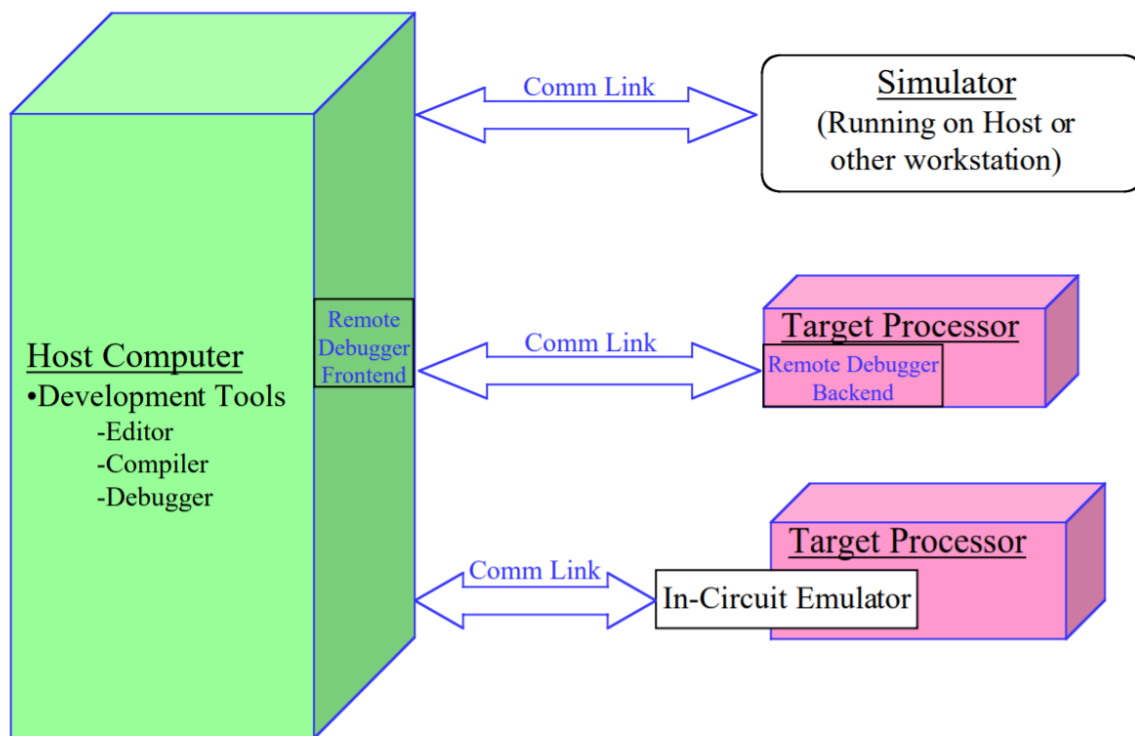
Linker

A linker or link editor is a program that takes one or more objects generated by compilers and assembles them into a single executable program or a library that can later be linked to in itself. All of the object files resulting from compiling must be combined in a special way before the program loader will produce an output file that contains a binary image that can be loaded into the target ROM. A commonly used linker/loader for embedded systems is ld (GNU).

Debugger

A debugger is a computer program that is used to test and debug other programs. It is a piece of software running on the PC, which has to be tightly integrated with the emulator that you use to validate your code. A Debugger allows you to download your code to the emulator's memory and then control all of the functions of the emulator from a PC.

Process for Developing Embedded Software



1. Develop software for a General Purpose Computer:
 - Create source file

- Type in C code
- Build: compile and link
- Execute: load and run
- 2. Develop software for an embedded system
 - Create source file (on Host)
 - Type in C code (on Host)
 - Compile/Assemble: translate into machine code (on Host)
 - Link: combine all object files and libraries, resolve all symbols (on Host)
 - Locate: assign memory addresses to code and data (on Host)
 - Download: copy executable image into Target processor memory
 - Execute: reset Target processor

Compiling Embedded Systems:

Compiler translates program written in human-readable language into machine language:

- Source Code --> Object file
- Object file is binary file that contains set of machine-language instructions (opcodes) and data resulting from language translation process

The process of converting the source code into object file is called compiling. Machine-language instructions are specific to a particular processor and platforms are different for development. The compiler that runs on a computer platform and produces code for that same computer platform called as **Native-compiler**. The compiler that runs on one computer platform and produces code for another computer platform is called **Cross-compiler**.

Assembling/Interpreters for Embedded Systems:

In some cases, a compiler is not used. In this case assembler and interpreter is used.

- Assembler performs one-to-one translation from human-readable assembly language mnemonics to equivalent machine-language opcodes and this process is called assembling.
- Interpreter constantly runs and interprets source code as a set of directives
 - Performs syntax checking as program is typed in
 - Result is slow performance - can be ~1000x slower than an equivalent compiled language ^a
 - Interactive capability provides more feedback -- easier to learn
- Handyboard runs a C interpreter called Interactive C

Linking Embedded Systems:

The Linker combines object files (from compiler) and resolves variable and function references and corresponding process is called linking.

- Source code may be contained in >1 file, which must be combined
- Resolve variables which may be referenced in one file and defined in another file
- Resolve calls to library functions, like sqrt
- May include operating system

Linker creates a relocatable version of the program.

Locating Embedded Systems:

- A Locator is the tool that performs the conversion from relocatable program to executable binary image and corresponding procedure is called locating.
- The Locator assigns physical memory addresses to code and data sections within the relocatable program
- The Locator produces a binary memory image that can be loaded into the target ROM
- In contrast, On General Purpose Computers, the operating system assigns the addresses at load time

Executing Embedded System Program:

Once a program has been successfully compiled, linked, and located, it must be moved to the target platform. Executable binary image is transferred and loaded into a memory device on target board.

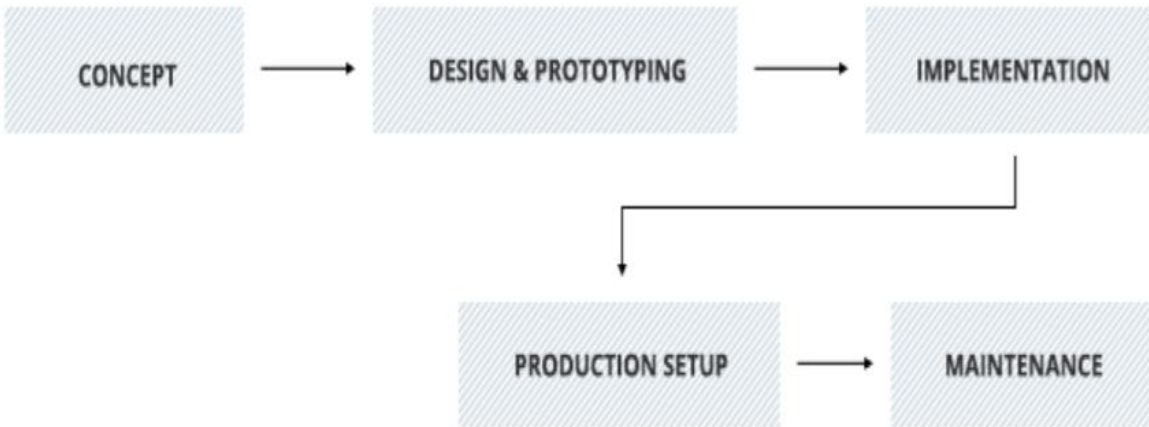
Debugging Tools:

When it comes to debugging your code and testing your application there are several different tools you can utilize that differ greatly in terms of development time spend and debugging features available. In this section we take a look at simulators, and emulators.

Simulators try to model the behavior of the complete microcontroller in software. Some simulators go even a step further and include the whole system (simulation of peripherals outside of the microcontroller). No matter how fast you're PC, there is no simulator on the market that can actually simulate a microcontroller's behavior in real-time. Simulating external events can become a time-consuming exercise, as you have to manually create "stimulus" files that tell the simulator what external waveforms to expect on which microcontroller pin. A simulator can also not talk to your target system, so functions that rely on external components are difficult to verify. For that reason simulators are best suited to test algorithms that run completely within the microcontroller.

An emulator is a piece of hardware that ideally behaves exactly like the real microcontroller chip with all its integrated functionality. It is the most powerful debugging tool of all. A microcontroller's functions are emulated in real-time.

Embedded Product Development Lifecycle:



Concept

- Description of the original idea in a formal technical form (verbal requirements)
- Investigation of the existing prototypes and/or models that match the idea
- Comparative analysis of existing implementations
- Proposal of implementation and materials options

Design

Functional Requirements

- Development of hardware functional specification
- Development of software and firmware functional requirements
- Analysis of the third-party requirements documentation

Architecture Design

- Development of the system architecture concept
- Design of the mechanics parts/molding of the system
- Development of hardware design documentation (including FPGA design)
- Development of the detailed software design specification
- Analysis of the third-party design documents

Hardware Modeling

- Schematics design
- PCB Layout Design
- Re-engineering and repairing
- Samples & Prototypes Assembly

Prototyping

- Product prototyping (including all types of mechanics, hardware, software and the whole system prototyping)
- Mechanical parts manufacturing (including molding/press forms manufacturing) – Hardware development
- Software and firmware coding
- System integration (software with hardware and mechanics)

Application Specific Processor (ASPs)

General purpose processors are designed to execute multiple applications and perform multiple tasks. General purpose processors can be quite expensive especially for small devices that are designed to perform special tasks. Also general purpose processors might lack high performance that a certain task required. Therefore, application specific processors emerged as a solution for high performance and cost effective processors. Application specific processors have become a part of our life's and can be found almost in every device we use on a daily basis. Devices such as TVs, cell phones, and GPSs they all have a form of application specific processors. An application specific processor combines high performance, low cost, and low power consumption.

Application specific processors can be classified into three major categories:

- **Digital Signal Processor (DSP):** Programmable microprocessor for extensive real-time mathematical computations.
- **Application Specific Instruction Set Processors (ASIP):** Programmable microprocessor where hardware and instruction set are designed together for one special application.
- **Application Specific Integrated Circuit (ASIC):** Algorithm completely implemented in hardware.

Application Specific Systems

Some of the typical approaches of building an application specific system or an embedded system are to use one or more of the following implementation methods: GPP, ASIC or ASIP.

- **GPP:**
Functionality of the system is exclusively build on the software level. Although the biggest advantage of such system is the flexibility but it is not optimal in term of performance, power consumption, cost, physical space and heat dissipation.
- **ASIC:**
Compared to GPP, ASIC based systems offer better performance and power consumption but at the cost of flexibility and extensibility. Although it is difficult to use the ASIC for tasks other than what they were designed for, but it is possible to use GPP to perform the more general less demanding tasks in addition to ASIC in the same system.
- **ASIP:**
In this approach, an ASIP is basically a compromise between the two extremes; The Application specific integrated circuit processors ASIC being designed to do mostly a very specific job with high performance but with minimal room for modifications and the general purpose processors which costs a lot more than ASIP but with extreme flexibility at what they do. Due to this flexibility and low price, ASIP are great to be used in embedded and system-on-a-chip solutions.

Table below shows the comparison between the three approaches in term of performance and flexibility and other design considerations.

	GPP	ASIP	ASIC
Performance	low	high	very high
flexibility	excellent	good	poor
HW design	none	large	very large
SW design	small	large	none
power	large	medium	small
reuse	excellent	good	poor
market	very large	relatively large	small
cost	mainly on SW	SOC	volume sensitive

Application-Specific Instruction Set Processors

An ASIP is typically a programmable architecture that is designed in a specific way to perform certain tasks more efficiently. This extra efficiency is not exclusively associated with faster performance. Other factors like reduced production costs, simplified manufacturing process and less power consumption can all be considered efficiency qualities for ASIP. The term “Application” in ASIP is not necessarily related to software applications, it actually describe the class of tasks the ASIP platform was designed to efficiently accomplish. As the name suggests, the Instruction set seems to be the core characteristic of any ASIP based platform; but this is entirely not true. Considering a whole platform, other very important attributes like interfaces and micro-architecture do contribute a lot to the overall system performance.

Interfaces outline how the system and the architecture communicate with each other. Having only an extremely parallelized instruction-set ASIP in a data extensive processing platform doesn't mean necessarily a more efficiently performing system. For example, if the load/store unit cannot handle data processing as quick, there will be a performance bottle-neck due to system interfaces. Similarly for the micro-architecture, the pipeline of the ASIP must be designed in a specific way that optimizes the performance of the whole system. A traditional RISC stages (Fetch, Decode, Execute, memory & write-back) might not be the optimal pipeline for the application. Specifically designing any of these aspects of ASIP will be always associated with trade-offs. Having an extremely parallelized instruction-set will propose issues with the size of the instruction-set and it will affect the program memory, interfaces and fetch & decode stages of the pipeline.

There are many advantages of using a ASIPs as compared to GPP. The following are the major ones:

- **Speed:** The hardware of ASIPs is specially tailored to execute the applications specific instruction. For example an image processor may have an instruction differentiating the input values or may have the instruction for recognizing a bit pattern in communication interface circuit. The hardware is designed to implement the instruction so provides the faster processing.
- **Reprogram ability:** ASIPs can be reprogrammed. The scope of program executed by an ASIPs may be limited to particular application, but still feature of reprogrammability gives the flexibility, though limited, for upgrades and modifications. This saves the time and cost.
- **NRE:** ASIPs are not designed for a specific single task, they are designed for class of task. Unlike single purpose processor, they can be reprogrammed when application requirement changes. Thus the cost of engineering work for producing an ASIC can be distributed which lowers the overall cost. For same task ASIPs are cheaper than single purpose processor.

- **Power Consumption:** compared to GPP, an ASIPs may consume less power. An ASIPs designed to execute specific task; it would not contain unnecessary hardware components required by GPP. It implies lower the hardware lower the cost.

The importance of ASIPs cannot be undermined when there is an increasing use of microprocessor controlled system. The availability of hardware unit including the processor in HDL allows one to implement the processor in ASIP form.

Selecting a Processor:

The selection basis is the requirement, what the processor needs to do. A processor may be selected on the basis of following parameters.

1. **Speed:**
How fast a processor can compute has always been a point of high interest for designers and developers. There is always demand for fast processors. But good practice is to use a processor required by application. A data logging application that records temperature in every 5 minutes might not require a faster processor but a X-ray machine in the emergency ward should have faster one. MIPS (Millions instruction set second) is also used to measure Speed.
2. **Instruction Set:**
The instruction set defines what a processor can do. Based on task to be performed, an additional set of instruction or totally different instruction set may be required. In robotic system the processor for driving a motor and analyzing an environment are required to perform different task, instruction set may be different for them.
3. **Bit/ Word Length:**
It is a size of data the processor can handled, the length of register around processor. If processor needs to process floating point data, the word length that works for integer type data is no sufficient. A narrower option may works but takes more time.
4. **Power Consumption:**
Power consumption may not a deciding factor for fixed system but when it comes to a portable handheld device, it becomes a crucial point. Both the standby and peak power consumption of processor are to be considered. For example, the low power consumed by mobile phone in standby or sleep and active mode should low.
5. **Prior Experience:**
While working in a project, a designer would chose a processor with which he has experience. The availability of development and libraries for the SW for a processor contributes to the performance.
6. **Size:**
The actual physical size of the processor may impact the design when the trend is going for slim smart devices. Everything, including processor is required to be small.
7. **Cost:**
The price of the processor is the ultimate selection factor. The available project budget may not accommodate expensive processor then designer selects the low cost processor.

Other factors may be type/ version, no of register etc., may also be used as selection criteria.

General Purpose Processor Design:

The general purpose processor design follows the states as like in singe processor design as:

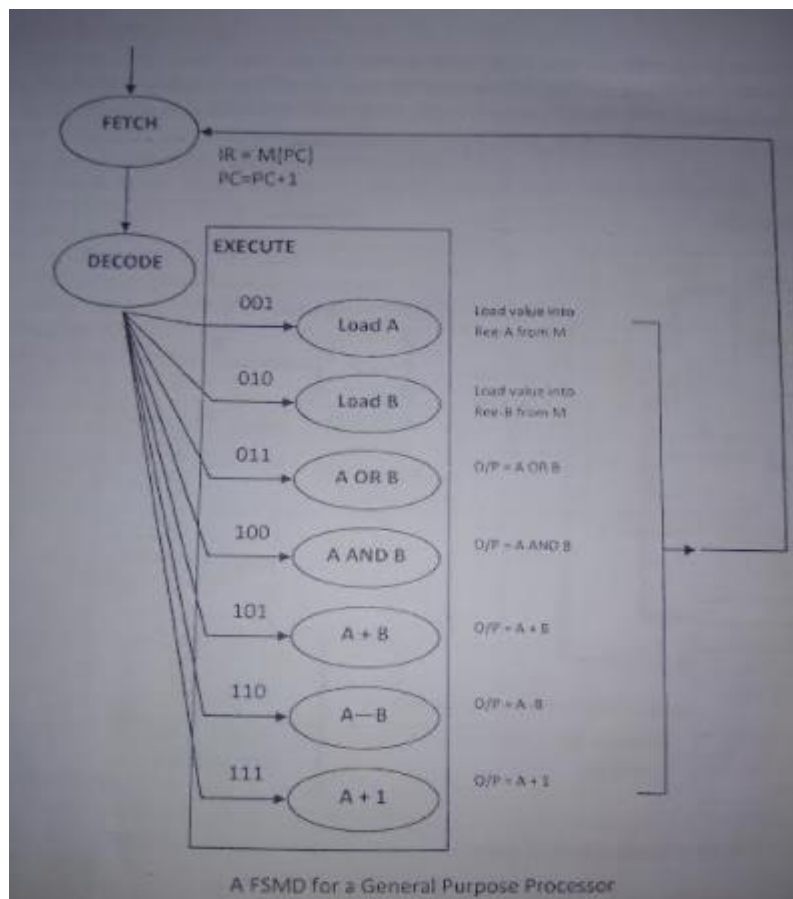
1. Create a FSMD to describe a behavior of processor. I.e. design instruction set.

2. Built the data path to carry the data flow between different functional units.
3. Rewrite the FSMD to FSM
4. Design the FSM controller.

A general purpose processor is characterized by its nature of reprogramming for a wide variety of applications. A GPP is designed to execute generalized basic instructions which can be used to write programs that perform different tasks. A lot of effort is required in the design phase in order to generalize the processor so that it can be programmed for different solutions. The high design cost is acceptable if the NRE cost is very low.

The design of a general purpose processor requires listing the operations required to perform. A binary code that specifies an operation is called an instruction, and a list of instructions is called an instruction set. An instruction set for a general purpose processor with 7 instructions is as in the table.

The FSMD of a general purpose processor requires no sub-operation to execute the instruction. The data flow to execute the above instruction set is constructed as:



Instruction	Operation Code
Load A	001
Load B	010
A OR B	011
A AND B	100
A+B	101
A-B	110
A+1	111

The execution of instruction requires following steps:

1. Fetch Instruction (FI): It is a task that reads the instruction from memory pointed by PC and loaded into the instruction register.
2. Decode Instruction (DI): In this phase, an instruction be decoded to separate the operand reference as well as operation code to represent the particular operation as ADD, AND, SUB etc.
3. Fetch Operand (FO): In this stage, the operand be read from the memory represented by the effective address (EA). EA calculation is needed for indirect address.
4. Execute Instruction (EI): In this phase, the instruction be executed in accordance with its opcode an operand and generates the result.
5. Store Result (SR): The result is stored on the particular destination that may be register or memory.

The design of data follows from the instruction set. Each expression shown in FSMD are carried out by different functional unit. The datapath must contain the functional units required by instruction. The operation of functional unit is define by the control unit. These control signal are generated from the controller. A MUX is used to select the functional units, memory and register used in data path. The data path must be able to transfer the information about address, data and control.

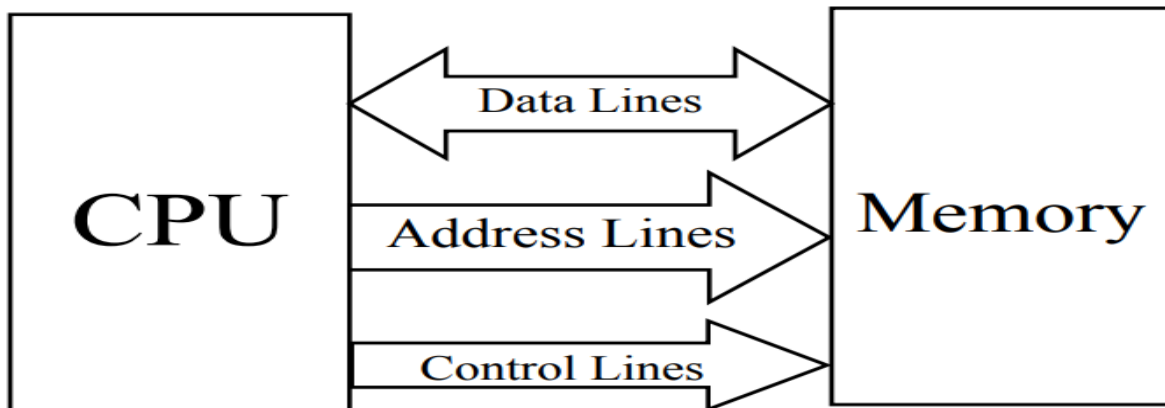
The controller is a finite state machine that goes from one to another state and its design involves sequential design processor. The state diagram be different but procedure is almost same.

Chapter- 3

Memory

The Memory stores the instructions as well as data. No one can distinguish an instruction and data. The processor has to be directed to the address of the instruction codes. The memory is connected to the processor through the following lines.

1. Address
2. Data
3. Control



In a memory read operation the CPU loads the address onto the address bus. Most cases these lines are fed to a decoder which selects the proper memory location. The CPU then sends a read control signal. The data is stored in that location is transferred to the processor via the data lines. In the memory write operation after the address is loaded the CPU sends the write control signal followed by the data to the requested memory location. The memory can be classified in various ways i.e. based on the location, power consumption, way of data storage etc. The memory at the basic level can be classified as:

1. Processor Memory (Register Array)
2. Internal on-chip Memory
3. Primary Memory
4. Cache Memory
5. Secondary Memory

✓ **Processor Memory (Register Array)**

Most processors have some registers associated with the arithmetic logic units. They store the operands and the result of an instruction. The data transfer rates are much faster without needing any additional clock cycles. The number of registers varies from processor to processor. The more is the number the faster is the instruction execution. But the complexity of the architecture puts a limit on the amount of the processor memory.

✓ **Internal on-chip Memory**

In some processors there may be a block of memory location. They are treated as the same way as the external memory. However it is very fast.

✓ **Primary Memory**

This is the one which sits just outside the CPU. It can also stay in the same chip as of CPU. These memories can be static or dynamic.

✓ **Cache Memory**

This is situated in between the processor and the primary memory. This serves as a buffer to the immediate instructions or data which the processor anticipates. There can be more than one levels of cache memory.

✓ **Secondary Memory**

These are generally treated as Input/Output devices. They are much cheaper mass storage and slower devices connected through some input/output interface circuits. They are generally magnetic or optical memories such as Hard Disk and CDROM devices. The memory can also be divided into Volatile and Non-volatile memory.

✓ **Volatile Memory**

The contents are erased when the power is switched off. Semiconductor Random Access Memories fall into this category.

✓ **Non-volatile Memory**

The contents are intact even of the power is switched off. Magnetic Memories (Hard Disks), Optical Disks (CDROMs), Read Only Memories (ROM) fall under this category.

Data Storage

An m word memory can store $m \times n$: m words of n bits each. One word is located at one address therefore to address m words we need.

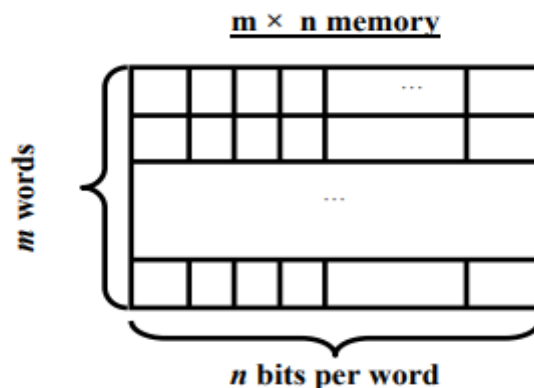
$$k = \log_2(m) \text{ address input signals}$$

OR

$$k \text{ number address lines can address } m = 2^k \text{ words.}$$

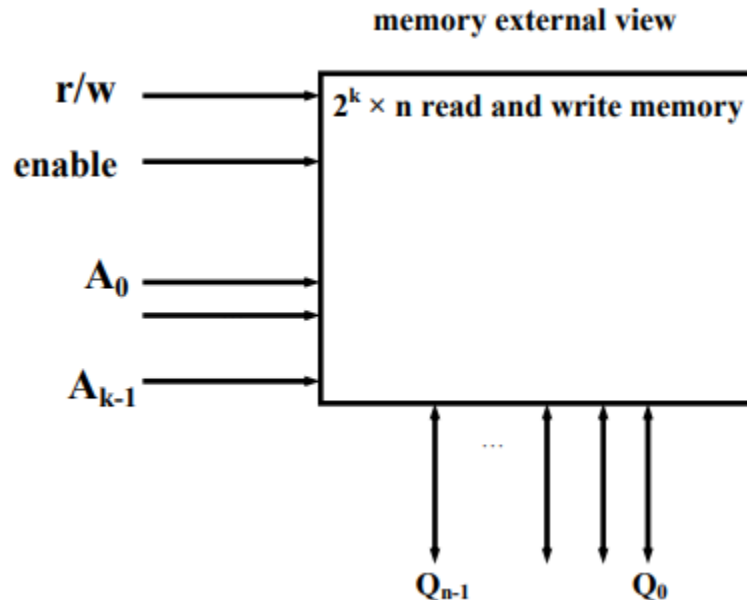
Example 4,096 x 8 memory:

- 32,768 bits
- 12 address input signals
- 8 input/output data signals



Memory Access:

The memory location can be accessed by placing the address on the address lines. The control lines read/write selects read or write. Some memory devices are multi-port i.e. multiple accesses to different locations simultaneously.



Memory Specifications

The specification of a typical memory is as follows

- The storage capacity: The number of bits/bytes or words it can store.
- The memory access time (read access and write access): How long the memory takes to load the data on to its data lines after it has been addressed or how fast it can store the data upon supplied through its data lines. This reciprocal of the memory access time is known as Memory.
- The Power Consumption and Voltage Levels: The power consumption is a major factor in embedded systems. The lesser is the power consumption the more is packing density.
- Size: Size is directly related to the power consumption and data storage capacity.

There are two important specifications for the Memory as far as Real Time Embedded Systems are concerned.

- Write Ability
- Storage Performance

Write Ability:

It is the manner and speed that a particular memory can be written. The memory write ability refers to the process of putting the bits in specific location of the memory and the ease and speed with which the process can be completed. The writing process may be time consuming like in ROMs or faster like in registers.

A basic writing process involves providing the address values to the address line and data to data lines and selecting the writing function. But there are different methods to actually write the data into memory.

A ROM memory can be built using combinational logic whose inputs act as address lines and output acts as data lines. The circuit has to be designed such that each combination of inputs gives out data that is stored in the address given by input lines, once implemented in hardware the stored values cannot be changed. Another circuit has to be designed for another set of values. The write time is comparatively large.

A register is made up of flip-flops. Writing is easily accomplished, just put the data in the data lines and enable load. Similarly, RAMs which are built around basic transistor storage cells, also have faster writing ability. These memory are rewritable.

On the basis of writing ability, memory can be:

- High end:
The memory with high end write ability is easiest one to write data into. These are flip-flop based memory that processor can write directly into. Examples are: registers, RAMs. These memory are used in computation process in embedded system.
- Middle Range:
The memories in this range are little bit difficult to write and are slower than high end ones. The processors can still write in them, but with a slower speed. These memories are not accessed frequently and used for storing the data for a longer period of time. Examples are FLASH, EEPROM. These memory are used in testing phase of embedded system.
- Lower Range:
A special programmer device is required to write data into this type of memory. It is slower memory. Examples are: UV EPROM in which data is written by using voltage higher than that of normal operation, one time programmable (OTP) ROM, in which data is written by blowing connections which represent bit values. The OTPROM can be programmed once.
- Low End
In the low end memory device, data is written only during manufacturing. The memory device is manufactured with data into it. The data writing process starts with design of chip. Once manufactured, the data cannot be rewritten. The mask programmed ROM is the example. In embedded system, such memories can be used to hold program or some data that are used frequently, but values to be stored must be final.

Storage permanence:

It is the ability to hold the stored bits. The ability is temporary or permanent. Range of storage permanence are:

- High end
This range of memory essentially never loses bits e.g., mask-programmed ROM, OTPROM. The programs of embedded system are placed in this range of memory.
- Middle range
This range memory holds for a bits over certain period of time as days, months, or years after memory's power source turned off e.g., NVRAM, a battery backed RAM or flash memory.
- Lower range
This range memory holds bits as long as power supplied to memory • e.g., SRAM

- Low end
It begins to lose bits almost immediately after written i.e refreshing circuit is needed to hold data correctly e.g., DRAM

Common Memory Types

Read Only Memory (ROM)

This is a nonvolatile memory. It can only be read from but not written to, by a processor in an embedded system. Traditionally written to, “programmed”, before inserting to embedded system.

Uses:

- Store software program for general-purpose processor i.e. program instructions can be one or more ROM words
- Store constant data needed by system
- Implement combinational circuit

Example:

The figure shows the structure of a ROM. Horizontal lines represents the words. The vertical lines give out data. These lines are connected only at circles. If address input is 010 the decoder sets 2nd word line to 1. The data lines Q3 and Q1 are set to 1 because there is a “programmed” connection with word 2’s line. The word 2 is not connected with data lines Q2 and Q0. Thus the output is 1010.

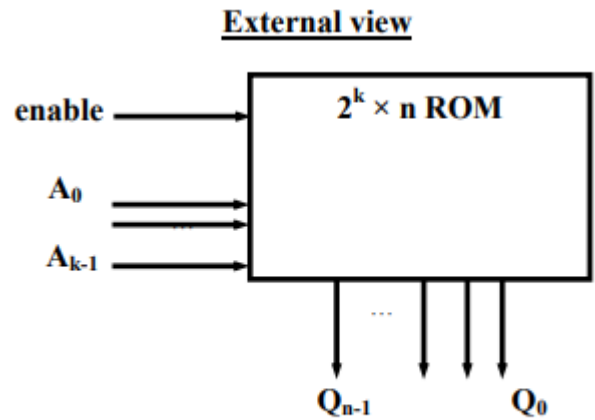


Fig. 5.7 The ROM Structure

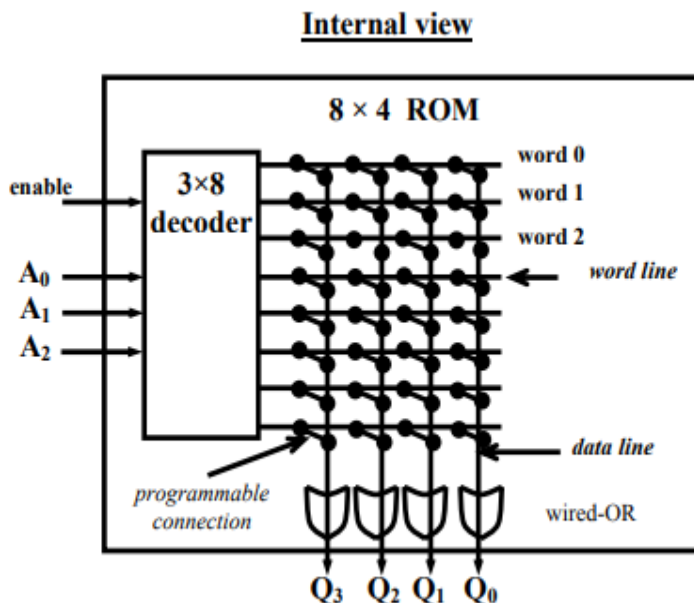


Fig. 5.8 The example of a ROM with decoder and data storage

Implementation of Combinatorial Functions:

Any combinational circuit of n functions of same k variables can be done with $2^k \times n$ ROM. The inputs of the combinational circuit are the address of the ROM locations. The output is the word stored at that location.

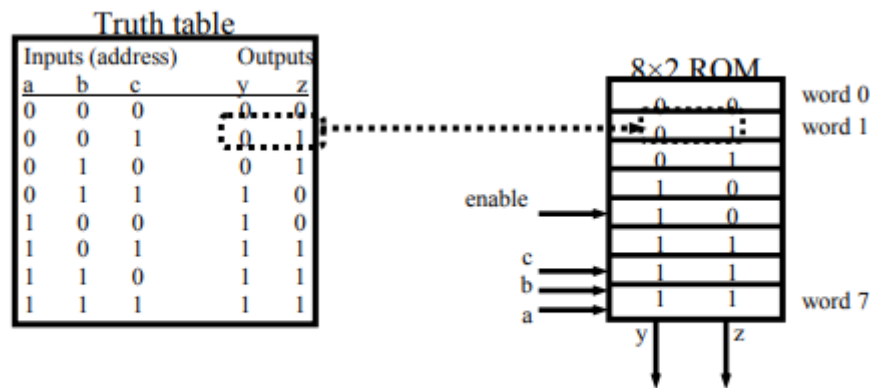


Fig. 5.9 The combinatorial table

Mask-programmed ROM

The connections “programmed” at fabrication. They are a set of masks. It can be written only once (in the factory). But it stores data for ever. Thus it has the highest storage permanence. The bits never change unless damaged. These are typically used for final design of high-volume systems.

OTP ROM: One-time programmable ROM

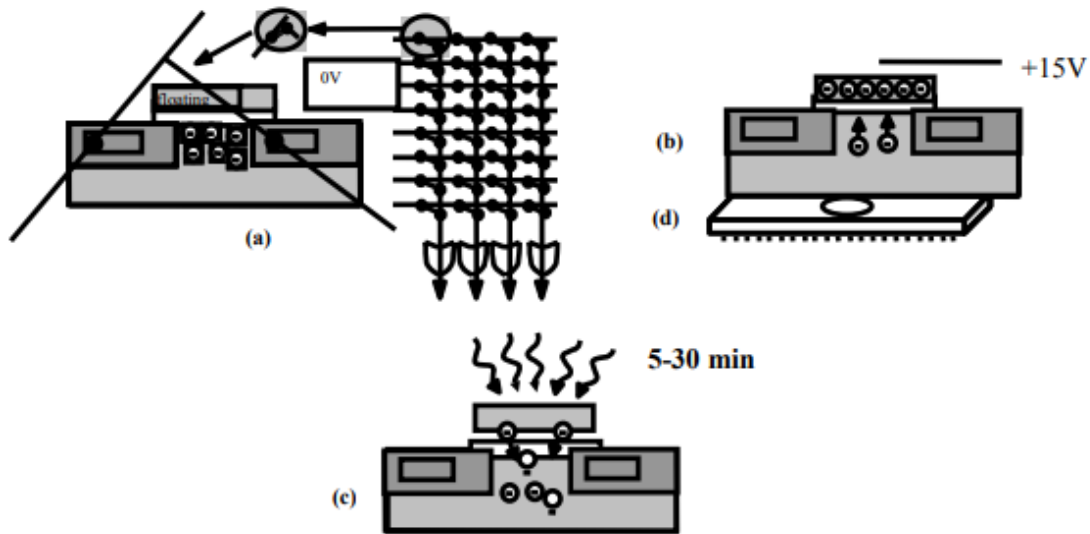
The Connections “programmed” after manufacture by user. The user provides file of desired contents of ROM. The file input to machine called ROM programmer. Each programmable connection is a fuse. The ROM programmer blows fuses where connections should not exist.

- Very low write ability: typically written only once and requires ROM programmer device
- Very high storage permanence: bits don’t change unless reconnected to programmer and more fuses blown
- Commonly used in final products: cheaper, harder to inadvertently modify

EPROM: Erasable programmable ROM

This is known as erasable programmable read only memory. The programmable component is a MOS transistor. This transistor has a “floating” gate surrounded by an insulator. The Negative charges form a channel between source and drain storing a logic 1. The Large positive voltage at gate causes negative charges to move out of channel and get trapped in floating gate storing a logic 0. The (Erase) Shining UV rays on surface of floating-gate causes negative charges to return to channel from floating gate restoring the logic 1. An EPROM package showing quartz window through which UV light can pass. The EPROM has

- Better write ability – can be erased and reprogrammed thousands of times
- Reduced storage permanence – program lasts about 10 years but is susceptible to radiation and electric noise
- Typically used during design development



EEPROM

EEPROM is otherwise known as Electrically Erasable and Programmable Read Only Memory. It is erased typically by using higher than normal voltage. It can program and erase individual words unlike the EPROMs where exposure to the UV light erases everything. It has

- Better write ability
 - can be in-system programmable with built-in circuit to provide higher than normal voltage i.e. built-in memory controller commonly used to hide details from memory user
 - Writes very slowly due to erasing and programming. “busy” pin indicates to processor EEPROM still writing
 - Can be erased and programmed tens of thousands of times
- Similar storage permanence to EPROM (about 10 years)
- Far more convenient than EPROMs, but more expensive.

Flash Memory:

It is an extension of EEPROM. It has the same floating gate principle and same write ability and storage permanence. It can be erased at a faster rate i.e. large blocks of memory erased at once, rather than one word at a time. The blocks are typically several thousand bytes large

- Writes to single words may be slower
 - Entire block must be read, word updated, then entire block written back
- Used with embedded systems storing large data items in nonvolatile memory
 - e.g., digital cameras, TV set-top boxes, cell phones

RAM: “Random-access” memory:

- Typically volatile memory

- bits are not held without power supply
- Read and written to easily by embedded system during execution
- Internal structure more complex than ROM
 - a word consists of several memory cells, each storing 1 bit
 - each input and output data line connects to each cell in its column
 - rd/wr connected to every cell
 - when row is enabled by decoder, each cell has logic that stores input data bit when rd/wr indicates write or outputs stored bit when rd/wr indicates read

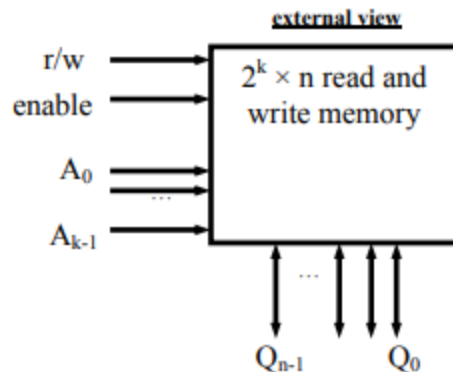


Fig. 5.11 The structure of RAM

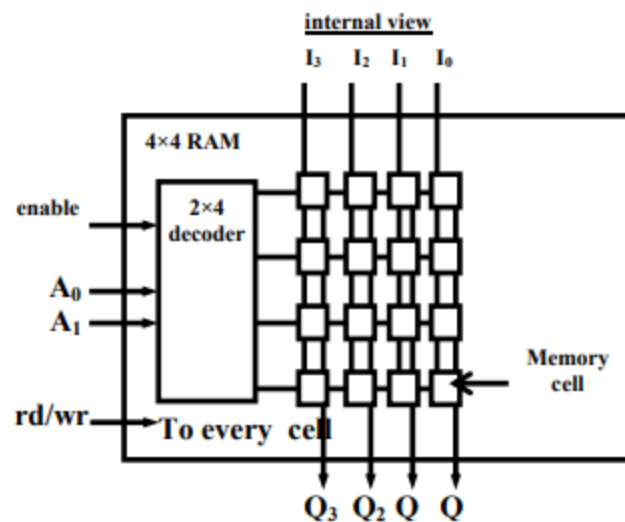
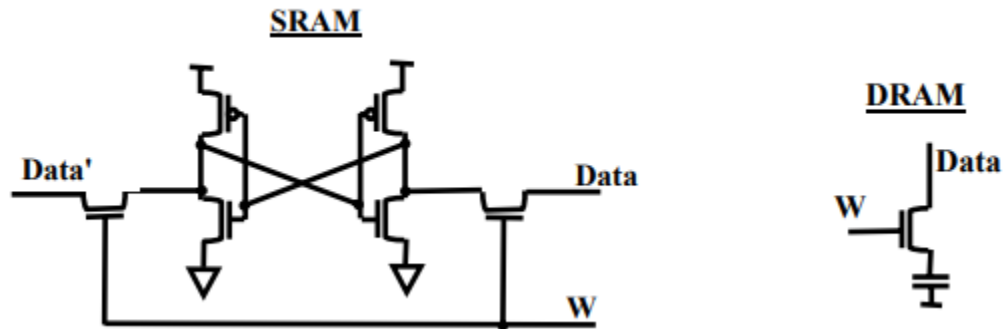


Fig. 5.12 The RAM decoder and access

Basic types of RAM

- SRAM: Static RAM
 - Memory cell uses flip-flop to store bit
 - Requires 6 transistors
 - Holds data as long as power supplied
- DRAM: Dynamic RAM

- Memory cell uses MOS transistor and capacitor to store bit
- More compact than SRAM – “Refresh” required due to capacitor leak i.e. word’s cells refreshed when read
- Typical refresh rate 15.625 micro sec.
- Slower to access than SRAM

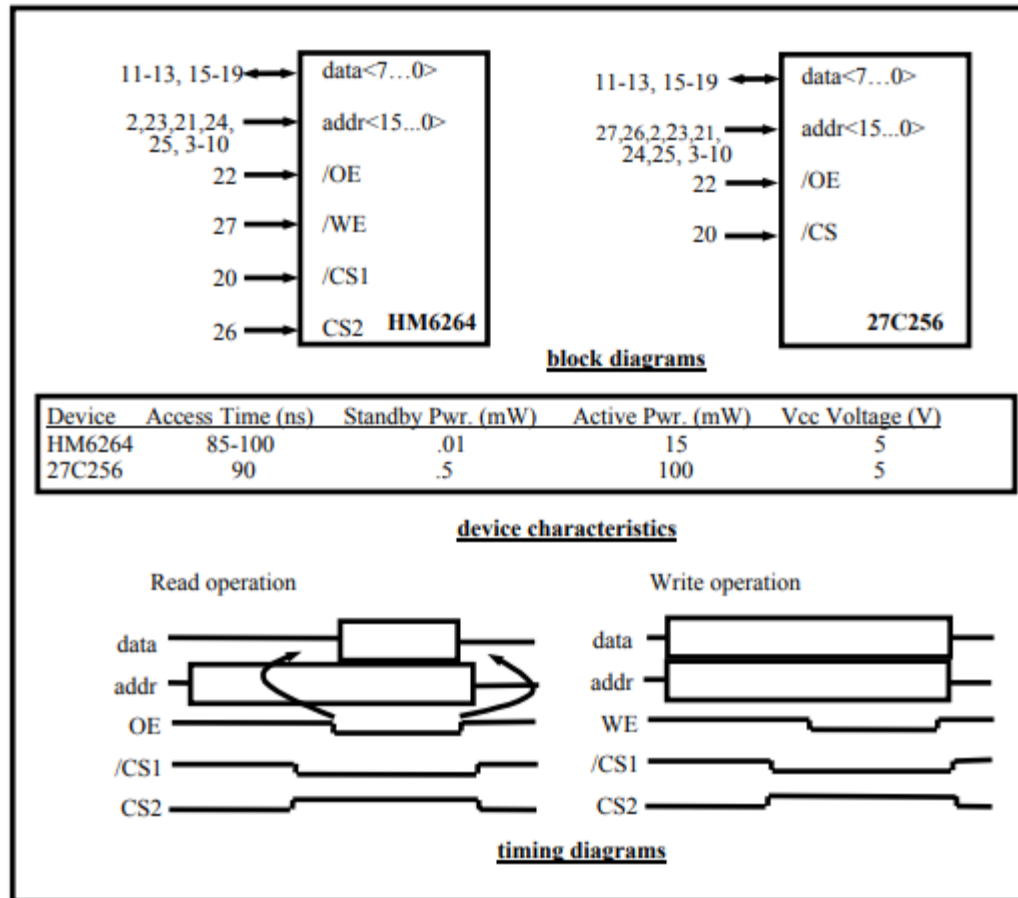


Ram variations

- PSRAM: Pseudo-static RAM
 - DRAM with built-in memory refresh controller
 - Popular low-cost high-density alternative to SRAM
- NVRAM: Nonvolatile RAM
 - Holds data after external power removed
 - Battery-backed RAM
 - SRAM with own permanently connected battery
 - writes as fast as reads
 - no limit on number of writes unlike nonvolatile ROM-based memory
 - SRAM with EEPROM or flash stores complete RAM contents on EEPROM or flash before power

Example: HM6264 & 27C256 RAM/ROM devices

- Low-cost low-capacity memory devices
- Commonly used in 8-bit microcontroller-based embedded systems • First two numeric digits indicate device type
 - RAM: 62
 - ROM: 27
- Subsequent digits indicate capacity in kilobits



Composing memory:

- Memory size needed often differs from size of readily available memories
- When available memory is larger, simply ignore unneeded high-order address bits and higher data lines
- When available memory is smaller, compose several smaller memories into one larger memory
 - Connect side-by-side to increase width of words
 - Connect top to bottom to increase number of words
 - added high-order address line selects smaller memory containing desired word using a decoder
 - Combine techniques to increase number and width of words

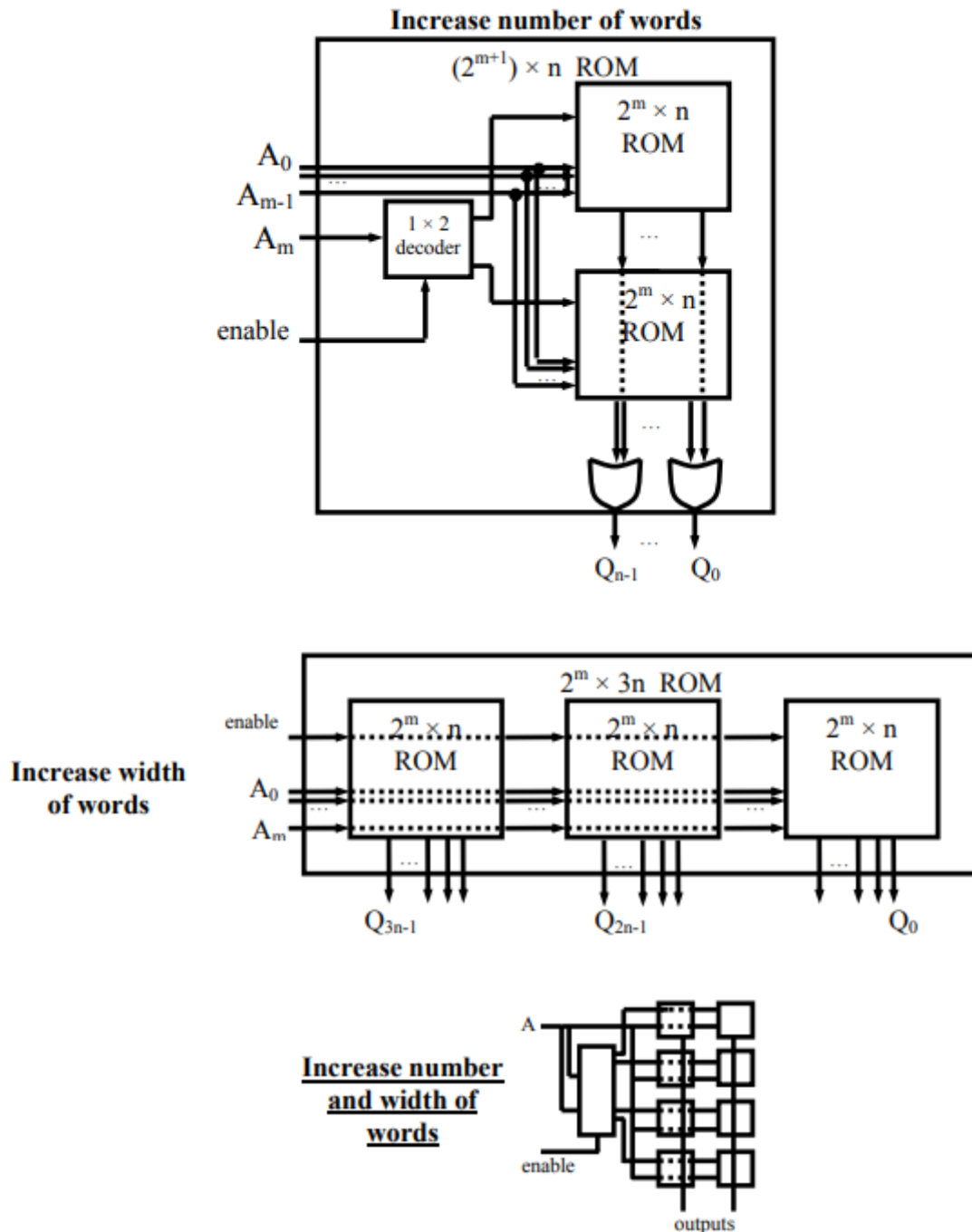


Fig. 5.13 Composing Memory

Memory Hierarchy

The memory hierarchy separates computer storage into a hierarchy based on response time. Since response time, complexity, and capacity are related, the levels may also be distinguished by their performance and controlling technologies. The embedded designer can choose the memory from this classification in terms of cost, size and response time. Objective is to use inexpensive, fast memory for the embedded system design.

- Main memory
 - Large, inexpensive, slow memory stores entire program and data
- Cache
 - Small, expensive, fast memory stores copy of likely accessed parts of larger memory
 - Can be multiple levels of cache.

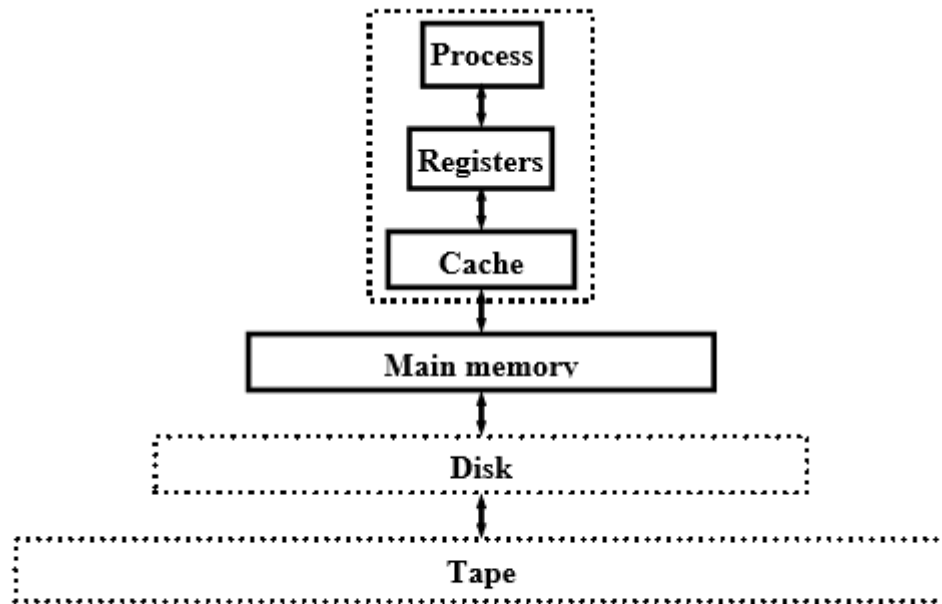


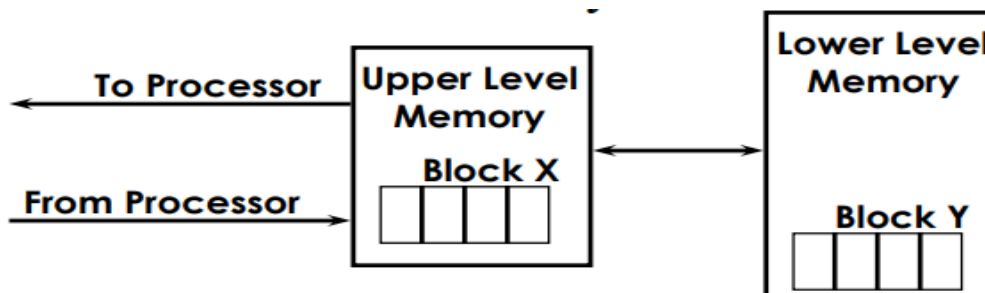
Fig. 6.1 The memory Hierarchy

Cache

At any given time, data is copied between only two adjacent levels:

- **Upper level:** the one closer to the processor
 - Smaller, faster, uses more expensive technology
- **Lower level:** the one away from the processor
 - Bigger, slower, uses less expensive technology

The basic unit of information transfer is Block. The minimum unit of information that can either be present or not present in a level of the hierarchy is called as block.



Program access a relatively small portion of the address space at any instant of time i.e. 10% of code executed 90% of time. The processor access the same data and code confined in an area due to use of branching, subroutine call and return, interrupt processing etc. this principle is called **locality of reference**. There are two types of locality:

- **Temporal locality**: if an item is referenced, it will tend to be referenced again soon.
- **Spatial locality**: if an item is referenced, items whose addresses are close by tend to be referenced soon.

A small and fastest memory is attached in between main memory and processor is called as cache memory. It is used to implement the principle of locality of reference and hold the frequently or recently accessed data.

- Usually designed with SRAM
 - faster but more expensive than DRAM
- Usually on same chip as processor
 - space limited, so much smaller than off-chip main memory
 - faster access (1 cycle vs. several cycles for main memory)
- Cache operation
 - Request for main memory access (read or write)
 - First, check cache for copy
 - cache hit - copy is in cache, quick access
 - cache miss - copy not in cache, read address and possibly its neighbors into cache
- Several cache design choices
 - cache mapping, replacement policies, and write techniques

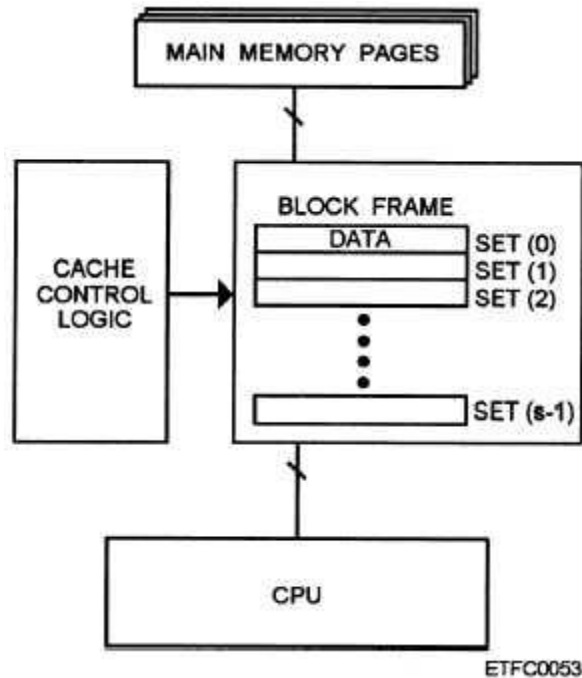
Cache Mapping:

Cache mapping is the method by which the contents of main memory are brought into the cache and referenced by the CPU. The mapping method used directly affects the performance of the entire embedded system. It is necessary as there are far fewer number of available cache addresses than the memory

- Are address' contents in cache?
- Cache mapping used to assign main memory address to cache address and determine hit or miss.
- Three basic techniques:
 - Direct mapping
 - Fully associative mapping
 - Set-associative mapping
- Caches partitioned into indivisible blocks or lines of adjacent memory addresses.
 - usually 4 or 8 addresses per line

Direct Mapping:

Main memory locations can only be copied into one location in the cache. This is accomplished by dividing main memory into pages that correspond in size with the cache.



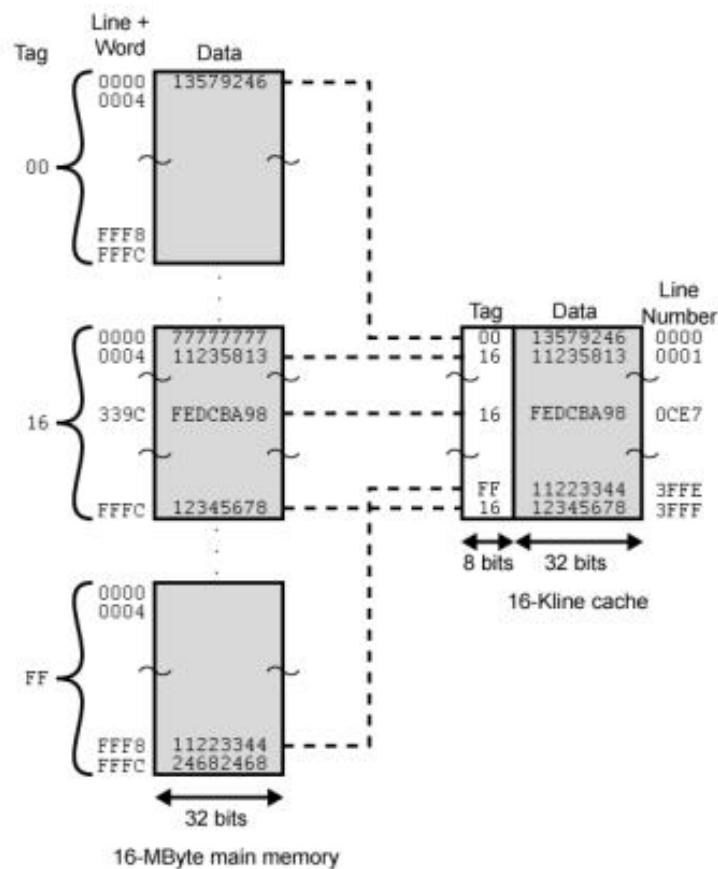
- It is the simplex technique, maps each block of main memory into only one possible cache line i.e. a given main memory block can be placed in one and only one place on cache. $i = j \text{ modulo } m$
Where I = cache line number; j = main memory block number; m = number of lines in the cache.
- The mapping function is easily implemented using the address. For purposes of cache access, each main memory address can be viewed as consisting of three fields.
- The least significant w bits identify a unique word or byte within a block of main memory. The remaining s bits specify one of the 2^s blocks of main memory.
- The cache logic interprets these s bits as a tag of $(s-r)$ bits most significant position and a line field of r bits. The latter field identifies one of the $m = 2^r$ lines of the cache
 - Address length = $(s + w)$ bits
 - Number of addressable units = 2^{s+w} words or bytes
 - Block size = line size = 2^w words or bytes
 - Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
 - Number of lines in cache = $m = 2^r$
 - Size of tag = $(s - r)$ bits

Consider a cache memory system with following parameters.

- The cache can hold 64 Kbytes
- Data is transferred between main memory and the cache in blocks of 4 bytes each. This means that the cache is organized as $16\text{Kbytes} = 2^{14}$ lines of 4 bytes each.
- The main memory consists of 16 Mbytes with each byte directly addressable by a 24 bit address ($2^{24} = 16\text{Mbytes}$). Thus, for mapping purposes, we can consider main memory to consist of 4Mbytes blocks of 4 bytes each.

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
- 8 bit tag (=22-14), 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

Tag s-r	Line or Slot r	Word w
8	14	2



	Tag	Line	Word
Main memory address =	8	14	2

Fig: Direct mapping example

Pros and Cons

- Simple
- Inexpensive
- Fixed location for given block
- If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high.

Associated Mapping

- It overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of cache.
- Cache control logic interprets a memory address simply as a tag and a word field
- Tag uniquely identifies block of memory
- Cache control logic must simultaneously examine every line's tag for a match which requires fully associative memory
- very complex circuitry, complexity increases exponentially with size
- Cache searching gets expensive.
 - Address length = $(s + w)$ bits
 - Number of addressable units = 2^{s+w} words or bytes
 - Block size = line size = 2^w words or bytes
 - Number of blocks in main memory = 2^{s+w}
 - Number of lines in cache = 2^s
 - Number of lines in cache = 2^s
 - Size of tag = s bits
- 22 bit tag stored with each 32 bit block of data

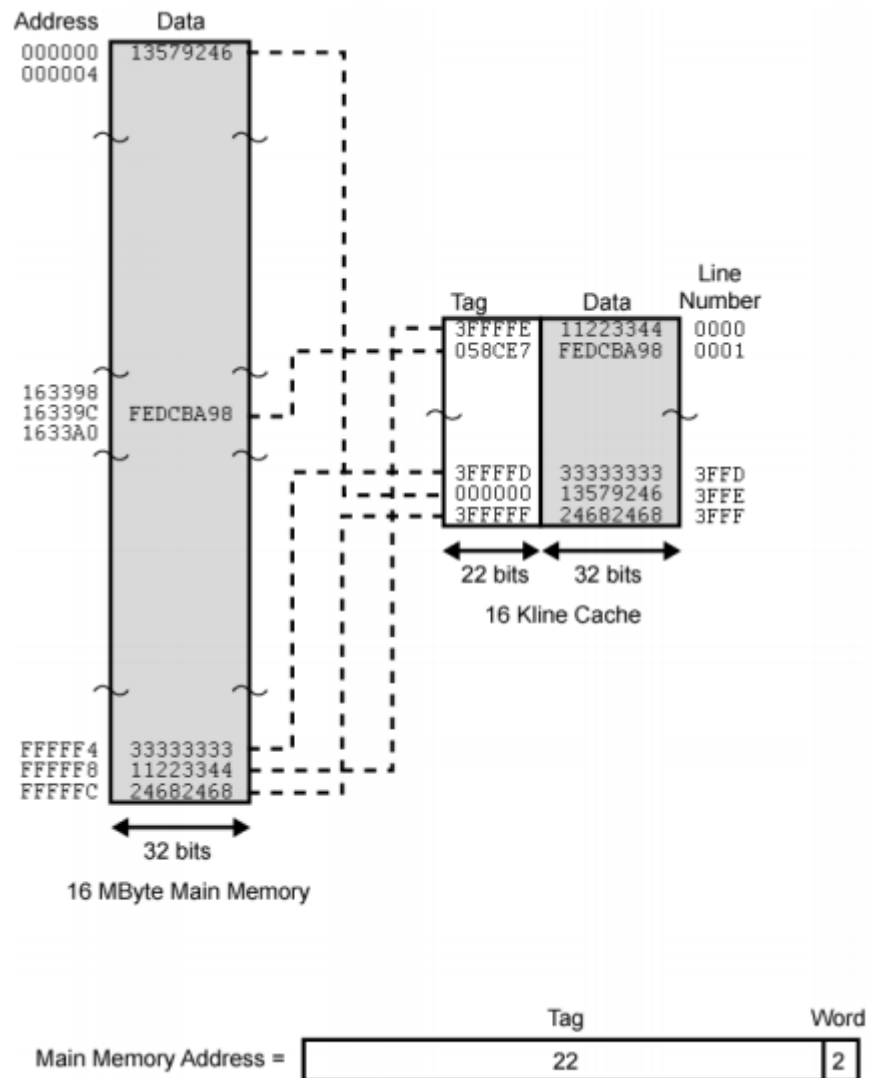


Fig: Associative mapping example

- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which 16 bit word is required from 32 bit data block
- e.g.

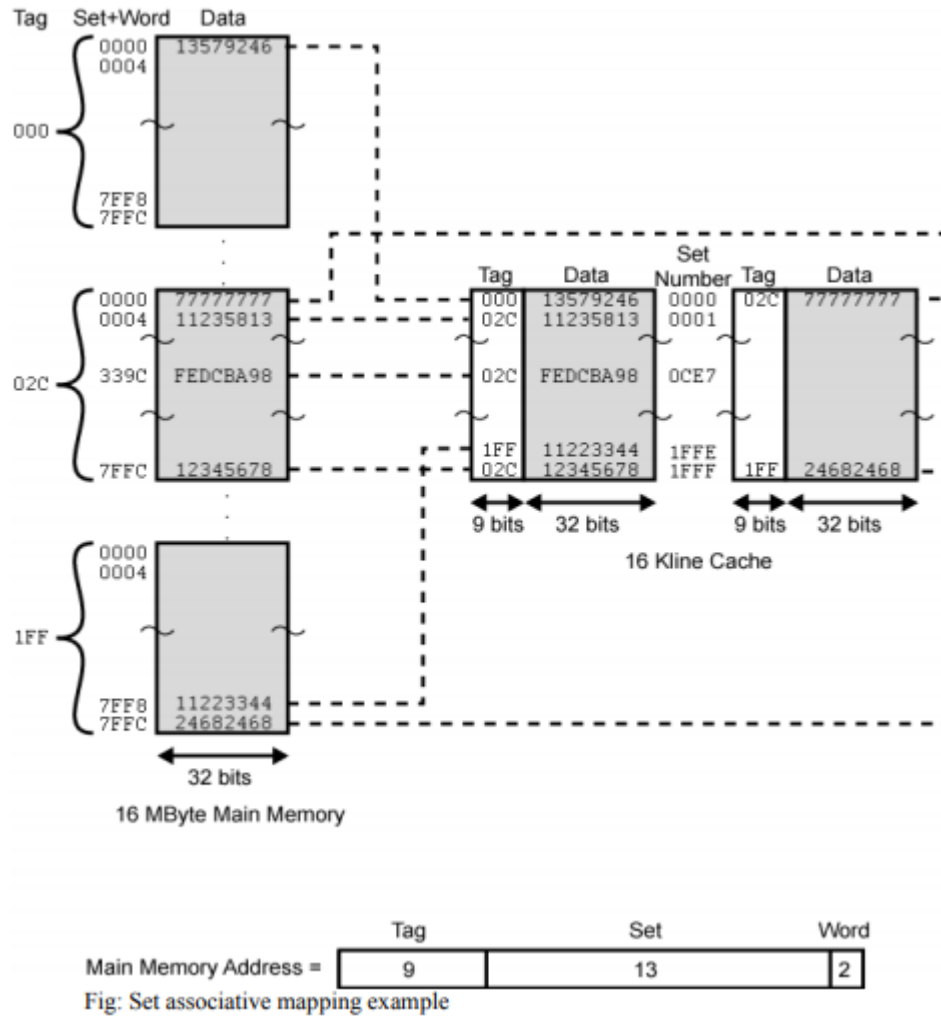
Address	Tag	Data	Cache line
FFFFFC	FFFFFC	24682468	3FFF

Set Associated Mapping

- It is a compromise between direct and associative mappings that exhibits the strength and reduces the disadvantages
- Cache is divided into v sets, each of which has k lines; number of cache lines = vk
 $M = v \times k \times I = j \text{ modulo } v$ Where, i = cache set number; j = main memory block number; m = number of lines in the cache
- So a given block will map directly to a particular set, but can occupy any line in that set (associative mapping is used within the set)
- Cache control logic interprets a memory address simply as three fields tag, set and word. The d set bits specify one of $v = 2^d$ sets. Thus s bits of tag and set fields specify one of the 2^s block of main memory.
- The most common set associative mapping is 2 lines per set, and is called two-way set associative. It significantly improves hit ratio over direct mapping, and the associative hardware is not too expensive
 - Address length = $(s + w)$ bits
 - Number of addressable units = 2^{s+w} words or bytes
 - Block size = line size = 2^w words or bytes
 - Number of blocks in main memory = 2^d
 - Number of lines in set = k
 - Number of sets = $v = 2^d$
 - Number of lines in cache = $kv = k * 2^d$
 - Size of tag = $(s - d)$ bits

- 13 bit set number
- Block number in main memory is modulo 2^{13}
- 000000, 00A000, 00B000, 00C000 ... map to same set
- Use set field to determine cache set to look in
- Compare tag field to see if we have a hit
- e.g.

Address	Tag	Data	Set number
1FF 7FFC	1FF	12345678	1FFF
001 7FFC	001	11223344	1FFF



Replacement algorithm

When all lines are occupied, bringing in a new block requires that an existing line be overwritten. • Algorithms must be implemented in hardware for speed and they are:

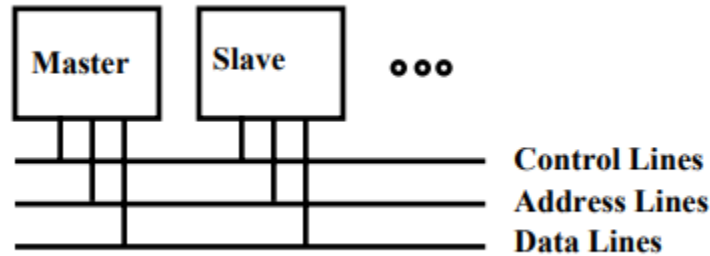
- Least Recently used (LRU)
- First in first out (FIFO)
- Least-frequently-used (LFU)
- Random

Chapter – 4

Interfacing

Interfacing is a way to communicate and transfer information in either way without ending into deadlocks. In our context it is a way of effective communication in real time. This involves:

- Addressing
- Arbitration
- protocol



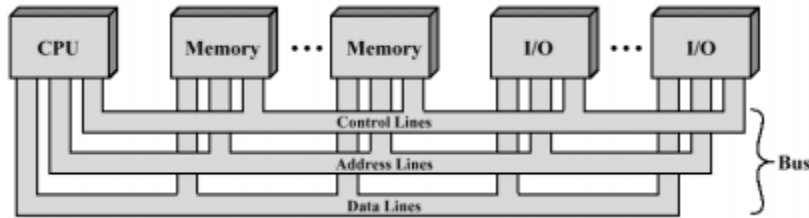
- Addressing: The data sent by the master over a specified set of lines which enables just the device for which it is meant.
- Protocols: The literal meaning of protocol is a set of rules. Here it is a set of formal rules describing how to transfer data, especially between two devices.
- Arbitration: it specifies the process of generating the logic to select the bus connection over different connected peripherals.

The interface is required:

- For speed synchronization between processor and peripherals since peripherals operates in serial communication mode whereas processor operates in parallel mode.
- To manage the data format since the peripheral operates in bit or byte format where memory and processor operates in word format. The word consists no of bytes.
- To maintain the energy of operation since processor and memory uses the semiconductor and operates using electronic energy whereas peripherals are electromechanical.

System Buses: Interfacing Processors and Peripherals:

The system bus provides a common path to transfer the data between the two devices i.e. between source and destination. When it is use to transfer the information between different devices with in processor then it is called as internal bus. If they are used to transfer the information between peripherals processor and memory then called as external bus.



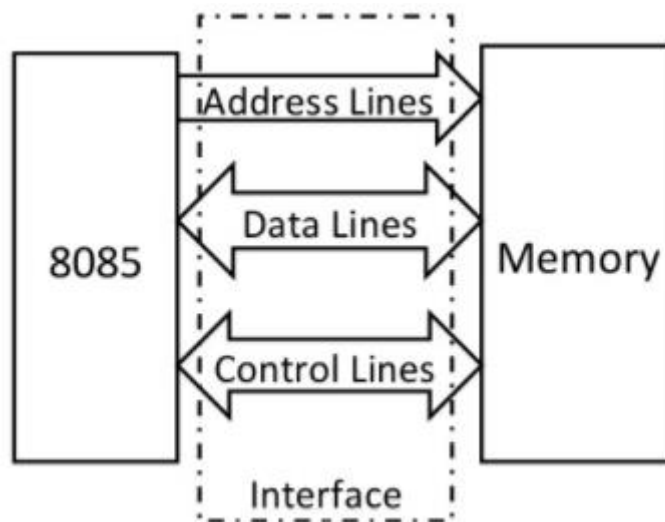
System interconnection structures may support these transfers:

- Memory to processor: the processor reads instructions and data from memory
- Processor to memory: the processor writes data to memory
- I/O to processor: the processor reads data from I/O device
- Processor to I/O: the processor writes data to I/O device
- I/O to or from memory: I/O module allowed to exchange data directly with memory without going through the processor - Direct Memory Access (DMA)

Signals found on a bus are:

- Memory write: data on the bus written into the addressed location
- Memory read: data from the addressed location placed on the bus
- I/O write: data on the bus output to the addressed I/O port
- I/O read: data from the addressed I/O port placed on the bus
- Bus REQ: indicates a module needs to gain control of the bus
- Bus GRANT: indicates that requesting module has been granted control of the bus
- Interrupt REQ: indicates that an interrupt is pending
- Interrupt ACK: Acknowledges that pending interrupt has been recognized
- Reset: Initializes everything connected to the bus
- Clock: on a synchronous bus, everything is synchronized to this signal

Processor and Memory Interfacing:



- Processor and memory are interconnected through the system bus and system bus consists the three different lines as data, control and address.
- A enable signal is use to active the memory for memory operation.
- The devices they participate in communication are called as actor. There are two types of actor as: master actor and slave actor. The master actor initiates the communication whereas the slave actor response the communication.
- Address line provides the unique identification of memory location that stores the word which is going to transfer or write.
- Data lines transfers data between master and slave i.e. it is called as bidirectional.
- Control lines are used to transfer the read/write signals for memory read or write operation.
- Same lines are used to transfer the different values like control, data or address so system bus can be multiplexed. The system bus can act as address bus or data bus or control bus at a time. This method of selecting the bus for particular operation is called time multiplexing.
- Different types of control methods are used for communication they are.
 - Simple transfer
 - Strobe transfer
 - Single handshake transfer
 - Double handshake transfer

Simple Transfer

For simple transfer, memory and processor transfer the valid data by placing on data bus blindly.



Here cross line indicate the time for new valid data.

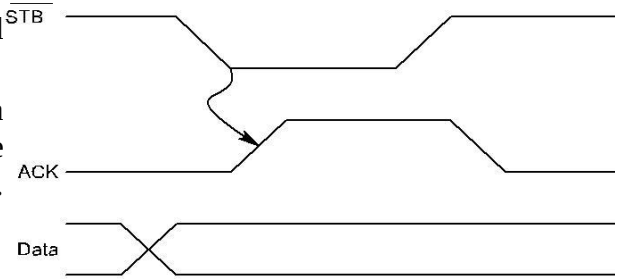
Strobe Transfer:

In this mode the transmitter transmit the data by placing valid data on data bus and raises the strobe pulse to indicate initiation of data transfer.



Single Handshake Transfer:

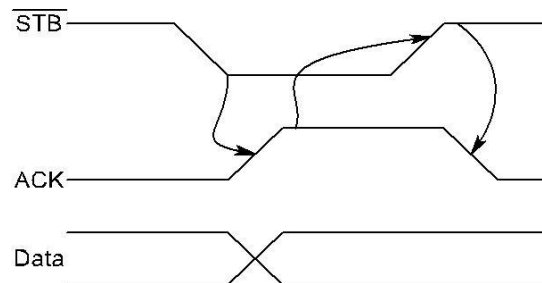
- The peripheral outputs some data and send $\overline{STB}$ signal to processor. —here is the data for you.
- Processor detects asserted $\overline{STB}$ signal, reads the data and sends an acknowledge signal (ACK) to indicate data has been read and peripheral can send next data. —I got that one, send me another.



- MP sends or receives data when peripheral is ready.

Double Handshake Transfer:

Fig: Single Handshaking

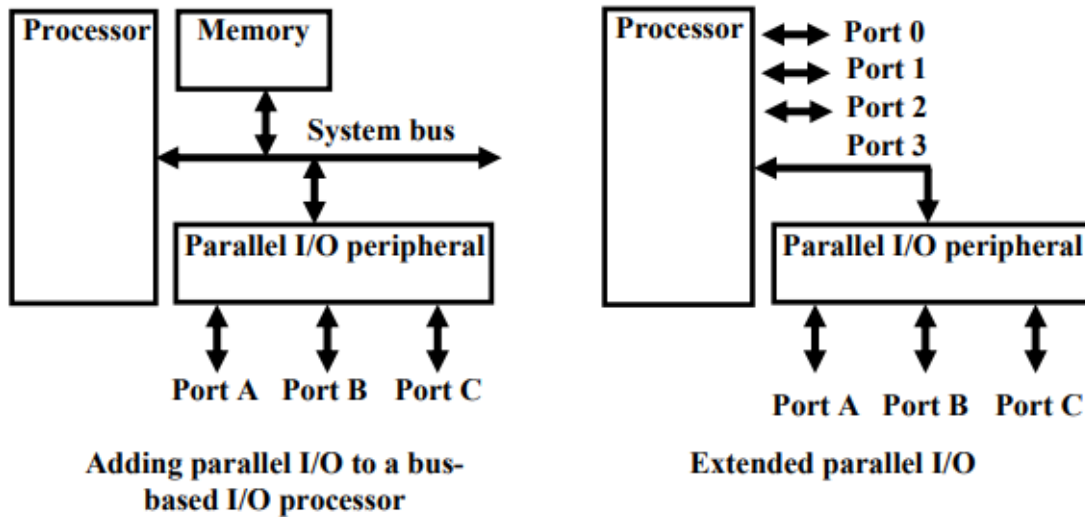


- The peripheral asserts its $\overline{STB}$ line low to ask processor —Are you ready?
- The processor raises its ACK line high to say — I am ready.
- Peripheral then sends data and raises its $\overline{STB}$ line low to say —Here is some valid data for you.
- Processor then reads the data and drops its ACK line to say, —I have the data, thank you, and I await your request to send the next byte of data.

I/O Addressing:

A microprocessor communicates with other devices using some of its pins. Broadly we can classify them as

- Port-based I/O (parallel I/O)
 - Processor has one or more N-bit ports
 - Processor's software reads and writes a port just like a register
- Bus-based I/O
 - Processor has address, data and control ports that form a single bus
 - Communication protocol is built into the processor
 - A single instruction carries out the read or write protocol on the bus
- Parallel I/O peripheral
 - When processor only supports bus-based I/O but parallel I/O needed.
 - Each port on peripheral connected to a register within peripheral that is read/written by the processor
- Extended parallel I/O
 - When processor supports port-based I/O but more ports needed
 - One or more processor ports interface with parallel I/O peripheral extending total number of ports available for I/O



Memory-mapped I/O and standard I/O

- Processor talks to both memory and peripherals using same bus – two ways to talk to peripherals
- Memory-mapped I/O
 - Peripheral registers occupy addresses in same address space as memory
 - e.g., Bus has 16-bit address
 - lower 32K addresses may correspond to memory
 - upper 32k addresses may correspond to peripherals
- Standard I/O (I/O-mapped I/O)
 - Additional pin (M/IO) on bus indicates whether a memory or peripheral access
 - e.g., Bus has 16-bit address
 - all 64K addresses correspond to memory when M/IO set to 0
 - all 64K addresses correspond to peripherals when M/IO set to 1

Memory-mapped I/O vs. Standard I/O

- Memory-mapped I/O
 - Requires no special instructions
 - Assembly instructions involving memory like MOV and ADD work with peripherals as well
 - Standard I/O requires special instructions (e.g., IN, OUT) to move data between peripheral registers and memory
- Standard I/O – No loss of memory addresses to peripherals
 - Simpler address decoding logic in peripherals possible
 - When number of peripherals much smaller than address space then high-order address bits can be ignored – smaller and/or faster comparators.

Address decoding:

Microprocessor is connected with memory and I/O devices via common address and data bus. Only one device can send data at a time and other devices can only receive that data. If more than one device sends data at the same time, the data gets garbled. In order to avoid this situation, ensuring that the proper device gets addressed at proper time, the technique called address decoding is used.

In address decoding method, all devices like memory blocks, I/O units etc. are assigned with a specific address. The address of the device is determined from the way in which the address lines are used to derive a special device selection signal k/a chip select (CS). If the microprocessor has to write or to read from a device, the CS signal to that block should be enabled and the address decoding circuit must ensure that CS signal to other devices are not activated. Depending upon the no. of address lines used to generate chip select signal for the device, the address decoding is classified as:

1. I/O mapped I/O

In this method, a device is identified with an 8 bit address and operated by I/O related functions IN and OUT for that IO/M =1. Since only 8 bit address is used, at most 256 bytes can be identified uniquely. Generally low order address bits A_0 - A_7 are used and upper bits A_8 - A_{15} are considered don't care. Usually I/O mapped I/O is used to map devices like 8255A, 8251A etc.

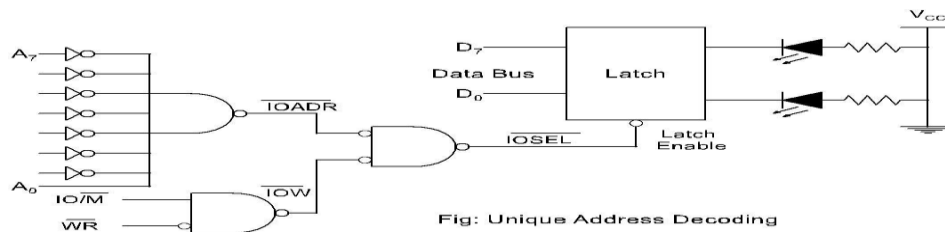
2. Memory mapped I/O

In this method, a device is identified with 16 bit address and enabled memory related functions such as STA, LDA for which IO/M =0, here chip select signal of each device is derived from 16 bit address lines thus total addressing capability is 64K bytes. Usually memory mapped I/O is used to map memories like RAM, ROM etc.

Depending on the address that are allocated to the device the address decoding are categorized in the following two groups.

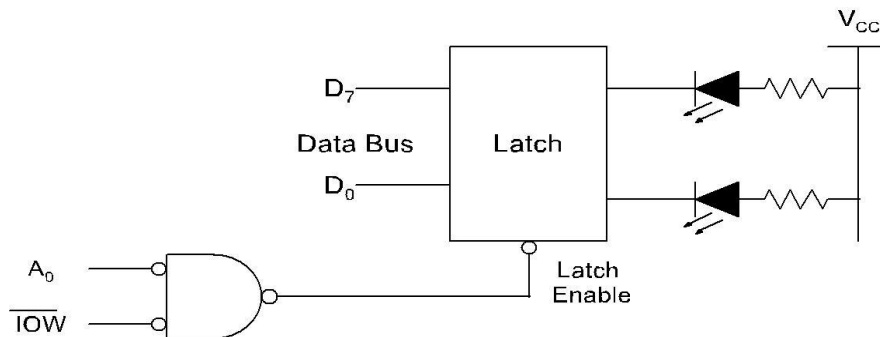
1. Unique Address Decoding:

If all the address lines on that mapping mode are used for address decoding then that decoding is called unique address decoding. It means all 8-lines in I/O mapped I/O and all 16 lines in memory mapped I/O are used to derive $\overline{CS}$ signal. It is expensive and complicated but fault proof in all cases.



2. Non Unique Address decoding:

If all the address lines available on that mode are not used in address decoding then that decoding is called non unique address decoding. Though it is cheaper there may be a chance of address conflict.



- If A_0 is low and $\overline{IOW}$ is low. Then latch gets enabled. Here A_1-A_7 is neglected that is any even address can enable the latch.

Interrupts:

- Interrupt is signals send by an external device to the processor, to request the processor to perform a particular task or work.
- Mainly in the microprocessor based system the interrupts are used for data transfer between the peripheral and the microprocessor.
- The processor will check the interrupts always at the 2nd T-state of last machine cycle.
- If there is any interrupt it accept the interrupt and send the INTA (active low) signal to the peripheral.
- The vectored address of particular interrupt is stored in program counter.
- The processor executes an interrupt service routine (ISR) addressed in program counter.
- It returned to main program by RET instruction.

Types of Interrupts

Interrupts can be broadly classified as

- Hardware Interrupts
 - These are interrupts caused by the connected devices.
- Software Interrupts
 - These are interrupts deliberately introduced by software instructions to generate user defined exceptions
- Trap
 - These are interrupts used by the processor alone to detect any exception such as divide by zero.

Depending on the service the interrupts also can be classified as:

- Fixed interrupt
 - Address of the ISR built into microprocessor, cannot be changed
 - Either ISR stored at address or a jump to actual ISR stored if not enough bytes available
- Vectored interrupt
 - Peripheral must provide the address of the ISR
 - Common when microprocessor has multiple peripherals connected by a system bus
- Compromise between fixed and vectored interrupts
 - One interrupt pin
 - Table in memory holding ISR addresses (maybe 256 words)
 - Peripheral doesn't provide ISR address, but rather index into table
 - Fewer bits are sent by the peripheral
 - Can move ISR location without changing peripheral

Maskable vs. Non-maskable interrupts

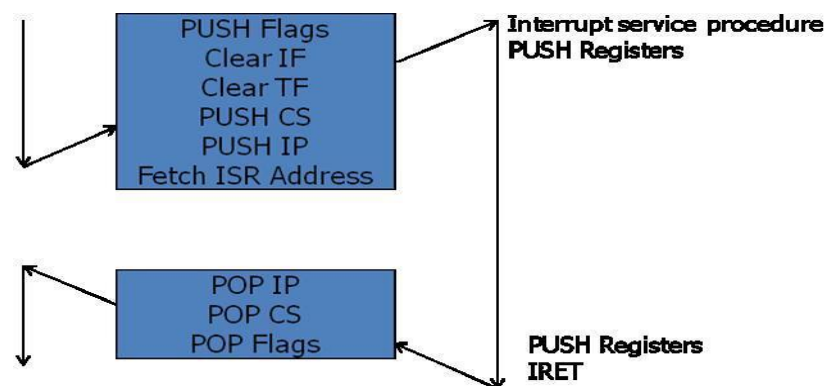
- Maskable:
 - programmer can set bit that causes processor to ignore interrupt
 - This is important when the processor is executing a time-critical code
- Non-maskable:
 - a separate interrupt pin that can't be masked
 - Typically reserved for drastic situations, like power failure requiring immediate backup of data to non-volatile memory

Process of interrupt Operation

- From the point of view of I/O unit
 - I/O device receives command from CPU
 - The I/O device then processes the operation
 - The I/O device signals an interrupt to the CPU over a control line.
 - The I/O device waits until the request from CPU.
- From the point of view of processor
 - The CPU issues command and then goes off to do its work.
 - When the interrupt from I/O device occurs, the processor saves its program counter & registers of the current program and processes the interrupt.
 - After completion for interrupt, processor requires its initial task.

Basic Interrupt Processing

The occurrence of interrupt triggers a number of events, both in processor hardware and in software. The interrupt driven I/O operation takes the following steps.



- The I/O unit issues an interrupt signal to the processor for exchange of data between them.
- The processor finishes execution of the current instruction before responding to the interrupt.
- The processor sends an acknowledgement signal to the device that it issued the interrupt.
- The processor transfers its control to the requested routine called —Interrupt Service Routine (ISR) by saving the contents of program status word (PSW) and program counter (PC).
- The processor now loads the PC with the location of interrupt service routine and the fetches the instructions. The result is transferred to the interrupt handler program.

- When interrupt processing is completed, the saved register's value are retrieved from the stack and restored to the register.
- Finally it restores the PSW and PC values from the stack.

The figure summarizes these steps. The processor pushes the flag register on the stack, disables the INTR input and does essentially an indirect call to the interrupt service procedure. An IRET function at the end of interrupt service procedure returns execution to the main program.

Direct Memory Access (DMA)

During any given bus cycle, one of the system components connected to the system bus is given control of the bus. This component is said to be the master during that cycle and the component it is communicating with is said to be the slave. The CPU with its bus control logic is normally the master, but other specially designed components can gain control of the bus by sending a bus request to the CPU. After the current bus cycle is completed the CPU will return a bus grant signal and the component sending the request will become the master.

The process of transferring the data in between memory and IO called as direct memory access (DMA). It is done by a controller called as DMA controller.

DMA Interfacing with Processor:

- DMA request signal is given from I/O device to DMA controller.
- DMA sends the bus request signal to CPU in response to which CPU disables its current instructions and initialize the DMA by sending the following information.
 - The starting address of the memory block where the data are available (for read) and where data to be stored (for write)
 - The word count which is the number of words in the memory block
 - Control to specify the mode of transfer
 - Sends a bus grant as 1 so that DMA controller can take the control of the buses
 - DMA sends the DMA acknowledge signal in response to which peripheral device puts the words in the data bus (for write) or receives a word from the data bus (for read).

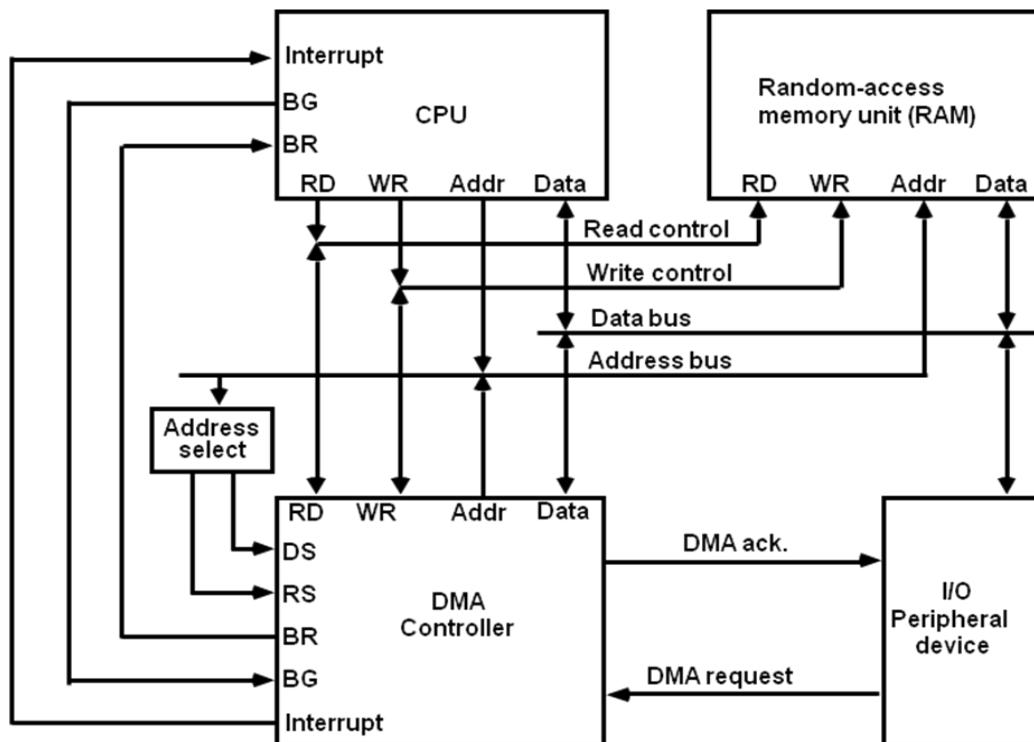


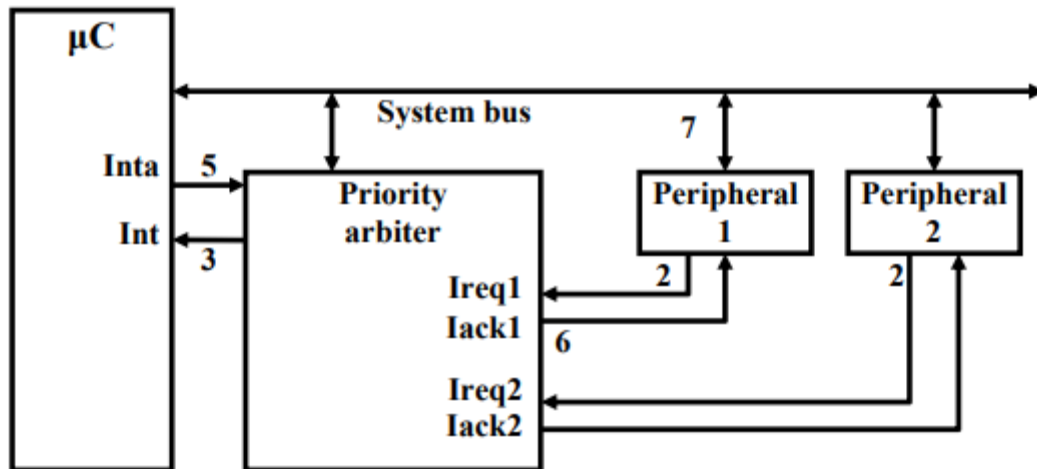
Fig: DMA transfer in a computer system

The Bus Arbitration:

- The device that is allowed to initiate transfers on the bus at any given time is called the bus master.
- When the current bus master relinquishes its status as the bus master, another device can acquire this status.
- The process by which the next device to become the bus master is selected and bus mastership is transferred to it is called bus arbitration.
- When there are more than one device need interrupt service then they have to be connected in specific manner. The processor responds to each one of them. This is called Arbitration. The method can be divided into following.
 - Priority Arbiter
 - Daisy Chain Arbiter

Priority Arbiter

Peripheral devices are connected with the processor and a priority arbiter is attached with processor selects the connection of peripheral with bus on the basis of fixed priority basis or round robin approach. If arbiter selects the PDs in fixed priority basis then such arbiter is called fixed priority arbiter. The round robin arbiter assigns a bus connection with PDs for a fixed interval of time and transfer to next connected device. The service taken by a device gets bus after a complete round. For example:



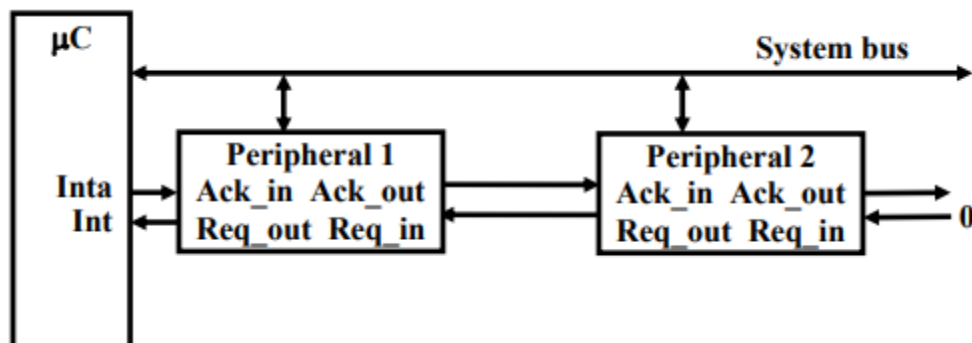
Let us assume that the Priority of the devices are Device1 > Device 2then priority arbitrer works as:

- The Processor is executing its program.
- Peripheral1 needs servicing so asserts **Ireq1**. Peripheral2 also needs servicing so asserts **Ireq2**.
- Priority arbitrer sees at least one **Ireq** input asserted, so asserts **Int**.
- Processor stops executing its program and stores its state.
- Processor asserts **Inta**.
- Priority arbitrer asserts **Iack1** to acknowledge Peripheral1.
- Peripheral1 puts its interrupt address vector on the system bus
- Processor jumps to the address of ISR read from data bus, ISR executes and returns (and completes handshake with arbitrer).

Thus in case of simultaneous interrupts the device with the highest priority will be served.

Daisy Chain Arbitrer

In this case the peripherals needing interrupt service are connected in a chain as shown in Fig. The requests are chained and hence any device interrupting shall be transmitted to the CPU in a chain.



Let us assume that the Priority of the devices are Device1 > Device 2 ... then daisy chain arbitrer works as:

- The Processor is executing its program.

- Any Peripheral needs servicing asserts **Req out**. This **Req out** goes to the **Req in** of the subsequent device in the chain
- Thus the peripheral nearest to the μC asserts Int.
- The processor stops executing its program and stores its state.
- Processor asserts **Inta** the nearest device.
- The **Inta** passes through the chain till it finds a flag which is set by the device which has generated the interrupt.
- The interrupting device sends the Interrupt Address Vector to the processor for its interrupt service subroutine.
- The processor jumps to the address of ISR read from data bus, ISR executes and returns.
- The flag is reset.

The processor now check for the next device which has interrupted simultaneously. In this case the device nearest to the processor has the highest priority.

Network-oriented arbitration

When multiple microprocessors share a bus (sometimes called a network) and Arbitration typically built into bus protocol. Separate processors may try to write simultaneously causing collisions such that

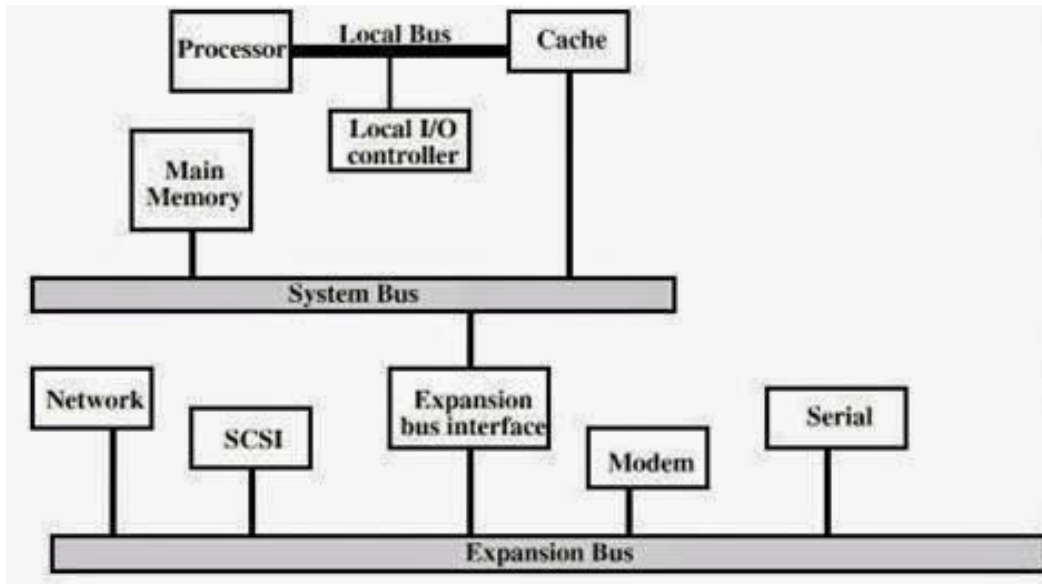
- Data must be resent
- Don't want to start sending again at same time

It is typically used for connecting multiple distant chips.

Multi-level Bus Hierarchy:

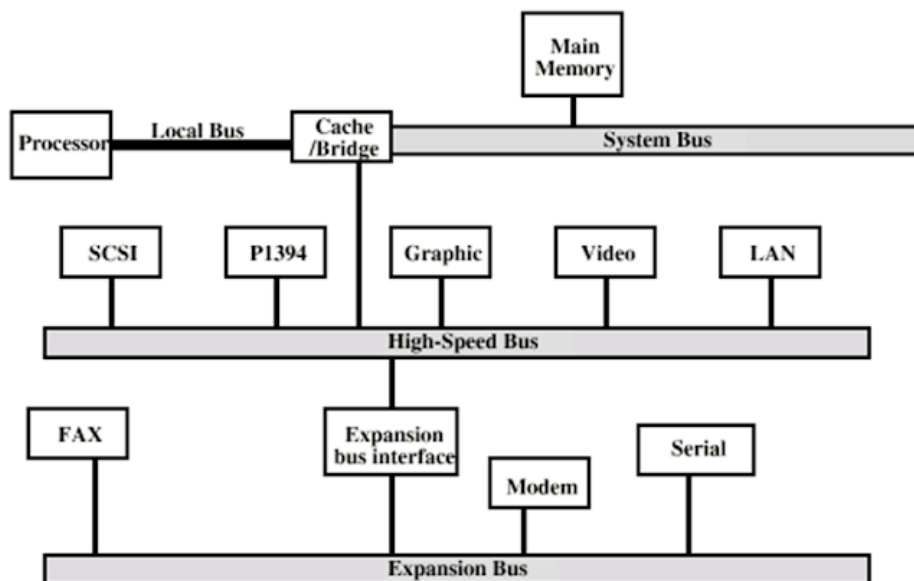
A great number of devices on a bus will cause performance to suffer. The ordinary system consists of a system bus that connects the multiple number of PDs. The longer bus suffers from the propagation delay problem. The time it takes for devices to coordinate the use of the bus. The bus may become a bottleneck as the aggregate data transfer demand approaches the capacity of the bus and multilevel bus hierarchy is comes to play.

- **Traditional Hierarchical Bus Architecture**
 - Use of a cache structure insulates CPU from frequent accesses to main memory.
 - Main memory can be moved off local bus to a system bus.
 - Expansion bus interface buffers data transfers between system bus and I/O controllers on expansion.



- High-Performance Hierarchical Bus Architecture

- Incorporates a high-speed bus specifically designed to support high-capacity I/O devices.
- Bring high-demand devices into closer integration with processor.



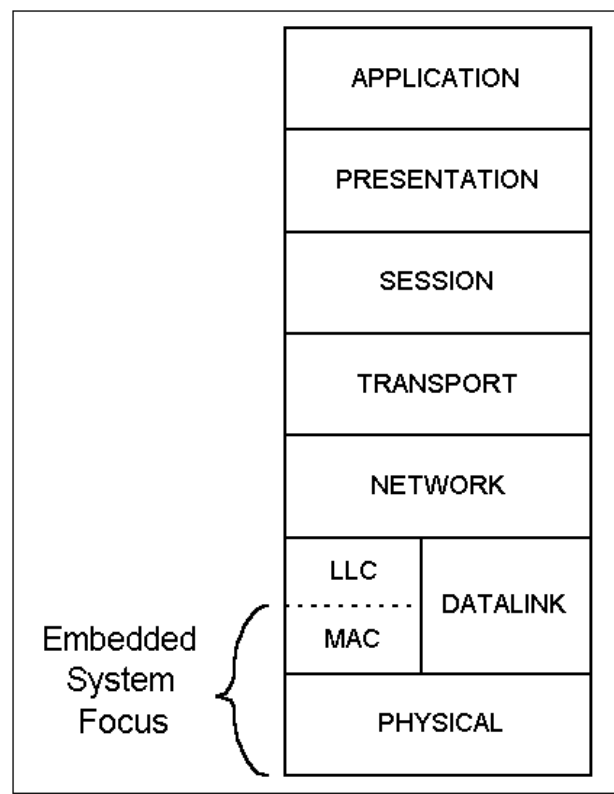
Advanced Communication Principles

- Embedded & Real-time systems could be standalone or connected
- A real-time system is often composed from a number of *periodic* (time triggered) and *sporadic* (event triggered) tasks which communicate their result by passing messages.
- In a distributed real-time systems these messages are sometimes sent between processors across a communication device.

- To guarantee that the timing requirements of all tasks are met, the communications delay between a sending task and a receiving task being able to access that message must be bounded.
- For examples
 - Control systems: between sensors and actuators via central computer
 - Multiprocessors: between processors, tasks communicating
-

Open System Interconnection

- Break complexity of communication protocol into pieces easier to design and understand
- Lower levels provide services to higher level
 - Lower level might work with bits while higher level might work with packets of data
- Physical layer
 - Lowest level in hierarchy
 - Medium to carry data from one actor (device or node) to another



- Parallel communication
 - Physical layer capable of transporting multiple bits of data
- Serial communication
 - Physical layer transports one bit of data at a time

- Wireless communication
 - No physical connection needed for transport at physical layer

Parallel communication

- Multiple data, control, and possibly power wires
 - One bit per wire
- High data throughput with short distances
- Typically used when connecting devices on same IC or same circuit board
 - Bus must be kept short
 - long parallel wires result in high capacitance values which requires more time to charge/discharge
 - Data misalignment between wires increases as length increases
- Higher cost, bulky

Serial communication

- Single data wire, possibly also control and power wires
- Words transmitted one bit at a time
- Higher data throughput with long distances
 - Less average capacitance, so more bits per unit of time
- Cheaper, less bulky
- More complex interfacing logic and communication protocol
 - Sender needs to decompose word into bits
 - Receiver needs to recombine bits into word
 - Control signals often sent on same wire as data increasing protocol complexity

Wireless communication

- Infrared (IR)
 - Electronic wave frequencies just below visible light spectrum
 - Diode emits infrared light to generate signal
 - Infrared transistor detects signal, conducts when exposed to infrared light
 - Cheap to build
 - Need line of sight, limited range
- Radio frequency (RF)

- Electromagnetic wave frequencies in radio spectrum
- Analog circuitry and antenna needed on both sides of transmission
- Line of sight not needed, transmitter power determines range

Error detection and correction

- Often part of bus protocol
- Error detection: ability of receiver to detect errors during transmission
- Error correction: ability of receiver and transmitter to cooperate to correct problem
 - Typically done by acknowledgement/retransmission protocol
- Bit error: single bit is inverted
- Burst of bit error: consecutive bits received incorrectly
- Parity: extra bit sent with word used for error detection
 - Odd parity: data word plus parity bit contains odd number of 1's
 - Even parity: data word plus parity bit contains even number of 1's
 - Always detects single bit errors, but not all burst bit errors
- Checksum: extra word sent with data packet of multiple words
 - e.g., extra word contains XOR sum of all data words in packet

Chapter- 5

Real Time Operating System (RTOS)

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond to an input and display of required updated information is termed as the **response time**. So in this method, the response time is very less as compared to online processing.

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail. For example, scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

There are two types of real-time operating systems.

- **Hard real-time systems**

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.

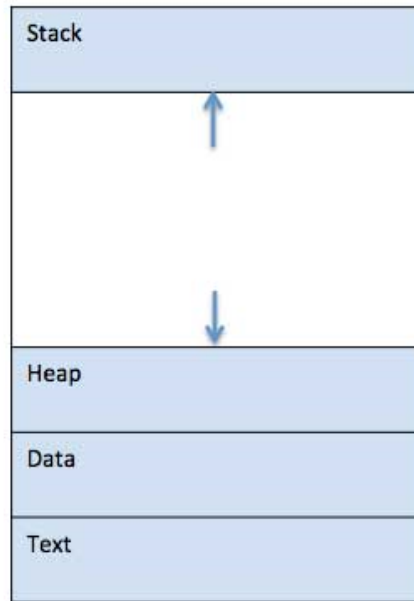
- **Soft real-time systems**

Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

Definition of Process, Task and Thread

Process:

- A process is basically a program in execution. The execution of a process must progress in a sequential fashion.
- A process is defined as an entity which represents the basic unit of work to be implemented in the system.
- To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
- Every process has its own address space.
- When a program is loaded into the memory and it becomes a process, it can be divided into four sections — stack, heap, text and data. The following image shows a simplified layout of a process inside main memory –



- **Stack:** The process Stack contains the temporary data such as method/function parameters, return address and local variables.
- **Heap:** This is dynamically allocated memory to a process during its run time.
- **Text:** This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
- **Data:** This section contains the global and static variables.

Task:

- A job is a unit of work that is scheduled and executed by a system
 - E.g. computation of a control-law, computation of an FFT on sensor data, transmission of a data packet, retrieval of a file
- A task is a set of related jobs which jointly provide some function
 - E.g. the set of jobs that constitute the “maintain constant altitude” task, keeping an airplane flying at a constant altitude
- The embedded system uses the three types of task as:
 - Periodic Task
 - Aperiodic Task
 - Sporadic Task

Periodic Task:

- The release time of job is known before its event triggering and repeated over the fixed interval of time called as period is known as periodic task and corresponding workload model is called as Periodic task model.
- A periodic task T_i be defined as $T_i = (\phi_i, p_i, e_i, D_i)$ where T_i refers a periodic task with phase ϕ_i , period p_i , execution time e_i , and relative deadline D_i .
- Default phase of T_i is $\phi_i = 0$, default relative deadline is the period $D_i = p_i$.
- Omit elements of the tuple that have default values.

Example:

i) $T_1 = (1, 10, 3, 6) \Rightarrow \phi_1 = 1, p_1 = 10, e_1 = 3, D_1 = 6$

$J_{1,1}$ released at 1, deadline 7 and $J_{1,2}$ released at 11, deadline 17.

ii) $T_2 = (10, 3, 6) \Rightarrow \phi_2 = 0, p_2 = 10, e_2 = 3, D_2 = 6$

$J_{2,1}$ released at 0, deadline 6 and $J_{2,2}$ released at 10 and so on.... deadline 16

iii) $T_3 = (10, 3) \Rightarrow \phi_3 = 0, p_3 = 10, e_3 = 3, D_3 = 10$.

$J_{3,1}$ released at 0, deadline 10 and $J_{3,2}$ released at 10, deadline 20.

Sporadic and Aperiodic Task:

Most embedded systems have to respond to external events which occur randomly. When such an event occurs the system executes a set of jobs in response. The release times of those jobs are not known until the event triggering them occurs. These jobs are called sporadic jobs or aperiodic jobs because they are released at random times.

If the tasks containing jobs that are released at random time instants and have hard deadlines then they are called sporadic task. Sporadic tasks are treated as hard real-time tasks. To ensure that their deadlines are met is the primary concern whereas minimizing their response times is of secondary importance. For example,

- An autopilot is required to respond to a pilot's command to disengage the autopilot and switch to manual control within a specified time.
- A fault tolerant system may be required to detect a fault and recover from it in time to prevent disaster

When the task or job have no any deadlines or soft deadline then it is called aperiodic task or job. For example,

- An operator adjusts the sensitivity of a radar system. The radar must continue to operate and in the near future change its sensitivity.

Threads

- A thread is a simple program that thinks it has the CPU all to itself. The design process for a real-time application involves splitting the work to be done into threads which are responsible for a portion of the problem. Each thread is assigned a priority, its own set of CPU registers and its own stack area.
- Each thread is typically an infinite loop that can be in one of four states: READY, RUNNING, WAITING or INTERRUPTED.

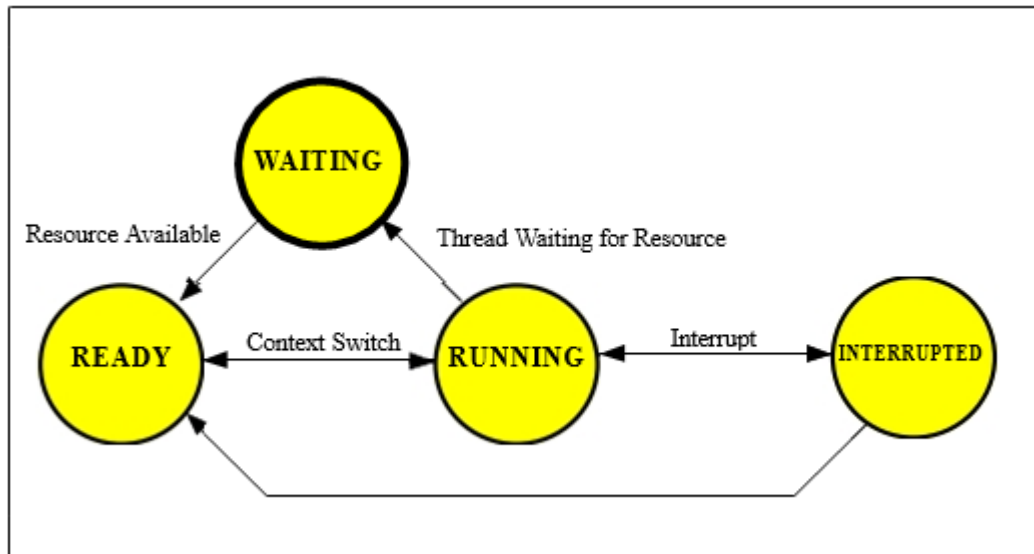
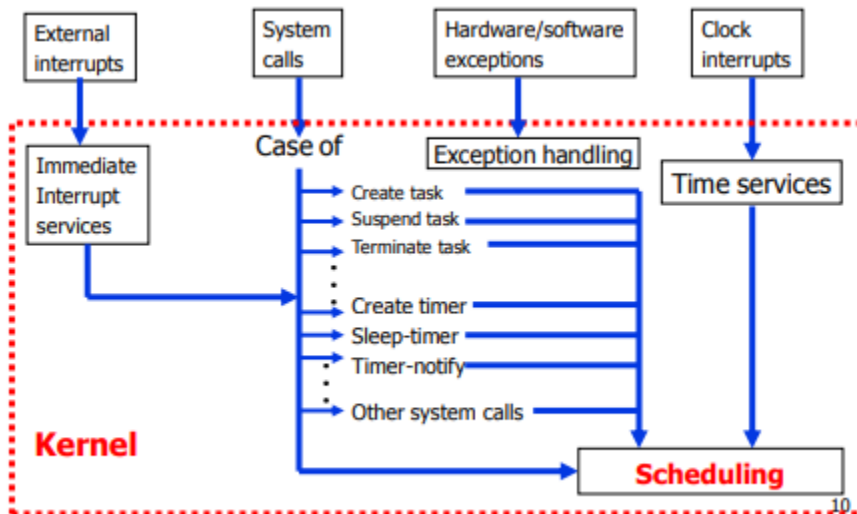


Figure – Thread states

- A thread is **READY** when it can execute but its priority is less than the current running thread.
- A thread is **RUNNING** when it has control of the CPU.
- A thread is **WAITING** when the thread suspends itself until a certain amount of time has elapsed, or when it requires the occurrence of an event: waiting for an I/O operation to complete, a shared resource to be available, a timing pulse to occur etc.
- Finally, a thread is **INTERRUPTED** when an interrupt occurred and the CPU is in the process of servicing the interrupt.

Real-Time Kernel Concepts:

- In most cases the real time os is an operating system kernel.
- An embedded system is designed for a single purpose so the user shell and file/disk access features are unnecessary.
- The kernel is the part of an OS that is responsible for the management of threads (i.e., managing the CPU's time) and for communication between threads. The fundamental service provided by the kernel is context switching.
- RTOS Kernel has following functions:
 - Time management
 - Task management
 - Interrupt handling
 - Memory management
 - Exception handling
 - Task synchronization
 - Task scheduling



Time Management

A high resolution hardware timer is programmed to interrupt the processor at fixed rate called as Time interrupt. Each time interrupt is called a system tick (time resolution).

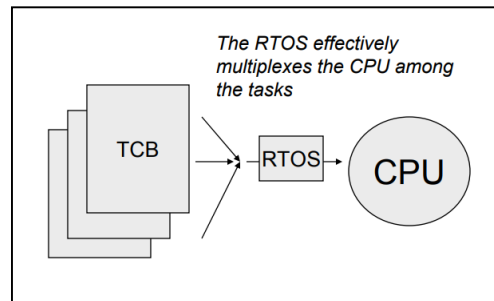
- Normally, the tick can vary in microseconds (depend on hardware)
- The tick may be selected by the user
- All time parameters for tasks should be the multiple of the tick
- Note: the tick may be chosen according to the given task parameters
- System time = 32 bits then
 - One tick = 1ms: your system can run 50 days
 - One tick = 20ms: your system can run 1000 days = 2.5 years
 - One tick = 50ms: your system can run 2500 days = 7 years

The time interrupt routine is used to serve the time interrupt and is a part of the RTOS kernel. Following operations are performed to serve the time interrupt by service routine.

- Save the context of the task in execution v
 - Increment the system time by 1, if current time > system lifetime, generate a timing error
 - Update timers (reduce each counter by 1)
 - Activation of periodic tasks in idling state
 - Schedule again - call the scheduler
 - Other functions e.g. v
 - (Remove all tasks terminated – de-allocate data structures e.g TCBs)
 - (Check if any deadline misses for hard tasks, monitoring)
 - load context for the first task in ready queue
- load context for the first task in ready queue

States of a Task in a system

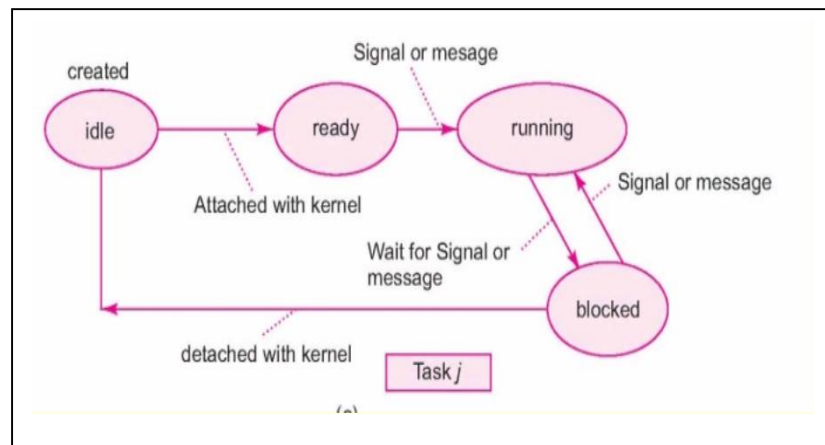
- A task is the combination of code, data and states (context) .Task State is stored in a Task Control Block (TCB) when the task is not running on the processor and the RTOS or kernel selects the task available on the different state as per need for operation.



The number of states required to process the task is deepens on the type and complexity of RTOS. The states of RTOS task are

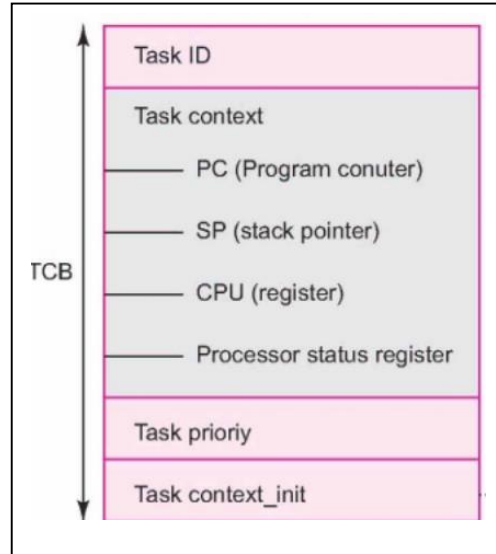
- Idle state
- Ready State
- Running state
- Blocked (waiting) state
- Deleted state

The finite state machine for task state is:



- **Idle (Created) State:** The task has been created and memory allotted to its structure. However, it is not ready and is not schedulable by kernel.
- **Ready (Active) State:** The created task is ready and is schedulable by the kernel but not running at present as another higher priority task is scheduled to run and gets the system resources at this instance.
- **Running state:** Executing the codes and getting the system resources at this instance. It will run till it needs some IPC (input) or wait for an event or till it gets preempted by another higher priority task than this one.
- **Blocked (waiting) state:** Execution of task codes suspends after saving the needed parameters into its context. It needs some IPC (input) or it needs to wait for an event or wait for higher priority task to block to enable running after blocking.
- **Deleted (finished) state:** The created task has memory de allotted to its structure i.e. task be deleted such that It frees the memory.

Task Control Block



- A data structure having the information using which the OS controls the process state.
- Task Information at the TCB are:

TaskID: The unique identifier used to define a task. For example, in case of 8-bit ID, a number between 0 and 255 can be used to define TaskID.

Task Context: It includes the current status of program counter, stack pointer, status of CPU register and Status Register.

Task priority: It stores the priority level of parent as well as child task available in Task List. The priority is a number used as the identifier.

Task Context_init: It is a pointer to the processor memory that stores following information.

- Allocated program memory address blocks in physical memory and in secondary (virtual) memory for the tasks-codes.
- Allocated task-specific data address blocks.
- Allocated task-stack addresses for the functions called during running of the process.
- Allocated addresses of CPU register-save area as a task context represents by CPU registers, which include the program counter and stack pointer.

Context Switch

When the multithreading kernel decides to run a different thread, it simply saves the current thread's context (CPU registers) in the current thread's context storage area (the thread control block, or TCB). Once this operation is performed, the new thread's context is restored from its TCB and the CPU resumes execution of the new thread's code. This process is called a context switch. Context switching adds overhead to the application.

Task Management:

The task management operation defines the following operations:

- Creation of new task with TCB.
- Task termination: remove the TCB
- Change Priority: modify the TCB
- State-inquiry: read the TCB

The major challenges for Task Management in RTOS kernel are:

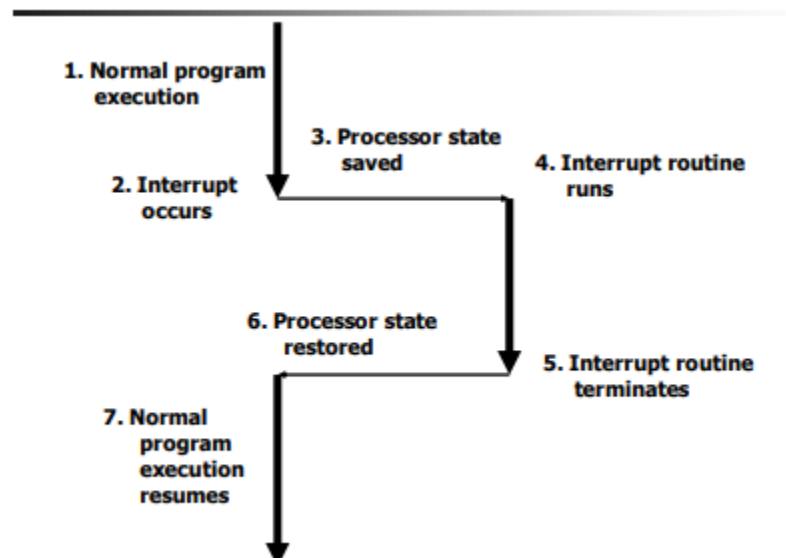
- Creating an RT task, it has to get the memory without delay: this is difficult because memory has to be allocated and a lot of data structures, code segment must be copied/initialized.
- Changing run-time priorities is dangerous: it may change the run-time behavior and predictability of the whole system.

Interrupt Handling:

An interrupt is a hardware mechanism used to inform the CPU that an asynchronous event has occurred. When an interrupt is recognized, the CPU saves all of its context (i.e., registers) and jumps to a special subroutine called an Interrupt Service Routine, or ISR. The ISR processes the event, and upon completion of the ISR, the program returns to:

- the background for a foreground / background system,
- the interrupted thread for a non-preemptive kernel, or
- The highest priority thread ready to run for a preemptive kernel.

Interrupts allow a microprocessor to process events when they occur. This prevents the microprocessor from continuously polling an event to see if it has occurred. Microprocessors allow interrupts to be ignored and recognized through the use of two special instructions: disable interrupts and enable interrupts, respectively.



The interrupt handlers handle the interrupt generated by external devices as below:

- The current context of the task is saved on stack.
- Block the task and branches the program control to beginning address of ISR and executes the ISR to serve the interrupt.
- Terminates from interrupt routine and read the context of the blocked task.

In a real-time environment, interrupts should be disabled as little as possible. Disabling interrupts affects interrupt latency and may cause interrupts to be missed. Processors generally allow interrupts to be nested. This means that while servicing an interrupt, the processor will recognize and service other (more important) interrupts, as shown in Figure below.

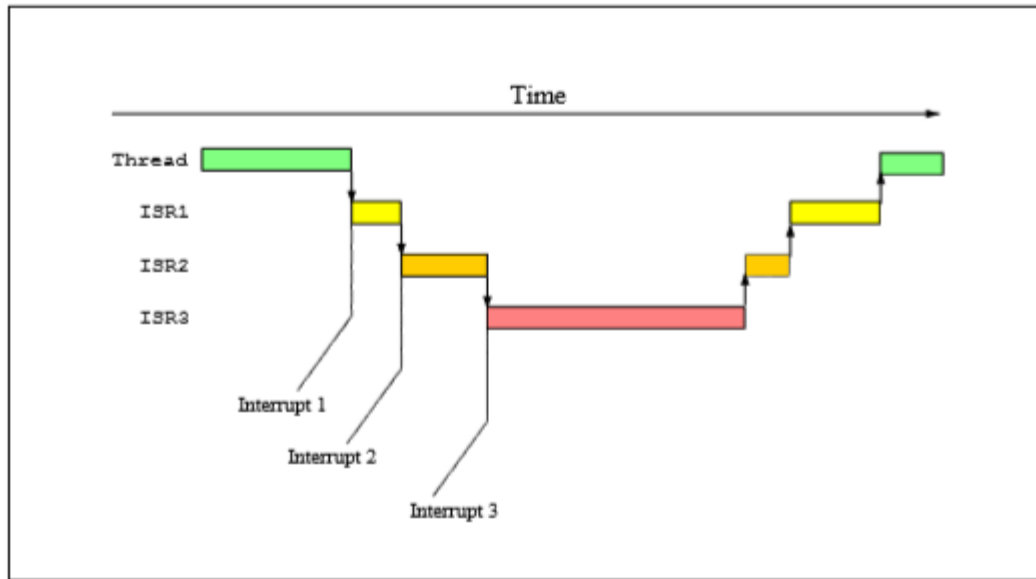


Figure – Interrupt nesting

Interrupt Latency

Probably the most important specification of a real-time kernel is the amount of time interrupts are disabled. All real-time systems disable interrupts to manipulate critical sections of code and renewable interrupts when the critical section has executed. The longer interrupts are disabled, the higher the interrupt latency. Interrupt latency is given by

Interrupt latency = Maximum amount of time interrupts are disabled + Time to start executing the first instruction in the ISR

Interrupt Response

Interrupt response is defined as the time between the reception of the interrupt and the start of the user code that handles the interrupt. The interrupt response time accounts for all the overhead involved in handling an interrupt.

For a foreground / background system, the user ISR code is executed immediately. The response time is given by

Interrupt recovery time = Time to execute the return from interrupt instruction

Interrupt Recovery

Interrupt recovery is defined as the time required for the processor to return to the interrupted code. Interrupt recovery in a foreground / background system simply involves restoring the processor's context and returning to the interrupted thread. Interrupt recovery is given by:

Interrupt recovery time = Time to execute the return from interrupt instruction

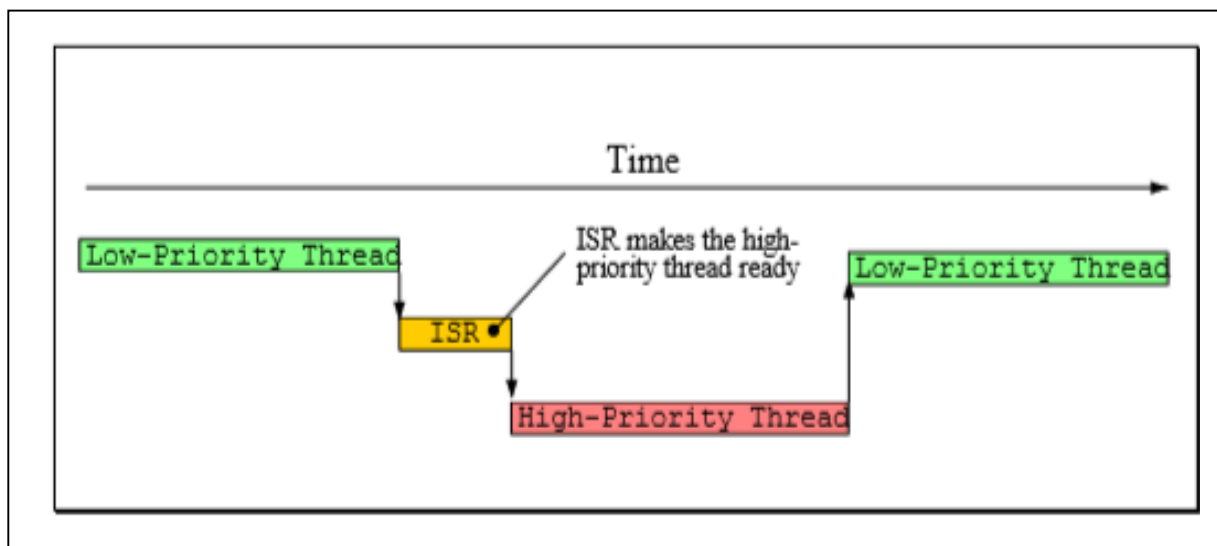
ISR Processing Time

Although ISRs should be as short as possible, there are no absolute limits on the amount of time for an ISR. One cannot say that an ISR must always be less than 100 ms, 500 ms, or 1 ms. If the ISR code is the most

important code that needs to run at any given time, it could be as long as it needs to be. In most cases, however, the ISR should recognize the interrupt, obtain data or a status from the interrupting device, and signal a thread to perform the actual processing.

Scheduler

The scheduler is the part of the kernel responsible for determining which thread will run next. Most real-time kernels are priority based. Each thread is assigned a priority based on its importance. Establishing the priority for each thread is application specific. In a priority-based kernel, control of the CPU will always be given to the highest priority thread ready to run. In a preemptive kernel, when a thread makes a higher priority thread ready to run, the current thread is pre-empted (suspended) and the higher priority thread is immediately given control of the CPU. If an interrupt service routine (ISR) makes a higher priority thread ready, then when the ISR is completed the interrupted thread is suspended and the new higher priority thread is resumed.



With a preemptive kernel, execution of the highest priority thread is deterministic; you can determine when the highest priority thread will get control of the CPU.

Application code using a preemptive kernel should not use non-reentrant functions, unless exclusive access to these functions is ensured through the use of mutual exclusion semaphores, because both a low- and a high-priority thread can use a common function. Corruption of data may occur if the higher priority thread preempts a lower priority thread that is using the function.

To summarize, a preemptive kernel always executes the highest priority thread that is ready to run. An interrupt preempts a thread. Upon completion of an ISR, the kernel resumes execution to the highest priority thread ready to run (not the interrupted thread). Thread-level response is optimum and deterministic.

Reentrancy

A reentrant function can be used by more than one thread without fear of data corruption. A reentrant function can be interrupted at any time and resumed at a later time without loss of data. Reentrant functions either use local variables (i.e., CPU registers or variables on the stack) or protect data when global variables are used. An example of a reentrant function is shown below:

```
char* strcpy(char* dst, const char* src)
{
    char* ptr = dst;
    while (*dst++ = *src++);
    return ptr;
}
```

Since copies of the arguments to `strcpy()` are placed on the thread's stack, and the local variable is created on the thread's stack, `strcpy()` can be invoked by multiple threads without fear that the threads will corrupt each other's pointers.

An example of a non-reentrant function is shown below:

```
static int Temp;

void swap(int* x, int* y)
{
    Temp = *x;
    *x = *y;
    *y = Temp;
}
```

`Swap ()` is a simple function that swaps the contents of its two arguments. Since `Temp` is a global variable, if the `swap ()` function gets preempted after the first line by a higher priority thread which also uses the `swap ()` function, then when the low priority thread resumes it will use the `Temp` value that was used by the high priority thread.

We can make `swap ()` reentrant with one of the following techniques:

- Declare `Temp` local to `swap ()`.
- Disable interrupts before the operation and enable them afterwards.
- Use a semaphore.

Thread Priority

A priority is assigned to each thread. The more important the thread, the higher the priority given to it.

- **Static Priorities**
Thread priorities are said to be static when the priority of each thread does not change during the application's execution. Each thread is thus given a fixed priority at compile time. All the threads and their timing constraints are known at compile time in a system where priorities are static
- **Dynamic Priorities**
Thread priorities are said to be dynamic if the priority of threads can be changed during the application's execution; each thread can change its priority at run time. This is a desirable feature to have in a real-time kernel to avoid priority inversions.
- **Priority Inversions**
Priority inversion is a problem in real-time systems and occurs mostly when you use a real-time kernel. Priority inversion is any situation in which a low priority thread holds a resource while a

higher priority thread is ready to use it. In this situation the low priority thread prevents the high priority thread from executing until it releases the resource.

To avoid priority inversion a multithreading kernel should change the priority of a thread automatically to help prevent priority inversions. This is called priority inheritance.

Mutual Exclusion

The easiest way for threads to communicate with each other is through shared data structures. This is especially easy when all threads exist in a single address space and can reference global variables, pointers, buffers, linked lists, FIFOs, etc. Although sharing data simplifies the exchange of information, we must ensure that each thread has exclusive access to the data to avoid contention and data corruption. The most common methods of obtaining exclusive access to shared resources are:

- disabling interrupts,
- performing test-and-set operations,
- disabling scheduling, and
- Using semaphores.

Semaphores

The semaphore was invented by Edgser Dijkstra in the mid-1960s. It is a protocol mechanism offered by most multithreading kernels. Semaphores are used to:

- control access to a shared resource (mutual exclusion),
- signal the occurrence of an event, and
- Allow two threads to synchronize their activities.

A semaphore is a key that code acquires in order to continue execution. If the semaphore is already in use, the requesting thread is suspended until the semaphore is released by its current owner. In other words, the requesting thread says: "Give me the key. If someone else is using it, I am willing to wait for it!" There are two types of semaphores: binary semaphores and counting semaphores. As its name implies, a binary semaphore can only take two values: 0 or 1. A counting semaphore allows values between 0 and 255, 65535, or 4294967295, depending on whether the semaphore mechanism is implemented using 8, 16, or 32 bits, respectively. The actual size depends on the kernel used. Along with the semaphore's value, the kernel also needs to keep track of threads waiting for the semaphore's availability.

Generally, only three operations can be performed on a semaphore: Create (), Wait (), and Signal (). The initial value of the semaphore must be provided when the semaphore is initialized. The waiting list of threads is always initially empty.

A thread desiring the semaphore will perform a Wait () operation. If the semaphore is available (the semaphore value is greater than 0), the semaphore value is decremented and the thread continues execution. If the semaphore's value is 0, the thread performing a Wait () on the semaphore is placed in a waiting list. Most kernels allow you to specify a timeout; if the semaphore is not available within a certain amount of time, the requesting thread is made ready to run and an error code (indicating that a timeout has occurred) is returned to the caller.

A thread releases a semaphore by performing a Signal () operation. If no thread is waiting for the semaphore, the semaphore value is simply incremented. If any thread is waiting for the semaphore, however, one of the threads is made ready to run and the semaphore value is not incremented; the key is given to one of the threads waiting for it. Depending on the kernel, the thread that receives the semaphore is either:

- The highest priority thread waiting for the semaphore, or
- The first thread that requested the semaphore (First In First Out).

Some kernels have an option that allows you to choose either method when the semaphore is initialized. For the first option, if the readied thread has a higher priority than the current thread (the thread releasing the semaphore), a context switch occurs (with a preemptive kernel) and the higher priority thread resumes execution; the current thread is suspended until it again becomes the highest priority thread ready to run.

Following listing shows how you can share data using a semaphore. Any thread needing access to the same shared data calls `OS_SemaphoreWait()`, and when the thread is done with the data, the thread calls `OS_SemaphoreSignal()`. Both of these functions are described later. You should note that a semaphore is an object that needs to be initialized before it is used; for mutual exclusion, a semaphore is initialized to a value of 1. Using a semaphore to access shared data doesn't affect interrupt latency. If an ISR or the current thread makes a higher priority thread ready to run while accessing shared data, the higher priority thread executes immediately.

```
OS_ECB* SharedDataSemaphore;

void Function(void)
{
    OS_ERROR error;

    error = OS_SemaphoreWait(SharedDataSemaphore, 0);
    .
    .    // You can access shared data in here
    .    // (interrupts are recognized)
    .
    error = OS_SemaphoreSignal(SharedDataSemaphore);
}
```

Semaphores are especially useful when threads share I/O devices. Imagine what would happen if two threads were allowed to send characters to a printer at the same time. The printer would contain interleaved data from each thread. For instance, the printout from Thread 1 printing **"I am Thread 1!"** and Thread 2 printing **"I am Thread 2!"** could result in:

"I Ia amm T Threahread d1 !2!"

In this case, use a semaphore and initialize it to 1 (i.e., a binary semaphore). The rule is simple: to access the printer each thread first must obtain the resource's semaphore.

In this case, use a semaphore and initialize it to 1 (i.e., a binary semaphore). The rule is simple: to access the printer each thread first must obtain the resource's semaphore.

Figure below shows threads competing for a semaphore to gain exclusive access to the printer. Note that the semaphore is represented symbolically by a key, indicating that each thread must obtain this key to use the printer.

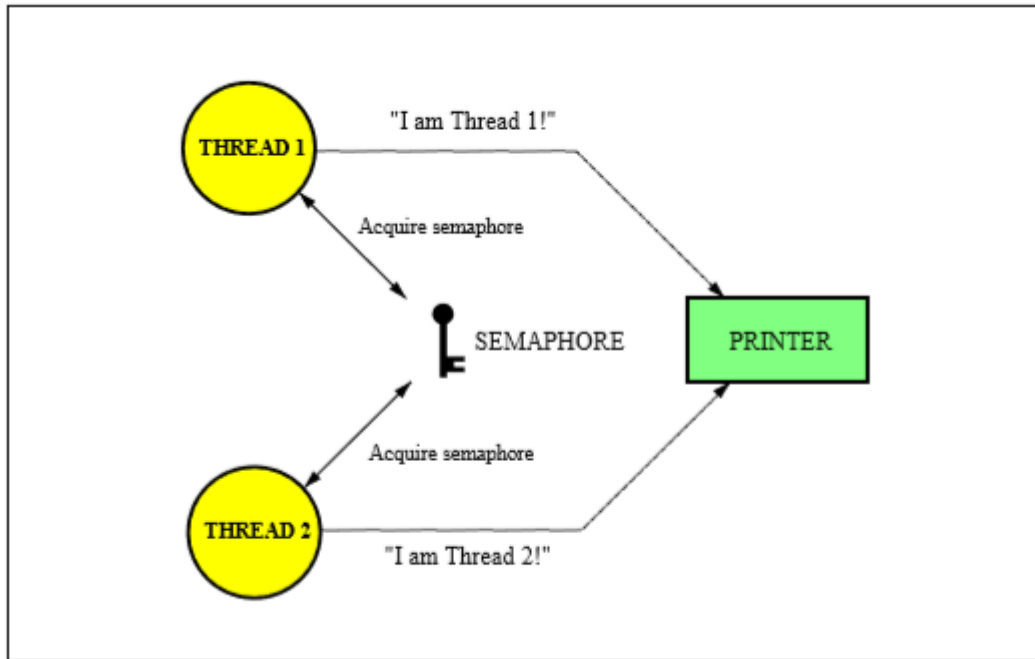


Figure – Using a semaphore to get permission to access a printer

The above example implies that each thread must know about the existence of the semaphore in order to access the resource. There are situations when it is better to encapsulate the semaphore. Each thread would thus not know that it is actually acquiring a semaphore when accessing the resource. For example, the UART port may be used by multiple threads to send commands and receive responses from a PC:

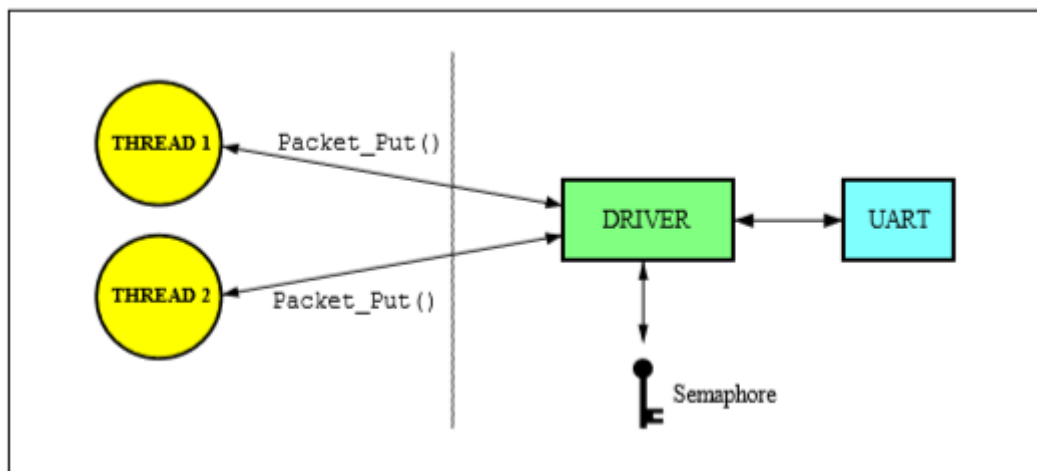


Figure – Hiding a semaphore from threads

The function `Packet_Put()` is called with two arguments: the packet and a timeout in case the device doesn't respond within a certain amount of time

Deadlock (or Deadly Embrace)

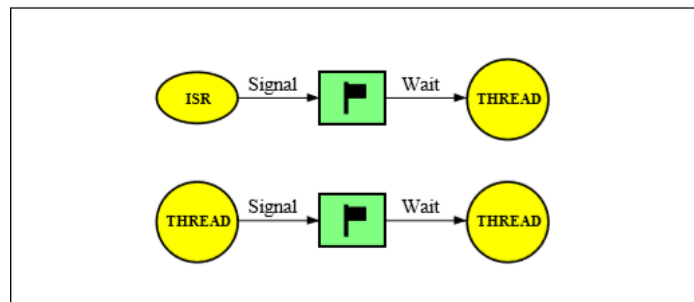
A deadlock, also called a deadly embrace, is a situation in which two threads are each unknowingly waiting for resources held by the other. Assume thread T1 has exclusive access to resource R1 and thread T2 has exclusive access to resource R2. If T1 needs exclusive access to R2 and T2 needs exclusive access to R1, neither thread can continue. They are deadlocked. The simplest way to avoid a deadlock is for threads to:

- acquire all resources before proceeding,
- acquire the resources in the same order, and
- release the resources in the reverse order

Most kernels allow to specify a timeout when acquiring a semaphore. This feature allows a deadlock to be broken. If the semaphore is not available within a certain amount of time, the thread requesting the resource resumes execution. Some form of error code must be returned to the thread to notify it that a timeout occurred. A return error code prevents the thread from thinking it has obtained the resource. Deadlocks generally occur in large multithreading systems, not in embedded systems.

Task Synchronization

A thread can be synchronized with an ISR (or another thread when no data is being exchanged) by using a semaphore as shown in Figure.



Note that, in this case, the semaphore is drawn as a flag to indicate that it is used to signal the occurrence of an event (rather than to ensure mutual exclusion, in which case it would be drawn as a key). When used as a synchronization mechanism, the semaphore is initialized to 0. Using a semaphore for this type of synchronization is called a **unilateral rendezvous**. A thread initiates an I/O operation and waits for the semaphore. When the I/O operation is complete, an ISR (or another thread) signals the semaphore and the thread is resumed.

If the kernel supports counting semaphores, the semaphore would accumulate events that have not yet been processed. Note that more than one thread can be waiting for an event to occur. In this case, the kernel could signal the occurrence of the event either to:

- the highest priority thread waiting for the event to occur or
- the first thread waiting for the event.

Depending on the application, more than one ISR or thread could signal the occurrence of the event. Two threads can synchronize their activities by using two semaphores, as shown in Figure below. This is called a bilateral rendezvous. A bilateral rendezvous is similar to a unilateral rendezvous, except both threads must synchronize with one another before proceeding.

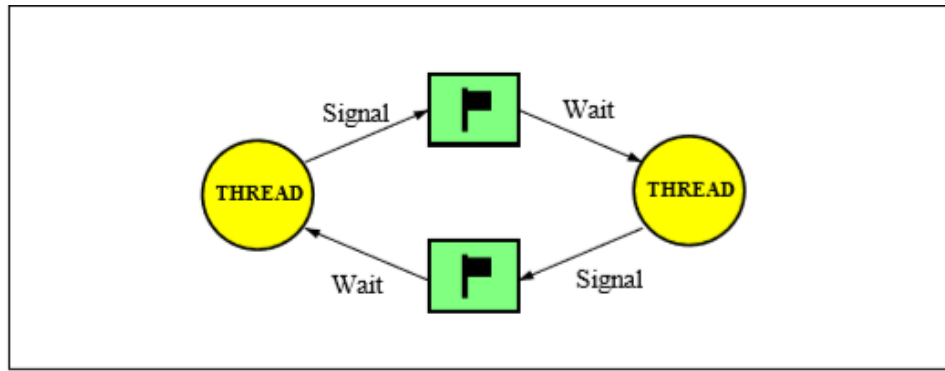


Figure – Threads synchronizing their activities

For example, two threads are executing as shown in Listing below. When the first thread reaches a certain point, it signals the second thread (1) then waits for a return signal (2). Similarly, when the second thread reaches a certain point, it signals the first thread (3) and waits for a return signal (4). At this point, both threads are synchronized with each other. A bilateral rendezvous cannot be performed between a thread and an ISR because an ISR cannot wait on a semaphore:

```

void Thread1(void)
{
    for (;;)
    {
        Perform operation 1;
        Signal thread #2;           (1)
        Wait for signal from thread #2; (2)
        Continue operation 1;
    }
}

void Thread2(void)
{
    for (;;)
    {
        Perform operation 2;
        Signal thread #1;           (3)
        Wait for signal from thread #1; (4)
        Continue operation 2;
    }
}
  
```

Interthread Communication

It is sometimes necessary for a thread or an ISR to communicate information to another thread. This information transfer is called interthread communication. Information may be communicated between threads in two ways: through global data or by sending messages.

When using global variables, each thread or ISR must ensure that it has exclusive access to the variables. If an ISR is involved, the only way to ensure exclusive access to the common variables is to disable interrupts. If two threads are sharing data, each can gain exclusive access to the variables either by disabling and enabling interrupts or with the use of a semaphore (as we have seen). Note that a thread can only communicate information to an ISR by using global variables. A thread is not aware when a global variable is changed by an ISR, unless the ISR signals the thread by using a semaphore or unless the thread polls the contents of the variable periodically.

To correct this situation, we should consider using either a message mailbox or a message queue.

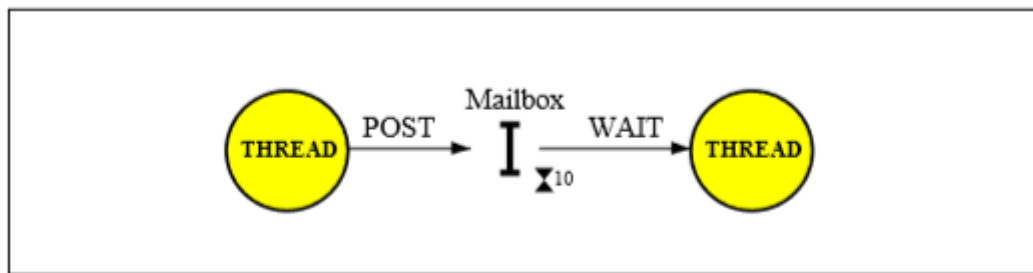


Figure – Message mailbox

Messages can be sent to a thread through kernel services. A Message Mailbox, also called a message exchange, is typically a pointer-size variable. Through a service provided by the kernel, a thread or an ISR can deposit a message (the pointer) into this mailbox. Similarly, one or more threads can receive messages through a service provided by the kernel. Both the sending thread and receiving thread agree on what the pointer is actually pointing to.

A waiting list is associated with each mailbox in case more than one thread wants to receive messages through the mailbox. Kernels typically provide the following mailbox services:

- Initialize the contents of a mailbox. The mailbox initially may or may not contain a message.
 - Deposit a message into the mailbox (POST).
 - Wait for a message to be deposited into the mailbox (WAIT).
 - Get a message from a mailbox if one is present, but do not suspend the caller if the mailbox is empty (ACCEPT). If the mailbox contains a message, the message is extracted from the mailbox.
- A return code is used to notify the caller about the outcome of the call.

Message mailboxes can also simulate binary semaphores. A message in the mailbox indicates that the resource is available, and an empty mailbox indicates that the resource is already in use by another thread.

Message Queues

A message queue is used to send one or more messages to a thread. A message queue is basically an array of mailboxes. Through a service provided by the kernel, a thread or an ISR can deposit a message (the pointer) into a message queue. Similarly, one or more threads can receive messages through a service provided by the kernel. Both the sending thread and receiving thread agree as to what the pointer is actually pointing to. Generally, the first message inserted in the queue will be the first message extracted from the queue (FIFO).

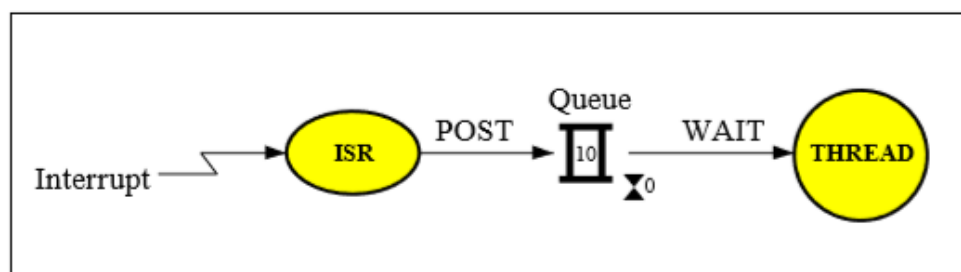


Figure – Message queue

As with the mailbox, a waiting list is associated with each message queue, in case more than one thread is to receive messages through the queue. A thread desiring a message from an empty queue is suspended and placed on the waiting list until a message is received. Typically, the kernel allows the thread waiting for a message to specify a timeout. If a message is not received before the timeout expires, the requesting thread is made ready to run and an error code (indicating a timeout has occurred) is returned to it. When a message is deposited into the queue, either the highest priority thread or the first thread to wait for the message is given the message. Kernels typically provide the message queue services listed below.

- Initialize the queue. The queue is always assumed to be empty after initialization.
- Deposit a message into the queue (POST).
- Wait for a message to be deposited into the queue (WAIT).
- Get a message from a queue if one is present, but do not suspend the caller if the queue is empty (ACCEPT). If the queue contains a message, the message is extracted from the queue. A return code is used to notify the caller about the outcome of the call.

Interrupts

Clock Tick

A clock tick is a special interrupt that occurs periodically. This interrupt can be viewed as the system's heartbeat. The time between interrupts is application specific and is generally between 1 and 200 ms. The clock tick interrupt allows a kernel to delay threads for an integral number of clock ticks and to provide timeouts when threads are waiting for events to occur. The faster the tick rate, the higher the overhead imposed on the system.

All kernels allow threads to be delayed for a certain number of clock ticks. The resolution of delayed threads is one clock tick; however, this does not mean that its accuracy is one clock tick.

Memory Requirement

If we are designing a foreground / background system, the amount of memory required depends solely on application code. With a multithreading kernel, things are quite different. To begin with, a kernel requires extra code space (Flash). The size of the kernel depends on many factors. Depending on the features provided by the kernel, we can expect anywhere from 1 to 100 KiB. A minimal kernel for a 32-bit CPU that provides only scheduling, context switching, semaphore management, delays, and timeouts should require about 1 to 3 KiB of code space.

Because each thread runs independently of the others, it must be provided with its own stack area (RAM). As a designer, you must determine the stack requirement of each thread as closely as possible (this is sometimes a difficult undertaking). The stack size must not only account for the thread requirements (local variables, function calls, etc.), it must also account for maximum interrupt nesting (saved registers, local storage in ISRs, etc.). Depending on the target processor and the kernel used, a separate stack can be used to handle all interrupt-level code. This is a desirable feature because the stack requirement.

For each thread can be substantially reduced. Another desirable feature is the ability to specify the stack size of each thread on an individual basis. Conversely, some kernels require that all thread stacks be the same size. All kernels require extra RAM to maintain internal variables, data structures, queues, etc. The total RAM required if the kernel does not support a separate interrupt stack is given by:

Total RAM requirements = Application code requirements + Data space (i.e., RAM) needed by the kernel + SUM (thread stacks + MAX (ISR nesting))

Unless we have large amounts of RAM to work with, you need to be careful how you use the stack space. To reduce the amount of RAM needed in an application, we must be careful how you use each thread's stack for:

- large arrays and structures declared locally to functions and ISRs,
- function (i.e., subroutine) nesting,
- interrupt nesting,
- library functions stack usage, and
- Function calls with many arguments.

To summarize, a multithreading system requires more code space (Flash) and data space (RAM) than a foreground / background system. The amount of extra Flash depends only on the size of the kernel, and the amount of RAM depends on the number of threads in system.

Typical Semaphore Use

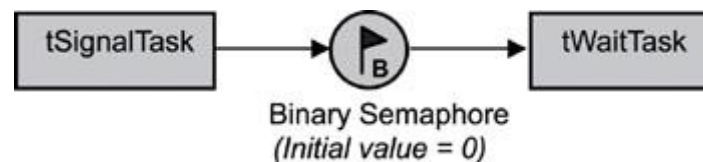
Semaphores are useful either for synchronizing execution of multiple tasks or for coordinating access to a shared resource. The following examples and general discussions illustrate using different types of semaphores to address common synchronization design requirements effectively, as listed:

- wait-and-signal synchronization,
- multiple-task wait-and-signal synchronization,
- credit-tracking synchronization,
- single shared-resource-access synchronization,
- recursive shared-resource-access synchronization, and
- multiple shared-resource-access synchronization.

Note that, for the sake of simplicity, not all uses of semaphores are listed here. Also, later chapters of this book contain more advanced discussions on the different ways that mutex semaphores can handle priority inversion.

Wait-and-Signal Synchronization

Two tasks can communicate for the purpose of synchronization without exchanging data. For example, a binary semaphore can be used between two tasks to coordinate the transfer of execution control, as shown in figure below.



Multiple-Task Wait-and-Signal Synchronization

When coordinating the synchronization of more than two tasks, use the flush operation on the task-waiting list of a binary semaphore, as shown in Figure below.

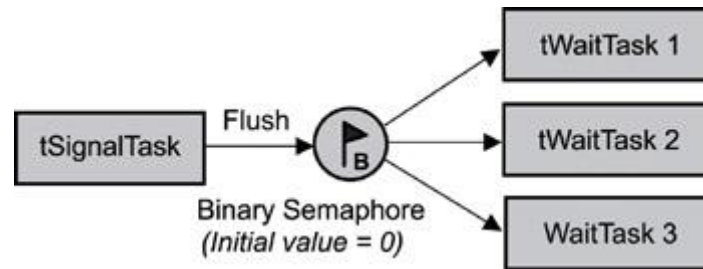


Figure: Wait-and-signal synchronization between multiple tasks.

As in the previous case, the binary semaphore is initially unavailable (value of 0). The higher priority **tWaitTasks** 1, 2, and 3 all do some processing; when they are done, they try to acquire the unavailable semaphore and, as a result, block. This action gives **tSignalTask** a chance to complete its processing and execute a flush command on the semaphore, effectively unblocking the three **tWaitTasks**.

Credit-Tracking Synchronization

Sometimes the rate at which the signaling task executes is higher than that of the signaled task. In this case, a mechanism is needed to count each signaling occurrence. The counting semaphore provides just this facility. With a counting semaphore, the signaling task can continue to execute and increment a count at its own pace, while the wait task, when unblocked, executes at its own pace, as shown in figure below.

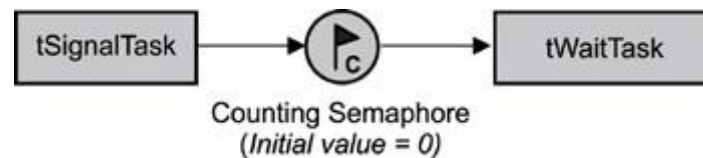


Figure : Credit-tracking synchronization between two tasks.

Again, the counting semaphore's count is initially 0, making it unavailable. The lower priority **tWaitTask** tries to acquire this semaphore but blocks until **tSignalTask** makes the semaphore available by performing a release on it. Even then, **tWaitTask** will wait in the ready state until the higher priority **tSignalTask** eventually relinquishes the CPU by making a blocking call or delaying itself.

Single Shared-Resource-Access Synchronization

One of the more common uses of semaphores is to provide for mutually exclusive access to a shared resource. A shared resource might be a memory location, a data structure, or an I/O device—essentially anything that might have to be shared between two or more concurrent threads of execution. A semaphore can be used to serialize access to a shared resource, as shown in figure below.

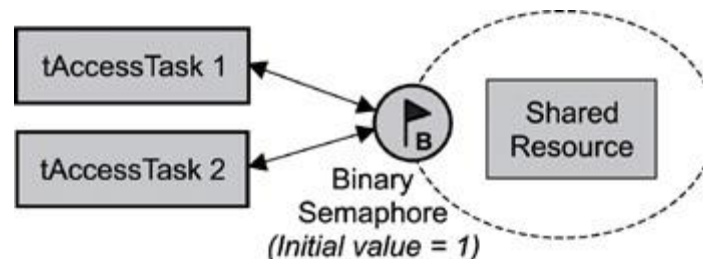


Figure: Single shared-resource-access synchronization.

In this scenario, a binary semaphore is initially created in the available state (value = 1) and is used to protect the shared resource. To access the shared resource, task 1 or 2 needs to first successfully acquire the binary semaphore before reading from or writing to the shared resource.

Recursive Shared-Resource-Access Synchronization

Sometimes a developer might want a task to access a shared resource recursively. This situation might exist if *tAccessTask* calls Routine A that calls Routine B, and all three need access to the same shared resource, as shown in figure below.

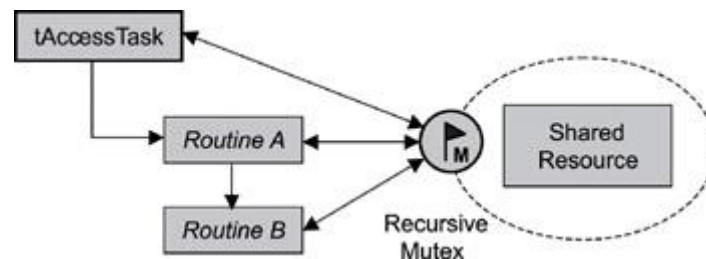


Figure: Recursive shared- resource-access synchronization.

If a semaphore were used in this scenario, the task would end up blocking, causing a deadlock. When a routine is called from a task, the routine effectively becomes a part of the task. When Routine A runs, therefore, it is running as a part of *tAccessTask*. Routine A trying to acquire the semaphore is effectively the same as *tAccessTask* trying to acquire the same semaphore. In this case, *tAccessTask* would end up blocking while waiting for the unavailable semaphore that it already has.

One solution to this situation is to use a recursive mutex. After *tAccessTask* locks the mutex, the task owns it. Additional attempts from the task itself or from routines that it calls to lock the mutex succeed. As a result, when Routines A and B attempt to lock the mutex, they succeed without blocking.

Multiple Shared-Resource-Access Synchronization

For cases in which multiple equivalent shared resources are used, a counting semaphore comes in handy, as shown in Figure

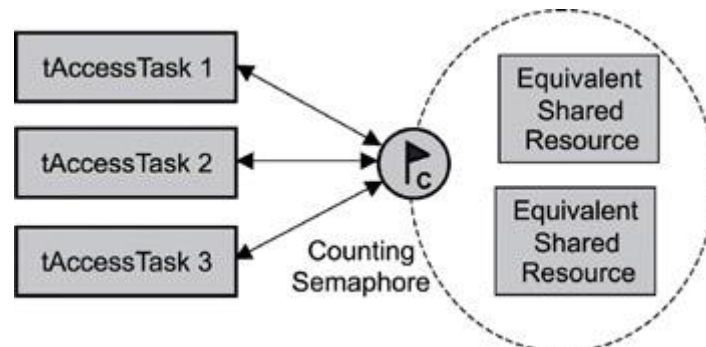


Figure: Single shared-resource-access synchronization.

Note that this scenario does not work if the shared resources are not equivalent. The counting semaphore's count is initially set to the number of equivalent shared resources: in this example, 2. As a result, the first two tasks requesting a semaphore token are successful. However, the third task ends up blocking until one of the previous two tasks releases a semaphore token.

Memory Management

Embedded systems developers commonly implement custom memory-management facilities on top of what the underlying RTOS provides. Understanding memory management is therefore an important aspect of developing for embedded systems.

Knowing the capability of the memory management system can aid application design and help avoid pitfalls. For example, in many existing embedded applications, the dynamic memory allocation routine, `malloc`, is called often. It can create an undesirable side effect called memory fragmentation. This generic memory allocation routine, depending on its implementation, might impact an application's performance. In addition, it might not support the allocation behavior required by the application.

Many embedded devices (such as PDAs, cell phones, and digital cameras) have a limited number of applications (tasks) that can run in parallel at any given time, but these devices have small amounts of physical memory onboard. Larger embedded devices (such as network routers and web servers) have more physical memory installed, but these embedded systems also tend to operate in a more dynamic environment, therefore making more demands on memory. Regardless of the type of embedded system, the common requirements placed on a memory management system are minimal fragmentation, minimal management overhead, and deterministic allocation time.

Dynamic Memory Allocation in Embedded Systems

It is known that the program code, program data, and system stack occupy the physical memory after program initialization completes. Either the RTOS or the kernel typically uses the remaining physical memory for dynamic memory allocation. This memory area is called the *heap*. Memory management in the context of this chapter refers to the management of a contiguous block of physical memory, although the concepts introduced in this apply to the management of non-contiguous memory blocks as well. These concepts also apply to the management of various types of physical memory. In general, a memory management facility maintains internal information for a heap in a reserved memory area called the *control block*. Typical internal information includes:

- the starting address of the physical memory block used for dynamic memory allocation,
- the overall size of this physical memory block, and
- the allocation table that indicates which memory areas are in use, which memory areas are free, and the size of each free region.

Memory Fragmentation and Compaction

In the example implementation, the heap is broken into small, fixed-size blocks. Each block has a unit size that is power of two to ease translating a requested size into the corresponding required number of units. In this example, the unit size is 32 bytes. The dynamic memory allocation function, `malloc`, has an input parameter that specifies the size of the allocation request in bytes. `malloc` allocates a larger block, which is made up of one or more of the smaller, fixed-size blocks. The size of this larger memory block is at least as large as the requested size; it is the closest to the multiple of the unit size. For example, if the allocation requests 100 bytes, the returned block has a size of 128 bytes (4 units x 32 bytes/unit). As a result, the requestor does not use 28 bytes of the allocated memory, which is called memory fragmentation. This specific form of fragmentation is called internal fragmentation because it is internal to the allocated block.

The allocation table can be represented as a bitmap, in which each bit represents a 32-byte unit. Figure shows the states of the allocation table after a series of invocations of the `malloc` and `free` functions. In this example, the heap is 256 bytes.



Figure: States of a memory allocation map.

Step 6 shows two free blocks of 32 bytes each. Step 7, instead of maintaining three separate free blocks, shows that all three blocks are combined to form a 128-byte block. Because these blocks have been combined, a future allocation request for 96 bytes should succeed.

Figure below shows another example of the state of an allocation table. Note that two free 32-byte blocks are shown. One block is at address 0x10080, and the other at address 0x101C0, which cannot be used for any memory allocation requests larger than 32 bytes. Because these isolated blocks do not contribute to the contiguous free space needed for a large allocation request, their existence makes it more likely that a large request will fail or take too long. The existence of these two trapped blocks is considered external fragmentation because the fragmentation exists in the table, not within the blocks themselves. One way to eliminate this type of fragmentation is to compact the area adjacent to these two blocks. The range of memory content from address 0x100A0 (immediately following the first free block) to address 0x101BF (immediately preceding the second free block) is shifted 32 bytes lower in memory, to the new range of 0x10080 to 0x1019F, which effectively combines the two free blocks into one 64-byte block. This new free block is still considered memory fragmentation if future allocations are potentially larger than 64 bytes. Therefore, memory compaction continues until all of the free blocks are combined into one large chunk.

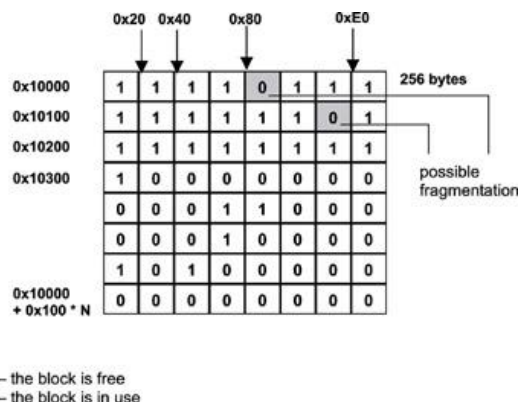


Figure : Memory allocation map with possible fragmentation.

Several problems occur with memory compaction. It is time-consuming to transfer memory content from one location to another. The cost of the copy operation depends on the length of the contiguous blocks in use. The tasks that currently hold ownership of those memory blocks are prevented from accessing the contents of those memory locations until the transfer operation completes. Memory compaction is almost never done in practice in embedded designs. The free memory blocks are combined only if they are immediate neighbors, as illustrated in Figure above.

Memory compaction is allowed if the tasks that own those memory blocks reference the blocks using virtual addresses. Memory compaction is not permitted if tasks hold physical addresses to the allocated memory blocks.

In many cases, memory management systems should also be concerned with architecture-specific memory alignment requirements. *Memory alignment* refers to architecture-specific constraints imposed on the address of a data item in memory. Many embedded processor architectures cannot access multi-byte data items at any address. For example, some architecture requires multi-byte data items, such as integers and long integers, to be allocated at addresses that are a power of two. Unaligned memory addresses result in bus errors and are the source of memory access exceptions.

Some conclusions can be drawn from this example. An efficient memory manager needs to perform the following chores quickly:

- Determine if a free block that is large enough exists to satisfy the allocation request. This work is part of the malloc operation.
- Update the internal management information. This work is part of both the malloc and free operations.
- Determine if the just-freed block can be combined with its neighboring free blocks to form a larger piece. This work is part of the free operation.

The structure of the allocation table is the key to efficient memory management because the structure determines how the operations listed earlier must be implemented. The allocation table is part of the overhead because it occupies memory space that is excluded from application use. Consequently, one other requirement is to minimize the management overhead.

Chapter -6

Embedded Software Development Tools

Follow the chapter -2 for development tools

The generic tool chain for development of embedded applications involves the integration of many tools. A brief description of these tools is explained as follows:

Cross Assemblers:

It converts source program written in the assembly language into the machine language of the target processor.

Cross assemblers are generally used to develop programs which are supposed to run on game consoles, appliances and other specialized small electronics systems which are not able to run a development environment. They can also be used to speed up development for low powered system, for example XAsm enables development on a PC based system for a Z80 powered MSX computer. Even though the MSX system is capable of running an assembler, having the additional memory, processor speed and storage capabilities like a hard disk significantly speeds up development efforts.

Cross Compilers:

It takes an expanded source program written in the high level language as an input and translates into equivalent assembly language program of a target processor.

A cross compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running. For example in order to compile for Linux/ARM you first need to obtain its libraries to compile against. A cross compiler is necessary to compile for multiple platforms from one machine. A platform could be infeasible for a compiler to run on, such as for the microcontroller of an embedded system because those systems contain no operating system. In para virtualization one machine runs many operating systems, and a cross compiler could generate an executable for each of them from one main source.

Locator

It does the linking and loading of the object programs. It is responsible for generating the executable program as needed by the target programmers. Locators take inputs that describe the target environment such as: memory layout of RAM/ROM/FLASH, system data structures, global descriptor Tables, interrupt structures, etc. and generate absolute addresses. The generated image from a locator is then directly downloadable to the target hardware. The locator also produces a MAP file that describes name, size and absolute and relative locations of the program segments and global symbols.

Debugger

A Debugger or debugging tool is a computer program that is used to test and debug other programs. The code to be examined should be running on an instruction set simulator to identify the fault in the code because the software problem cannot be identified when we are running it on the original hardware. The debugger can be used to identify if the program is running correctly, and identify the cause of failure when it fails. The debugger may be a source-level debugger, or a low-level debugger. If it is a source-level debugger, the debugger can show the actual position in the original code, when the program crashes. If it is a low-level debugger or a machine-language debugger, it shows that line in the program. Catching run-time

errors is not as obvious. Most embedded systems do not have a “screen”. Hence we cannot find the run time errors as in general software development.

Downloader:

Remote Debuggers also called as downloader is used to download, execute and debug embedded software over communications link (e.g., serial port)

- Front-end has text or GUI-based windows for source code, register contents, etc
- Backend provides low-level control of target processor, runs on target processor and communicates to front-end over comm-link
- Debugger and software being debugged are executing on two different computer systems
- Supports higher level of interaction between host and target
 - o Allows start/restart/kill, and stepping through program
 - o Software breakpoints (stop execution if instruction X is fetched)
 - o Read/write registers or data at specified address
- Disadvantage: Requires target processor to run more than final software package

Chapter -7

8051 Micro-Controller

Microprocessors Vs Microcontrollers:

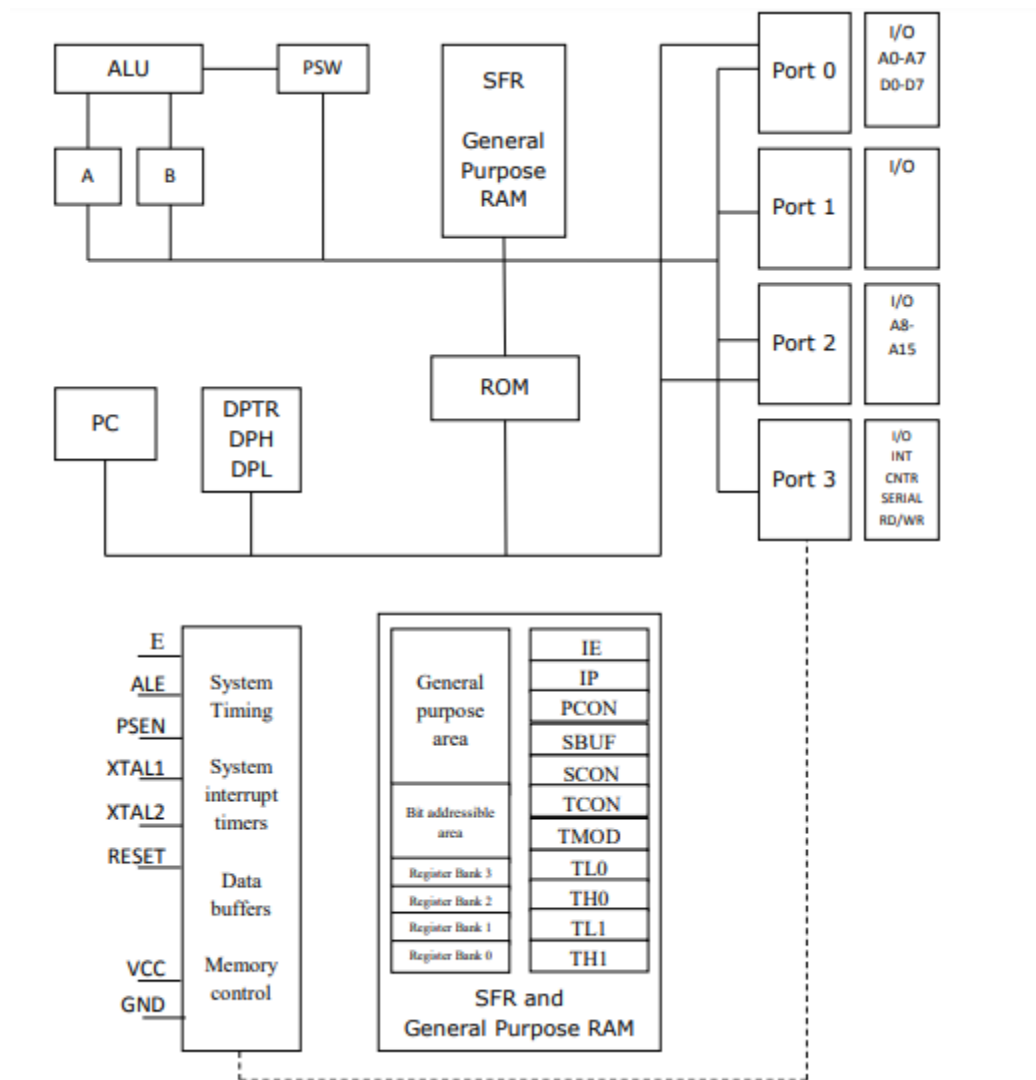
<u>Microprocessors</u>	<u>Microcontrollers</u>
Microprocessor contains ALU, General purpose registers, stack pointer, program counter, clock timing circuit, interrupt circuit.	Microcontroller contains the circuitry of microprocessor, and in addition it has built in ROM, RAM, I/O Devices, Timers/Counters etc.
Few bit handling instruction	It has many bit handling instructions
Less number of pins are multifunctional	More number of pins are multifunctional
Single memory map for data and code (program)	Separate memory map for data and code (program)
Access time for memory and IO are more	Less access time for built in memory and IO.
Microprocessor based system requires additional hardware	It requires less additional hardwires.
More flexible in the design point of view	Less flexible since the additional circuits which is residing inside the microcontroller is fixed for a particular microcontroller
Large number of instructions with flexible addressing modes	Limited number of instructions with few addressing modes

THE 8051 ARCHITECTURE: Introduction:

Salient features of 8051 microcontroller are given below.

- Eight bit CPU
- On chip clock oscillator
- 4Kbytes of internal program memory (code memory) [ROM]
- 128 bytes of internal data memory [RAM]
- 64 Kbytes of external program memory address space.
- 64 Kbytes of external data memory address space.
- 32 bi directional I/O lines (can be used as four 8 bit ports or 32 individually addressable I/O lines)
- Two 16 Bit Timer/Counter :T0, T1
- Full Duplex serial data receiver/transmitter
- Four Register banks with 8 registers in each bank.
- Sixteen bit Program counter (PC) and a data pointer (DPTR)
- 8 Bit Program Status Word (PSW)
- 8 Bit Stack Pointer
- Five vector interrupt structure (RESET not considered as an interrupt.)
- 8051 CPU consists of 8 bit ALU with associated registers like accumulator 'A', B register, PSW, SP, 16 bit program counter, stack pointer.
- ALU can perform arithmetic and logic functions on 8 bit variables.
- 8051 has 128 bytes of internal RAM which is divided into
 - Working registers [00 – 1F]
 - Bit addressable memory area [20 – 2F]
 - General purpose memory area (Scratch pad memory) [30-7F]

The 8051 Architecture



- 8051 has 4 K Bytes of internal ROM. The address space is from 0000 to 0FFFh. If the program size is more than 4 K Bytes 8051 will fetch the code automatically from external memory
- Accumulator is an 8 bit register widely used for all arithmetic and logical operations. Accumulator is also used to transfer data between external memory. B register is used along with Accumulator for multiplication and division. A and B registers together is also called MATH registers.
- PSW (Program Status Word). This is an 8 bit register which contains the arithmetic status of ALU and the bank select bits of register banks.

CY	AC	F0	RS1	RS0	OV	-	P
----	----	----	-----	-----	----	---	---

CY - carry flag

AC - auxiliary carry flag

F0 - available to the user for general purpose

RS1, RS0 - register bank select bits

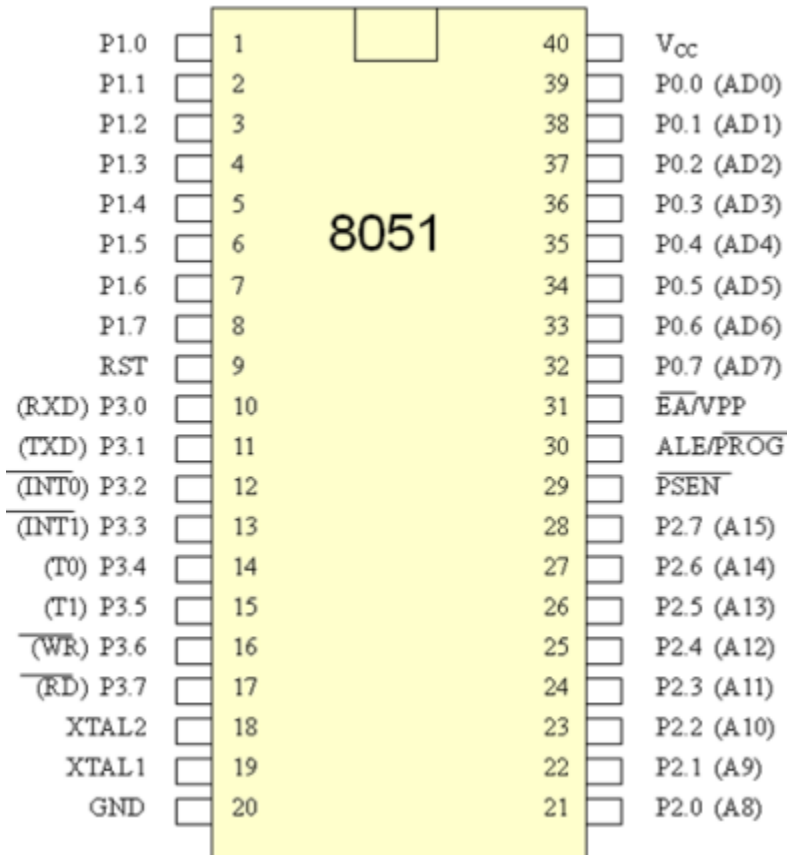
OV - overflow

P – Parity

- Stack Pointer (SP) – it contains the address of the data item on the top of the stack. Stack may reside anywhere on the internal RAM. On reset, SP is initialized to 07 so that the default stack will start from address 08 onwards.
- Data Pointer (DPTR) – DPH (Data pointer higher byte), DPL (Data pointer lower byte). This is a 16 bit register which is used to furnish address information for internal and external program memory and for external data memory.
- Program Counter (PC) – 16 bit PC contains the address of next instruction to be executed. On reset PC will set to 0000. After fetching every instruction PC will increment by one.

Pin Diagram:

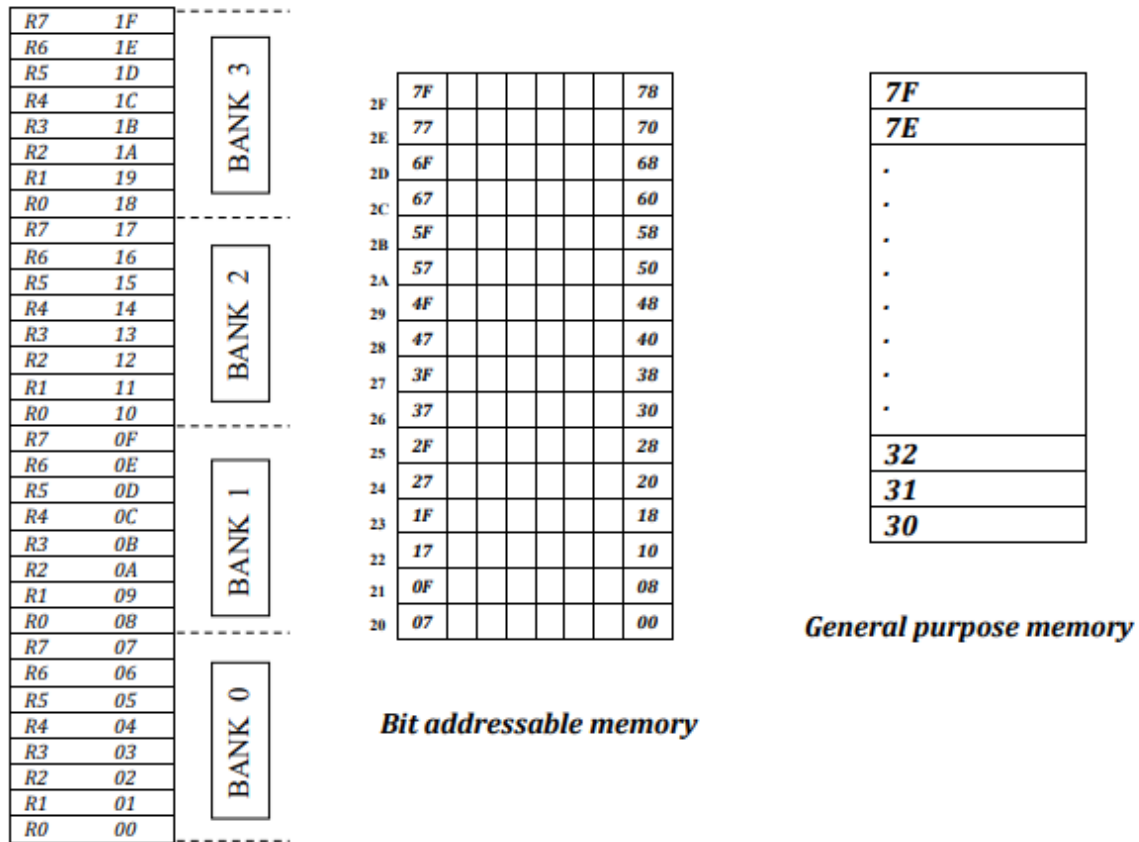
- **Pins 1-8: PORT 1.** Each of these pins can be configured as an input or an output.
- **Pin 9 RESET.** A logic one on this pin disables the microcontroller and clears the contents of most registers. In other words, the positive voltage on this pin resets the microcontroller. By applying logic zero to this pin, the program starts execution from the beginning.
- **Pins 10-17 PORT 3.** Similar to port 1, each of these pins can serve as general input or output. Besides, all of them have alternative functions.
- **Pin 10 RXD.** Serial asynchronous communication input or Serial synchronous communication output.
- **Pin 11 TXD.** Serial asynchronous communication output or Serial synchronous communication clock output.
- **Pin 12 INT0.** External Interrupt 0 input.
- **Pin 13 INT1.** External Interrupt 1 input.
- **Pin 14 T0.** Counter 0 clock input.
- **Pin 15 T1.** Counter 1 clock input
- **Pin 16 WR.** Write to external (additional) RAM.
- **Pin 17 RD.** Read from external RAM



- **Pin 18, 19 XTAL2, XTAL1.** Internal oscillator input and output. A quartz crystal which specifies operating frequency is usually connected to these pins.
- **Pin 20 GND.** Ground.
- **Pin 21-28 Port 2.** If there is no intention to use external memory then these port pins are configured as general inputs/outputs. In case external memory is used, the higher address byte, i.e. addresses A8-A15 will appear on this port. Even though memory with capacity of 64Kb is not used, which means that not all eight port bits are used for its addressing, the rest of them are not available as inputs/outputs.
- **Pin 29 PSEN.** If external ROM is used for storing program then a logic zero (0) appears on it every time the microcontroller reads a byte from memory.
- **Pin 30 ALE.** Prior to reading from external memory, the microcontroller puts the lower address byte (A0-A7) on P0 and activates the ALE output. After receiving signal from the ALE pin, the external latch latches the state of P0 and uses it as a memory chip address. Immediately after that, the ALE pin is returned its previous logic state and P0 is now used as a Data Bus.
- **Pin 32-39 PORT 0.** Similar to P2, if external memory is not used, these pins can be used as general inputs/outputs. Otherwise, P0 is configured as address output (A0-A7) when the ALE pin is driven high (1) or as data output (Data Bus) when the ALE pin is driven low (0).
- **Pin 40 VCC.** +5V power supply.

Memory Organization

Internal RAM organization



Working Registers

- Register Banks: 00h to 1Fh. The 8051 uses 8 general-purpose registers R0 through R7 (R0, R1, R2, R3, R4, R5, R6, and R7). There are four such register banks. Selection of register bank can be done through RS1, RS0 bits of PSW. On reset, the default Register Bank 0 will be selected.
- Bit Addressable RAM: 20h to 2Fh. The 8051 supports a special feature which allows access to bit variables. This is where individual memory bits in Internal RAM can be set or cleared. In all there are 128 bits numbered 00h to 7Fh. Being bit variables any one variable can have a value 0 or 1. A bit variable can be set with a command such as SETB and cleared with a command such as CLR. Example instructions are:
 SETB 25h ; sets the bit 25h (becomes 1)
 CLR 25h ; clears bit 25h (becomes 0) Note, bit 25h is actually bit 5 of Internal RAM location 24h. The Bit Addressable area of the RAM is just 16 bytes of Internal RAM located between 20h and 2Fh.
- General Purpose RAM: 30h to 7Fh. Even if 80 bytes of Internal RAM memory are available for general-purpose data storage, user should take care while using the memory location from 00 -2Fh since these locations are also the default register space, stack space, and bit addressable space. It is

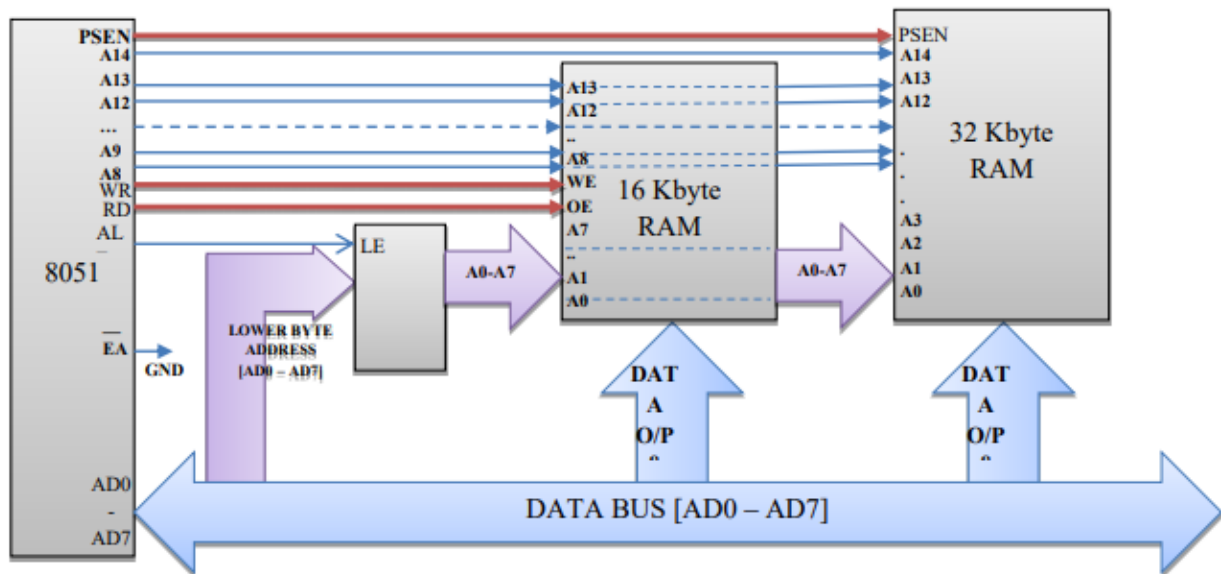
a good practice to use general purpose memory from 30 – 7Fh. The general purpose RAM can be accessed using direct or indirect addressing modes.

External Memory Interfacing:

Eg. Interfacing of 16 K Byte of RAM and 32 K Byte of EPROM to 8051

Number of address lines required for 16 Kbyte memory is 14 lines and that of 32Kbytes of memory is 15 lines.

The connections of external memory is shown below.



The lower order address and data bus are multiplexed. De-multiplexing is done by the latch. Initially the address will appear in the bus and this latched at the output of latch using ALE signal. The output of the latch is directly connected to the lower byte address lines of the memory. Later data will be available in this bus. Still the latch output is address itself. The higher byte of address bus is directly connected to the memory. The number of lines connected depends on the memory size.

The RD and WR (both active low) signals are connected to RAM for reading and writing the data. PSEN of microcontroller is connected to the output enable of the ROM to read the data from the memory.

EA (active low) pin is always grounded if we use only external memory. Otherwise, once the program size exceeds internal memory the microcontroller will automatically switch to external memory.

Stack:

A stack is a last in first out memory. In 8051 internal RAM space can be used as stack. The address of the stack is contained in a register called stack pointer. Instructions PUSH and POP are used for stack operations. When a data is to be placed on the stack, the stack pointer increments before storing the data on the stack so that the stack grows up as data is stored (pre-increment). As the data is retrieved from the stack the byte is read from the stack, and then SP decrements to point the next available byte

of stored data (post decrement). The stack pointer is set to 07 when the 8051 resets. So that default stack memory starts from address location 08 onwards (to avoid overwriting the default register bank ie., bank 0). Eg; Show the stack and SP for the following.

	[SP]=07	//CONTENT OF SP IS 07 (DEFAULT VALUE)	
MOV R6, #25H	[R6]=25H	//CONTENT OF R6 IS 25H	
MOV R1, #12H	[R1]=12H	//CONTENT OF R1 IS 12H	
MOV R4, #0F3H	[R4]=F3H	//CONTENT OF R4 IS F3H	
PUSH 6	[SP]=08	[08]=[06]=25H	//CONTENT OF 08 IS 25H
PUSH 1	[SP]=09	[09]=[01]=12H	//CONTENT OF 09 IS 12H
PUSH 4	[SP]=0A	[0A]=[04]=F3H	//CONTENT OF 0A IS F3H
POP 6	[06]=[0A]=F3H	[SP]=09	//CONTENT OF 06 IS F3H
POP 1	[01]=[09]=12H	[SP]=08	//CONTENT OF 01 IS 12H
POP 4	[04]=[08]=25H	[SP]=07	//CONTENT OF 04 IS 25H

Instruction Syntax.

General syntax for 8051 assembly language is as follows.

LABEL: OPCODE OPERAND; COMMENT

LABEL: (THIS IS NOT NECESSARY UNLESS THAT SPECIFIC LINE HAS TO BE ADDRESSED). The label is a symbolic address for the instruction. When the program is assembled, the label will be given specific address in which that instruction is stored. Unless that specific line of instruction is needed by a branching instruction in the program, it is not necessary to label that line.

OPCODE: Opcode is the symbolic representation of the operation. The assembler converts the opcode to a unique binary code (machine language).

OPERAND: While opcode specifies what operation to perform, operand specifies where to perform that action. The operand field generally contains the source and destination of the data. In some cases only source or destination will be available instead of both. The operand will be either address of the data, or data itself.

COMMENT: Always comment will begin with; or // symbol. To improve the program quality, programmer may always use comments in the program.

Addressing Modes

Various methods of accessing the data are called addressing modes.

8051 addressing modes are classified as follows.

1. Immediate addressing.
2. Register addressing.
3. Direct addressing.
4. Indirect addressing.
5. Relative addressing.

6. Absolute addressing.
7. Long addressing.
8. Indexed addressing.
9. Bit inherent addressing.
10. Bit direct addressing.

Immediate addressing.

In this addressing mode the data is provided as a part of instruction itself. In other words data immediately follows the instruction.

Eg. MOV A,#30H

ADD A, #83

Symbol indicates the data is immediate

Register addressing.

In this addressing mode the register will hold the data. One of the eight general registers (R0 to R7) can be used and specified as the operand.

Eg. MOV A, R0

ADD A, R6

R0 – R7 will be selected from the current selection of register bank. The default register bank will be bank 0.

Direct addressing

There are two ways to access the internal memory. Using direct address and indirect address. Using direct addressing mode we can not only address the internal memory but SFRs also. In direct addressing, an 8 bit internal data memory address is specified as part of the instruction and hence, it can specify the address only in the range of 00H to FFH. In this addressing mode, data is obtained directly from the memory.

Eg. MOV A, 60h

ADD A, 30h

Indirect addressing

The indirect addressing mode uses a register to hold the actual address that will be used in data movement. Registers R0 and R1 and DPTR are the only registers that can be used as data pointers. Indirect addressing cannot be used to refer to SFR registers. Both R0 and R1 can hold 8 bit address and DPTR can hold 16 bit address.

Eg. MOV A,@R0

ADD A,@R1

MOVX A,@DPTR

Indexed addressing.

In indexed addressing, either the program counter (PC), or the data pointer (DPTR)—is used to hold the base address, and the A is used to hold the offset address. Adding the value of the base address to the value

of the offset address forms the effective address. Indexed addressing is used with JMP or MOVC instructions. Look up tables are easily implemented with the help of index addressing.

Eg.

```
MOVC A, @A+DPTR      // copies the contents of memory location pointed by the
sum of the accumulator A and the DPTR into accumulator A.
```

```
MOVC A, @A+PC         // copies the contents of memory location pointed by the
sum of the accumulator A and the program counter into accumulator A.
```

Relative Addressing.

Relative addressing is used only with conditional jump instructions. The relative address, (offset), is an 8 bit signed number, which is automatically added to the PC to make the address of the next instruction. The 8 bit signed offset value gives an address range of +127 to —128 locations. The jump destination is usually specified using a label and the assembler calculates the jump offset accordingly. The advantage of relative addressing is that the program code is easy to relocate and the address is relative to position in the memory.

Eg.

```
SJMP LOOP1
```

```
JC BACK
```

Absolute addressing

Absolute addressing is used only by the AJMP (Absolute Jump) and ACALL (Absolute Call) instructions. These are 2 bytes instructions. The absolute addressing mode specifies the lowest 11 bit of the memory address as part of the instruction. The upper 5 bit of the destination address are the upper 5 bit of the current program counter. Hence, absolute addressing allows branching only within the current 2 Kbyte page of the program memory.

Eg.

```
AJMP LOOP1
```

```
ACALL LOOP2
```

Long Addressing

The long addressing mode is used with the instructions LJMP and LCALL. These are 3 byte instructions. The address specifies a full 16 bit destination address so that a jump or a call can be made to a location within a 64 Kbyte code memory space.

Eg.

```
LJMP FINISH
```

```
LCALL DELAY
```

Bit Inherent Addressing

In this addressing, the address of the flag which contains the operand, is implied in the opcode of the instruction.

Eg.

```
CLR C; Clears the carry flag to 0
```

Bit Direct Addressing

In this addressing mode the direct address of the bit is specified in the instruction. The RAM space 20H to 2FH and most of the special function registers are bit addressable. Bit address values are between 00H to 7FH.

Eg.

CLR 07h; Clears the bit 7 of 20h RAM space

SETB 07H; Sets the bit 7 of 20H RAM space.

Instruction Timings

- The 8051 internal operations and external read/write operations are controlled by the oscillator clock.
- T-state, Machine cycle and Instruction cycle are terms used in instruction timings.
- T-state is defined as one subdivision of the operation performed in one clock period.
- The terms 'Tstate' and 'clock period' are often used synonymously.
- Machine cycle is defined as 12 oscillator periods. A machine cycle consists of six states and each state lasts for two oscillator periods.
- An instruction takes one to four machine cycles to execute an instruction.
- Instruction cycle is defined as the time required for completing the execution of an instruction.
- The 8051 instruction cycle consists of one to four machine cycles.

Eg. If 8051 microcontroller is operated with 12 MHz oscillator, find the execution time for the following four instructions.

1. ADD A, 45H
2. SUBB A, #55H
3. MOV DPTR, #2000H
4. MUL AB

Since the oscillator frequency is 12 MHz, the clock period is, $\text{Clock period} = 1/12 \text{ MHz} = 0.08333 \mu\text{s}$.

Time for 1 machine cycle = $0.08333 \mu\text{s} \times 12 = 1 \mu\text{s}$.

Instruction	No. of machine cycles	Execution time
1. ADD A, 45H	1	1 μs
2. SUBB A, #55H	2	2 μs
3. MOV DPTR, #2000H	2	2 μs
4. MUL AB	4	4 μs

8051 Instructions

The instructions of 8051 can be broadly classified under the following headings.

1. Data transfer instructions
2. Arithmetic instructions
3. Logical instructions
4. Branch instructions
5. Subroutine instructions
6. Bit manipulation instructions

Data Transfer Instructions

In this group, the instructions perform data transfer operations of the following types.

- a. Move the contents of a register Rn to A
 - i. MOV A,R2 ii. MOV A,R7
- b. Move the contents of a register A to Rn
 - i. MOV R4,A ii. MOV R1,A
- c. Move an immediate 8 bit data to register A or to Rn or to a memory location(direct or indirect)
 - i. MOV A, #45H iv. MOV @R0, #0E8H
 - ii. MOV R6, #51H v. MOV DPTR, #0F5A2H
 - iii. MOV 30H, #44H vi. MOV DPTR, #5467H
- d. Move the contents of a memory location to A or A to a memory location using direct and indirect addressing
 - i. MOV A, 65H iii. MOV 45H, A
 - ii. MOV A, @R0 iv. MOV @R1, A
- e. Move the contents of a memory location to Rn or Rn to a memory location using direct addressing
 - i. MOV R3, 65H ii. MOV 45H, R2
- f. Move the contents of memory location to another memory location using direct and indirect addressing
 - i. MOV 47H, 65H
 - ii. MOV 45H, @R0
- g. Move the contents of an external memory to A or A to an external memory
 - i. MOVX A,@R1 iii. MOVX A,@DPTR
 - ii. MOVX @R0,A iv. MOVX@DPTR,A
- h. Move the contents of program memory to A
 - i. MOVC A, @A+PC ii. MOVC A, @A+DPTR

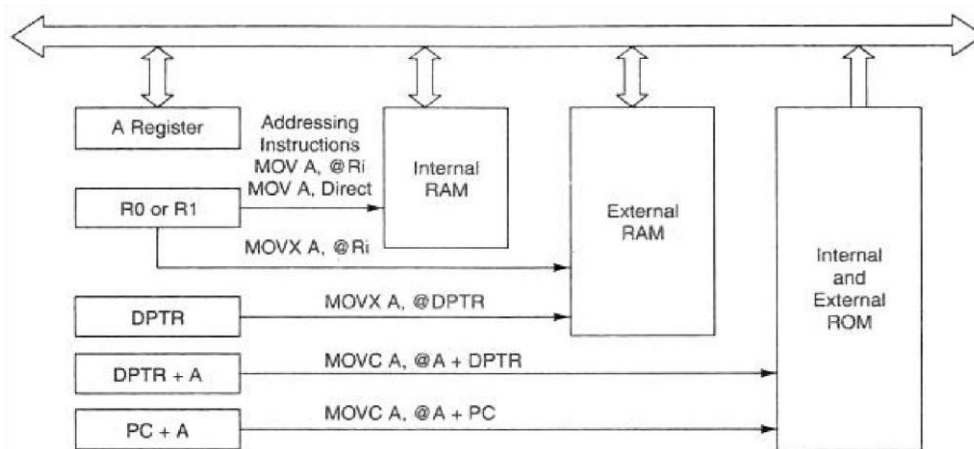


FIG. Addressing Using MOV, MOVX and MOVC

i. Push and Pop instructions

	[SP]=07 //CONTENT OF SP IS 07 (DEFAULT VALUE)
MOV R6, #25H	[R6]=25H //CONTENT OF R6 IS 25H
MOV R1, #12H	[R1]=12H //CONTENT OF R1 IS 12H
MOV R4, #0F3H	[R4]=F3H //CONTENT OF R4 IS F3H
PUSH 6	[SP]=08 [08]=[06]=25H //CONTENT OF 08 IS 25H
PUSH 1	[SP]=09 [09]=[01]=12H //CONTENT OF 09 IS 12H
PUSH 4	[SP]=0A [0A]=[04]=F3H //CONTENT OF 0A IS F3H
POP 6	[06]=[0A]=F3H [SP]=09 //CONTENT OF 06 IS F3H
POP 1	[01]=[09]=12H [SP]=08 //CONTENT OF 01 IS 12H
POP 4	[04]=[08]=25H [SP]=07 //CONTENT OF 04 IS 25H

j. Exchange instructions

The content of source i.e., register, direct memory or indirect memory will be exchanged with the contents of destination i.e., accumulator.

- i. XCH A,R3
- ii. XCH A,@R1
- iii. XCH A,54h

k. Exchange digit. Exchange the lower order nibble of Accumulator (A0-A3) with lower order nibble of the internal RAM location which is indirectly addressed by the register. i. XCHD A,@R1 ii. XCHD A,@R0

Arithmetic Instructions

The 8051 can perform addition, subtraction. Multiplication and division operations on 8 bit numbers.

Addition

In this group, we have instructions to

- i. Add the contents of A with immediate data with or without carry.
 - i. ADD A, #45H
 - ii. ADDC A, #0B4H
- ii. Add the contents of A with register Rn with or without carry.
 - i. ADD A, R5

- ii. ADDC A, R2
- iii. Add the contents of A with contents of memory with or without carry using direct and indirect addressing
 - i. ADD A, 51H
 - ii. ADDC A, 75H
 - iii. ADD A, @R1
 - iv. ADDC A, @R0

CY AC and OV flags will be affected by this operation.

Subtraction

In this group, we have instructions to

- i. Subtract the contents of A with immediate data with or without carry.
 - i. SUBB A, #45H
 - ii. SUBB A, #0B4H
- ii. Subtract the contents of A with register Rn with or without carry.
 - i. SUBB A, R5 ii. SUBB A, R2
- iii. Subtract the contents of A with contents of memory with or without carry using direct and indirect addressing
 - i. SUBB A, 51H ii. SUBB A, 75H
 - iii. SUBB A, @R1 iv. SUBB A, @R0

CY AC and OV flags will be affected by this operation.

Multiplication

MUL AB. This instruction multiplies two 8 bit unsigned numbers which are stored in A and B register. After multiplication the lower byte of the result will be stored in accumulator and higher byte of result will be stored in B register.

Eg. MOV A,#45H ;[A]=45H
 MOV B,#0F5H ;[B]=F5H
 MUL AB ;[A] x [B] = 45 x F5 = 4209
 ;[A]=09H, [B]=42H

Division

DIV AB. This instruction divides the 8 bit unsigned number which is stored in A by the 8 bit unsigned number which is stored in B register. After division the result will be stored in accumulator and remainder will be stored in B register.

Eg. MOV A,#45H ;[A]=0E8H
 MOV B,#0F5H ;[B]=1BH
 DIV AB ;[A] / [B] = E8 / 1B = 08 H with remainder 10H
 ;[A] = 08H, [B]=10H

DA A (Decimal Adjust After Addition).

When two BCD numbers are added, the answer is a non-BCD number. To get the result in BCD, we use DA A instruction after the addition. DA A works as follows.

- If lower nibble is greater than 9 or auxiliary carry is 1, 6 is added to lower nibble.
- If upper nibble is greater than 9 or carry is 1, 6 is added to upper nibble.

Eg 1: MOV A,#23H
 MOV R1,#55H
 ADD A,R1 // [A]=78
 DA A // [A]=78 *no changes in the accumulator after da a*

Eg 2: MOV A,#53H
 MOV R1,#58H
 ADD A,R1 // [A]=ABh
 DA A // [A]=11, C=1 . ANSWER IS 111. *Accumulator data is changed after DA A*

Increment: increments the operand by one.

INC A INC Rn INC DIRECT INC @Ri INC DPTR

INC increments the value of source by 1. If the initial value of register is FFh, incrementing the value will cause it to reset to 0. The Carry Flag is not set when the value "rolls over" from 255 to 0.

In the case of "INC DPTR", the value two-byte unsigned integer value of DPTR is incremented. If the initial value of DPTR is FFFFh, incrementing the value will cause it to reset to 0.

Decrement: decrements the operand by one.

DEC A DEC Rn DEC DIRECT DEC @Ri

DEC decrements the value of source by 1. If the initial value of is 0, decrementing the value will cause it to reset to FFh. The Carry Flag is not set when the value "rolls over" from 0 to FFh.

Logical Instruction:

Logical AND

ANL destination, source: ANL does a bitwise "AND" operation between *source* and *destination*, leaving the resulting value in *destination*. The value in source is not affected. "AND" instruction logically AND the bits of source and destination.

ANL A,#DATA ANL A, Rn
ANL A,DIRECT ANL A,@Ri
ANL DIRECT,A ANL DIRECT, #DATA

Logical OR

ORL destination, source: ORL does a bitwise "OR" operation between *source* and *destination*, leaving the resulting value in *destination*. The value in source is not affected. "OR" instruction logically OR the bits of source and destination.

ORL A,#DATA ORL A, Rn
ORL A,DIRECT ORL A,@Ri
ORL DIRECT,A ORL DIRECT, #DATA

Logical Ex-OR

XRL destination, source: XRL does a bitwise "EX-OR" operation between *source* and *destination*, leaving the resulting value in *destination*. The value in source is not affected. "XRL" instruction logically EX-OR the bits of source and destination.

XRL A,#DATA XRL A,Rn
XRL A,DIRECT XRL A,@Ri
XRL DIRECT,A XRL DIRECT, #DATA

Logical NOT

CPL complements *operand*, leaving the result in *operand*. If *operand* is a single bit then the state of the bit will be reversed. If *operand* is the Accumulator then all the bits in the Accumulator will be reversed.

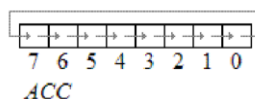
CPL A, CPL C, CPL bit address

SWAP A – Swap the upper nibble and lower nibble of A.

Rotate Instructions

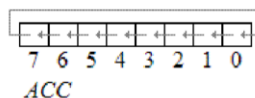
RR A

This instruction is rotate right the accumulator. Its operation is illustrated below. Each bit is shifted one location to the right, with bit 0 going to bit 7.



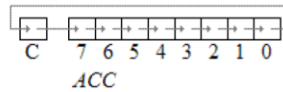
RL A

Rotate left the accumulator. Each bit is shifted one location to the left, with bit 7 going to bit 0



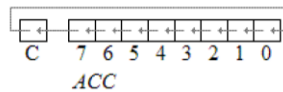
RRC A

Rotate right through the carry. Each bit is shifted one location to the right, with bit 0 going into the carry bit in the PSW, while the carry was at goes into bit 7



RLC A

Rotate left through the carry. Each bit is shifted one location to the left, with bit 7 going into the carry bit in the PSW, while the carry goes into bit 0.



Branch (JUMP) Instruction:

Jump and Call Program Range

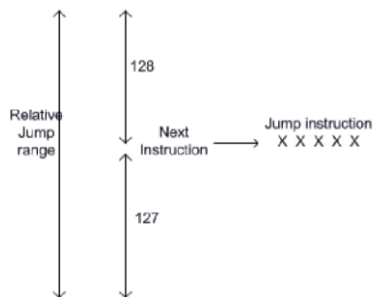
There are 3 types of jump instructions. They are:-

1. Relative Jump
2. Short Absolute Jump
3. Long Absolute Jump

Relative Jump

Jump that replaces the PC (program counter) content with a new address that is greater than (the address following the jump instruction by 127 or less) or less than (the address following the jump by 128 or less) is called a relative jump. Schematically, the relative jump can be shown as follows:

-



The advantages of the relative jump are as follows:-

1. Only 1 byte of jump address needs to be specified in the 2's complement form, ie. For jumping ahead, the range is 0 to 127 and for jumping back, the range is -1 to -128.
2. Specifying only one byte reduces the size of the instruction and speeds up program execution.
3. The program with relative jumps can be relocated without reassembling to generate absolute jump addresses.

Disadvantages of the absolute jump: -

1. Short jump range (-128 to 127 from the instruction following the jump instruction)

Instructions that use Relative Jump

SJMP <relative address>; *this is unconditional jump*

The remaining relative jumps are conditional jumps

JC <relative address>
JNC <relative address>
JB bit, <relative address>
JNB bit, <relative address>
JBC bit, <relative address>
CJNE <destination byte>, <source byte>, <relative address>
DJNZ <byte>, <relative address>
JZ <relative address>
JNZ <relative address>

Short Absolute Jump

In this case only 11bits of the absolute jump address are needed. The absolute jump address is calculated in the following manner.

In 8051, 64 kbyte of program memory space is divided into 32 pages of 2 kbyte each. The hexadecimal addresses of the pages are given as follows:-

<i>Page (Hex)</i>	<i>Address (Hex)</i>
00	0000 - 07FF
01	0800 - 0FFF
02	1000 - 17FF
03	1800 - 1FFF
.	
.	
1E	F000 - F7FF
1F	F800 - FFFF

It can be seen that the upper 5bits of the program counter (PC) hold the page number and the lower 11bits of the PC hold the address within that page. Thus, an absolute address is formed by taking page numbers of the instruction (from the program counter) following the jump and attaching the specified 11bits to it to form the 16-bit address.

Advantage: The instruction length becomes 2 bytes.

Example of short absolute jump: -

ACALL <address 11>

AJMP <address 11>

Long Absolute Jump/Call

Applications that need to access the entire program memory from 0000H to FFFFH use long absolute jump. Since the absolute address has to be specified in the op-code, the instruction length is 3 bytes (except for JMP @ A+DPTR). This jump is not re-locatable.

Example: -

```
LCALL <address 16>
LJMP  <address 16>
JMP @A+DPTR
```

Another classification of jump instructions is

1. Unconditional Jump
 2. Conditional Jump
-
1. **The unconditional jump** is a jump in which control is transferred unconditionally to the target location.
 - a. **LJMP** (long jump). This is a 3-byte instruction. First byte is the op-code and second and third bytes represent the 16-bit target address which is any memory location from 0000 to FFFFH
eg: LJMP 3000H
 - b. **AJMP**: this causes unconditional branch to the indicated address, by loading the 11 bit address to 0 -10 bits of the program counter. The destination must be therefore within the same 2K blocks.
 - c. **SJMP** (short jump). This is a 2-byte instruction. First byte is the op-code and second byte is the relative target address, 00 to FFH (forward +127 and backward -128 bytes from the current PC value). To calculate the target address of a short jump, the second byte is added to the PC value which is address of the instruction immediately below the jump.

2. Conditional Jump instructions.

JBC	Jump if bit = 1 and clear bit
JNB	Jump if bit = 0
JB	Jump if bit = 1
JNC	Jump if CY = 0
JC	Jump if CY = 1
CJNE reg,#data	Jump if byte ≠ #data
CJNE A,byte	Jump if A ≠ byte
DJNZ	Decrement and Jump if A ≠ 0
JNZ	Jump if A ≠ 0
JZ	Jump if A = 0

All conditional jumps are short jumps.

Bit level jump instructions:

Bit level JUMP instructions will check the conditions of the bit and if condition is true, it jumps to the address specified in the instruction. All the bit jumps are relative jumps.

JB bit, rel ; jump if the direct bit is set to the relative address specified.

JNB bit, rel ; jump if the direct bit is clear to the relative address specified.

JBC bit, rel ; jump if the direct bit is set to the relative address specified and then clear the bit.

Subroutine Call and Return Instruction:

Subroutines are handled by CALL and RET instructions

There are two types of CALL instructions

1. LCALL address(16 bit)

This is long call instruction which unconditionally calls the subroutine located at the indicated 16 bit address. This is a 3 byte instruction. The LCALL instruction works as follows.

- a. During execution of LCALL, $[PC] = [PC] + 3$; (if address where LCALL resides is say, 0x3254; during execution of this instruction $[PC] = 3254h + 3h = 3257h$)
- b. $[SP] = [SP] + 1$; (if SP contains default value 07, then SP increments and $[SP] = 08$)
- c. $[[SP]] = [PC_{7-0}]$; (lower byte of PC content ie., 57 will be stored in memory location 08.
- d. $[SP] = [SP] + 1$; (SP increments again and $[SP] = 09$)
- e. $[[SP]] = [PC_{15-8}]$; (higher byte of PC content ie., 32 will be stored in memory location 09.

With these the address (0x3254) which was in PC is stored in stack.

- f. $[PC] = \text{address (16 bit)}$; the new address of subroutine is loaded to PC. No flags are affected.

2. ACALL address(11 bit)

This is absolute call instruction which unconditionally calls the subroutine located at the indicated 11 bit address. This is a 2 byte instruction. The SCALL instruction works as follows.

- a. During execution of SCALL, $[PC] = [PC] + 2$; (if address where LCALL resides is say, 0x8549; during execution of this instruction $[PC] = 8549h + 2h = 854Bh$)
- b. $[SP] = [SP] + 1$; (if SP contains default value 07, then SP increments and $[SP] = 08$)
- c. $[[SP]] = [PC_{7-0}]$; (lower byte of PC content ie., 4B will be stored in memory location 08.
- d. $[SP] = [SP] + 1$; (SP increments again and $[SP] = 09$)
- e. $[[SP]] = [PC_{15-8}]$; (higher byte of PC content ie., 85 will be stored in memory location 09.

With these the address (0x854B) which was in PC is stored in stack.

- f. $[PC_{10-0}] = \text{address (11 bit)}$; the new address of subroutine is loaded to PC. No flags are affected.

RET instruction

RET instruction pops top two contents from the stack and load it to PC.

- g. $[PC_{15-8}] = [[SP]]$;content of current top of the stack will be moved to higher byte of PC.
- h. $[SP] = [SP] - 1$; (SP decrements)
- i. $[PC_{7-0}] = [[SP]]$;content of bottom of the stack will be moved to lower byte of PC.
- j. $[SP] = [SP] - 1$; (SP decrements again)

Bit Manipulation Instructions:

8051 has 128 bit addressable memory. Bit addressable SFRs and bit addressable PORT pins. It is possible to perform following bit wise operations for these bit addressable locations.

1. LOGICAL AND

- a. `ANL C,BIT(BIT ADDRESS)` ; 'LOGICALLY AND' CARRY AND CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
- b. `ANL C, /BIT;` ; 'LOGICALLY AND' CARRY AND COMPLEMENT OF CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY

2. LOGICAL OR

- a. `ORL C,BIT(BIT ADDRESS)` ; 'LOGICALLY OR' CARRY AND CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY
- b. `ORL C, /BIT;` ; 'LOGICALLY OR' CARRY AND COMPLEMENT OF CONTENT OF BIT ADDRESS, STORE RESULT IN CARRY

3. CLR bit

- a. `CLR bit` ; CONTENT OF BIT ADDRESS SPECIFIED WILL BE CLEARED.
- b. `CLR C` ; CONTENT OF CARRY WILL BE CLEARED.

4. CPL bit

- a. `CPL bit` ; CONTENT OF BIT ADDRESS SPECIFIED WILL BE COMPLEMENTED.
- `CPL C` ; CONTENT OF CARRY WILL BE COMPLEMENTED

Assembler Directives.

Assembler directives tell the assembler to do something other than creating the machine code for an instruction. In assembly language programming, the assembler directives instruct the assembler to

- 1. Process subsequent assembly language instructions
- 2. Define program constants
- 3. Reserve space for variables

The following are the widely used 8051 assembler directives.

ORG (origin)

The ORG directive is used to indicate the starting address. It can be used only when the program counter needs to be changed. The number that comes after ORG can be either in hex or in decimal.

Eg: ORG 0000H ;Set PC to 0000.

EQU and SET

EQU and SET directives assign numerical value or register name to the specified symbol name.

EQU is used to define a constant without storing information in the memory. The symbol defined with EQU should not be redefined.

SET directive allows redefinition of symbols at a later stage.

DB (DEFINE BYTE)

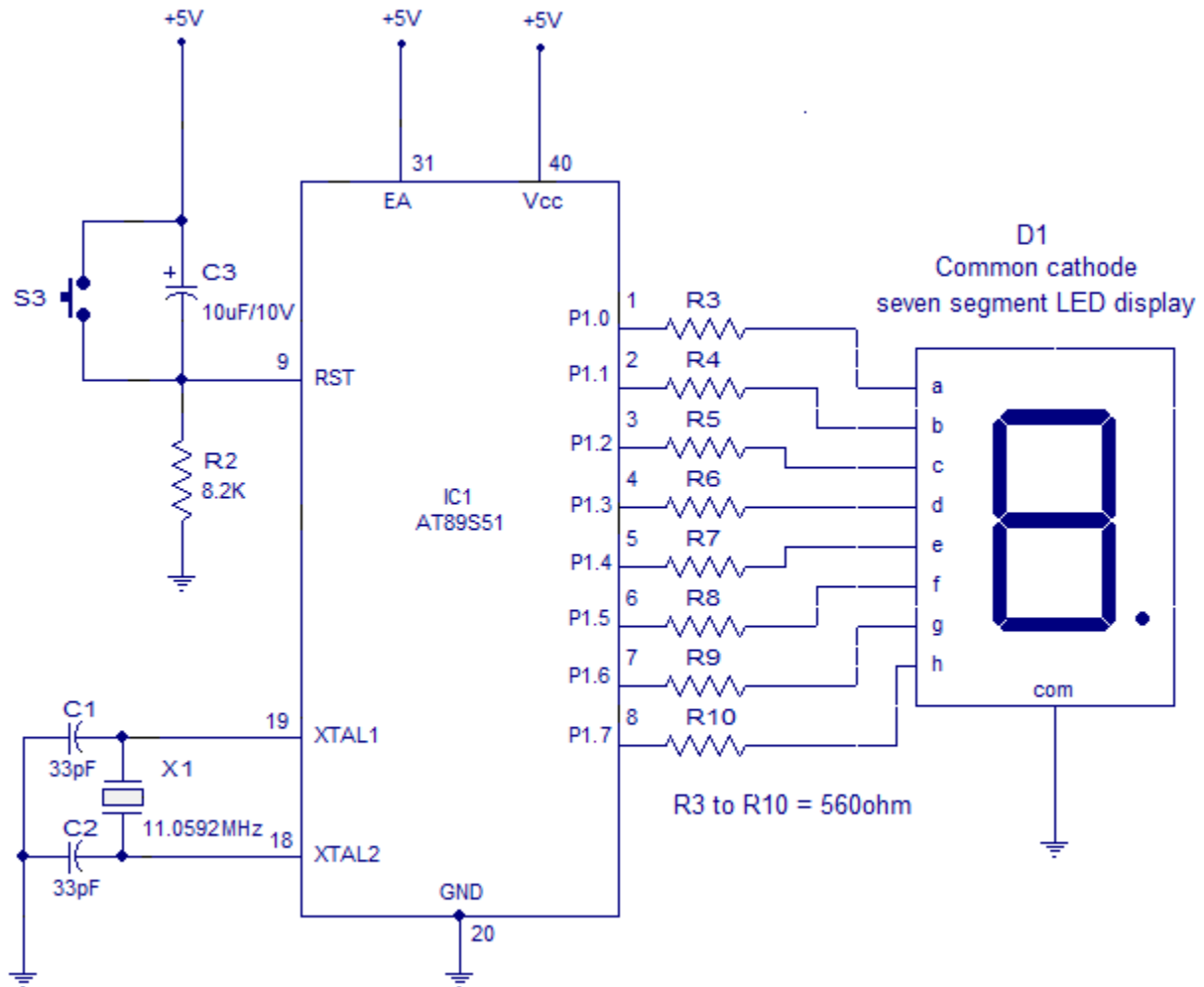
The DB directive is used to define an 8 bit data. DB directive initializes memory with 8 bit values. The numbers can be in decimal, binary, hex or in ASCII formats. For decimal, the 'D' after the decimal number is optional, but for binary and hexadecimal, 'B' and 'H' are required. For ASCII, the number is written in quotation marks ('LIKE This').

```
DATA1: DB 40H           ; hex
DATA2: DB 01011100B     ; b i n a r y
DATA3: DB 48            ; decimal
DATA4: DB 'HELLOW'      ; ASCII
```

END

The END directive signals the end of the assembly module. It indicates the end of the program to the assembler. Any text in the assembly file that appears after the END directive is ignored. If the END statement is missing, the assembler will generate an error message.

Interfacing seven segment display to 8051.



Interfacing 7 segment display to 8051

www.circuitstoday.com

The circuit diagram shown above is of an AT89S51 microcontroller based 0 to 9 counter which has a 7 segment LED display interfaced to it in order to display the count. This simple circuit illustrates two things. How to setup simple 0 to 9 up counter using 8051 and more importantly how to interface a seven segment LED display to 8051 in order to display a particular result. The common cathode seven segment display D1 is connected to the Port 1 of the microcontroller (AT89S51) as shown in the circuit diagram. R3 to R10 are current limiting resistors. S3 is the reset switch and R2, C3 forms a denouncing circuitry. C1, C2 and X1 are related to the clock circuit. The software part of the project has to do the following tasks.

- Form a 0 to 9 counter with a predetermined delay (around 1/2 second here).
- Convert the current count into digit drive pattern.
- Put the current digit drive pattern into a port for displaying.

All the above said tasks are accomplished by the program given below.

```
ORG 000H //initial starting address
START: MOV A,#00001001B // initial value of accumulator
```



```

MOV B,A
MOV R0,#0AH      //Register R0 initialized as counter which counts from 10 to 0
LABEL: MOV A,B
INC A
MOV B,A
MOVC A,@A+PC     // adds the byte in A to the program counters address
MOV P1,A
ACALL DELAY      // calls the delay of the timer
DEC R0           //Counter R0 decremented by 1
MOV A,R0         // R0 moved to accumulator to check if it is zero in next
instruction.

JZ START         //Checks accumulator for zero and jumps to START. Done to
check if counting has been finished.
SJMP LABEL
DB 3FH           // digit drive pattern for 0
DB 06H           // digit drive pattern for 1
DB 5BH           // digit drive pattern for 2
DB 4FH           // digit drive pattern for 3
DB 66H           // digit drive pattern for 4
DB 6DH           // digit drive pattern for 5
DB 7DH           // digit drive pattern for 6
DB 07H           // digit drive pattern for 7
DB 7FH           // digit drive pattern for 8
DB 6FH           // digit drive pattern for 9
DELAY: MOV R4,#05H // subroutine for delay
WAIT1: MOV R3,#00H
WAIT2: MOV R2,#00H
WAIT3: DJNZ R2,WAIT3
DJNZ R3,WAIT2
DJNZ R4,WAIT1
RET
END

```

Assembly Language Programs.

- 1. Write a program to add the values of locations 50H and 51H and store the result in locations in 52h and 53H.**

```

ORG 0000H      ; Set program counter 0000H
MOV A,50H      ; Load the contents of Memory location 50H into A
ADD ADD A,51H   ; Add the contents of memory 51H with CONTENTS
A MOV 52H,A     ; Save the LS byte of the result in 52H
MOV A, #00      ; Load 00H into A
ADDC A, #00     ; Add the immediate data and carry to A
MOV 53H,A       ; Save the MS byte of the result in location 53h

```

END

- 2. Write a program to store data FFH into RAM memory locations 50H to 58H using direct addressing mode**

```
ORG 0000H          ; Set program counter 0000H
MOV A, #0FFH        ; Load FFH into A
MOV 50H, A           ; Store contents of A in location 50H
MOV 51H, A           ; Store contents of A in location 51H
MOV 52H, A           ; Store contents of A in location 52H
MOV 53H, A           ; Store contents of A in location 53H
MOV 54H, A           ; Store contents of A in location 54H
MOV 55H, A           ; Store contents of A in location 55H
MOV 56H, A           ; Store contents of A in location 56H
MOV 57H, A           ; Store contents of A in location 57H
MOV 58H, A           ; Store contents of A in location 58H
END
```

- 3. Write a program to subtract a 16 bit number stored at locations 51H-52H from 55H-56H and store the result in locations 40H and 41H. Assume that the least significant byte of data or the result is stored in low address. If the result is positive, then store 00H, else store 01H in 42H.**

```
ORG 0000H          ; Set program counter 0000H
MOV A, 55H          ; Load the contents of memory location 55 into A
CLR C               ; Clear the borrow flag
SUBB A, 51H          ; Sub the contents of memory 51H from contents of A
MOV 40H, A           ; Save the LSByte of the result in location 40H
MOV A, 56H           ; Load the contents of memory location 56H into A
SUBB A, 52H          ; Subtract the content of memory 52H from the content
MOV 41H, A           ; Save the MSbyte of the result in location 41H.
MOV A, #00           ; Load 00H into A
ADDC A, #00          ; Add the immediate data and the carry flag to A
MOV 42H, A           ; If result is positive, store 00H, else store 01H in 42H
END
```

- 4. Write a program to add two 16 bit numbers stored at locations 51H-52H and 55H-56H and store the result in locations 40H, 41H and 42H. Assume that the least significant byte of data and the result is stored in low address and the most significant byte of data or the result is stored in high address.**

```
ORG 0000H          ; Set program counter 0000H
MOV A, 51H          ; Load the contents of memory location 51H into A
ADD A, 55H           ; Add the contents of 55H with contents of A
MOV 40H, A           ; Save the LS byte of the result in location 40H
MOV A, 52H           ; Load the contents of 52H into A
ADDC A, 56H          ; Add the contents of 56H and CY flag with A
MOV 41H, A           ; Save the second byte of the result in 41H
MOV A, #00           ; Load 00H into A
ADDC A, #00          ; Add the immediate data 00H and CY to A
```

```
MOV 42H,A           ; Save the MS byte of the result in location 42H
END
```

5. Write a program to store data FFH into RAM memory locations 50H to 58H using indirect addressing mode.

```
ORG 0000H           ; Set program counter 0000H
MOV A, #0FFH        ; Load FFH into A
MOV RO, #50H        ; Load pointer, R0-50H
MOV R5, #08H        ; Load counter, R5-08H
Start: MOV @RO, A    ; Copy contents of A to RAM pointed by R0
      INC RO        ; Increment pointer
      DJNZ R5, start ; Repeat until R5 is zero
      END
```

6. Write a program to add two Binary Coded Decimal (BCD) numbers stored at locations 60H and 61H and store the result in BCD at memory locations 52H and 53H. Assume that the least significant byte of the result is stored in low address.

```
ORG 0000H           ; Set program counter 0000H
MOV A,60H           ; Load the contents of memory location 60H into A
ADD A,61H           ; Add the contents of memory location 61H with contents of A
DA A                ; Decimal adjustment of the sum in A
MOV 52H, A          ; Save the least significant byte of the result in location 52H
MOV A,#00           ; Load 00H into A
ADDC A,#00H         ; Add the immediate data and the contents of carry flag to A
MOV 53H,A           ; Save the most significant byte of the result in location 53H;
END
```

7. Write a program to clear 10 RAM locations starting at RAM address 1000H.

```
ORG 0000H           ;Set program counter 0000H
MOV DPTR, #1000H    ;Copy address 1000H to DPTR
CLR A               ;Clear A
MOV R6, #0AH        ;Load 0AH to R6 again:
MOVX @DPTR, A       ;Clear RAM location pointed by DPTR
INC DPTR            ;Increment DPTR
DJNZ R6, again      ;Loop until counter R6=0
END
```

8. Write a program to compute $1 + 2 + 3 + N$ (say $N=15$) and save the sum at 70H

```
ORG 0000H           ; Set program counter 0000H
N EQU 15
MOV R0,#00          ; Clear R0
CLR A               ; Clear A
again: INC R0        ; Increment R0
      ADD A, R0      ; add the contents of R0 with A
      CJNE R0,#N,again ; Loop until counter, R0, N
      MOV 70H,A      ; Save the result in location 70H
      END
```

9. Write a program to multiply two 8 bit numbers stored at locations 70H and 71H and store the result at memory locations 52H and 53H. Assume that the least significant byte of the result is stored in low address.

```

ORG 0000H      ; Set program counter 00 0H
MOV A, 70H     ; Load the contents of memory location 70h into A
MOV B, 71H     ; Load the contents of memory location 71H into B
MUL AB        ; Perform multiplication
MOV 52H,A      ; Save the least significant byte of the result in location 52H
MOV 53H,B      ; Save the most significant byte of the result in location 53
END

```

10. Write a program to find the average of five 8 bit numbers. Store the result in H. (Assume that after adding five 8 bit numbers, the result is 8 bit only).

```

ORG 0000H
MOV 40H,#05H
MOV 41H,#55H
MOV 42H,#06H
MOV 43H,#1AH
MOV 44H,#09H
MOV R0,#40H
MOV R5,#05H
MOV B,R5
CLR A
Loop: ADD A,@R0
INC R0
DJNZ R5,Loop
DIV AB
MOV 55H,A
END

```

12. Write a program to shift a 24 bit number stored at 57H-55H to the left logically four places. Assume that the least significant byte of data is stored in lower address.

```

again:  ORG 0000H ; Set program counter 0000h
        MOV R1,#04 ; Set up loop count to 4
        MOV A,55H ; Place the least significant byte of data in A
        CLR C ; Clear the carry flag
        RLC A ; Rotate contents of A (55h) left through carry
        MOV 55H,A
        MOV A,56H
        RLC A ; Rotate contents of A (56H) left through carry
        MOV 56H,A
        MOV A,57H
        RLC A ; Rotate contents of A (57H) left through carry
        MOV 57H,A
        DJNZ R1,again ; Repeat until R1 is zero
        END

```

Chapter -8

VHDL

VHDL stands for very high-speed integrated circuit hardware description language. It is a programming language used to model a digital system by dataflow, behavioral and structural style of modeling. This language was first introduced in 1981 for the department of Defense (DoD) under the VHSIC program.

- The VHSIC Hardware Description Language (VHDL) is an industry standard language used to describe hardware from the abstract to concrete level.
- The language not only defines the syntax but also defines very clear simulation semantics for each language construct.
- It is strong typed language and is often verbose to write.
- Provides extensive range of modeling capabilities, it is possible to quickly assimilate a core subset of the language that is both easy and simple to understand without learning the more complex features.

Why Use VHDL?

- Quick Time-to-Market
 - Allows designers to quickly develop designs requiring tens of thousands of logic gates.
 - Provides powerful high-level constructs for describing complex logic.
 - Supports modular design methodology and multiple levels of hierarchy.
- One language for design and simulation.
- Allows creation of device-independent designs that are portable to multiple vendors. Good for ASIC Migration.
- Allows user to pick any synthesis tool, vendor, or device.

Basic Features of VHDL:

- Concurrency.
- Supports Sequential Statements.
- Supports for Test & Simulation.

- Strongly Typed Language.
- Supports Hierarchies.
- Supports for Vendor Defined Libraries.
- Supports Multivalued Logic.

Concurrency

- VHDL is a concurrent language.
- HDL differs with Software languages with respect to Concurrency only.
- VHDL executes statements at the same time in parallel, as in Hardware.

Supports Sequential Statements

- VHDL supports sequential statements also, it executes one statement at a time in sequence only.
- As the case with any conventional languages.
- Example:

```
if a='1' then
y<='0';
else y<='1';
end if ;
```

Supports for Test & Simulation

- To ensure that design is correct as per the specifications, the designer has to write another program known as “TEST BENCH”.
- It generates a set of test vectors and sends them to the design under test (DUT).
- Also gives the responses made by the DUT against a specifications for correct results to ensure the functionality.

Strongly Typed Language

- VHDL allows LHS & RHS operators of same type.
- Different types in LHS & RHS is illegal in VHDL.

- Allows different type assignment by conversion.
- Example:

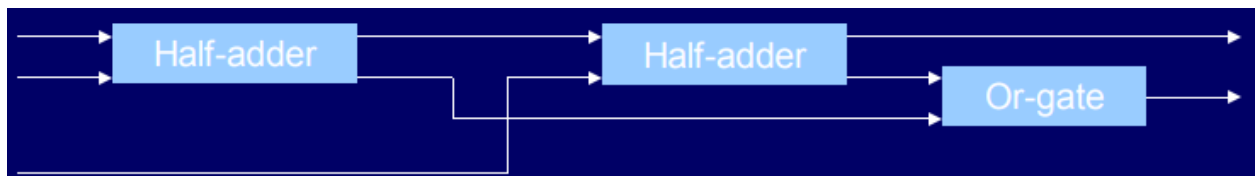
```

A : in std_logic_vector(3 downto 0).
B : out std_logic_vector(3 downto 0).
C : in bit_vector (3 downto 0).
B <= A; --perfect.
B <= C; --type miss match, syntax error.

```

Supports Hierarchies:

- Hierarchy can be represented using VHDL.
- Consider example of a Full-adder which is the top-level module, being composed of three lower level modules i.e. half-Adder and OR gate.
- Example :



Levels of Abstraction:

Data Flow level

- In this style of modeling the flow of data through the entity is expressed using concurrent signal assignment statements.

Structural level

- In this style of modeling the entity is described as a set of interconnected statements.

Behavioral level.

- This style of modeling specifies the behavior of an entity as a set of statements that are executed sequentially in the specified order.

VHDL Identifiers:

- Identifiers are used to name items in a VHDL model.
- A basic identifier may contain only capital 'A' - 'Z' , 'a' - 'z', '0' - '9', underscore character '_'.
- Must start with a alphabet.
- May not end with a underscore character.
- Must not include two successive underscore characters.
- Reserved word cannot be used as identifiers.
- VHDL is not case sensitive.

Objects:

There are three basic object types in VHDL.

- Signal: represents interconnections that connect components and ports.
- Variable: used for local storage within a process.
- Constant: a fixed value.

The object type could be a scalar or an array.

Data Types in VHDL:

Type

- Is a name which is associated with a set of values and a set of operations.

Major Types

- Major types
- Composite Types

Scalar Types

Integer

- Maximum range of integer is tool dependent type integer is range implementation_defined.
- For example:

constant loop_no : integer := 345;

Signal my_int : integer range 0 to 255;

Floating Point:

- Can be either positive or negative.
- Exponents have to be integer.

type real is range implementation_defined

Physical

- Predefined type “Time” used to specify delays.
- Example : type TIME is range -2147483647 to 2147483647

Enumeration

- Values are defined in ascending order.
- Example:
- type alu is (pass, add, subtract, multiply, divide)

Composite Types

There are two composite types

Array:

- Contain many elements of the same type.
- Array can be either single or multidimensional.
- Single dimensional array are synthesizable.
- The synthesis of multidimensional array depends upon the synthesizer being used.

Record:

- Contain elements of different types.

The Std Logic Type:

- It is a data type defined in the std_logic_1164 package of IEEE library.
- It is an enumerated type and is defined as

type std_logic is ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-')

'u'	unspecified
'x'	unknown
'0'	strong zero
'1'	strong one
'z'	high impedance
'w'	weak unknown
'l'	weak zero
'h'	weak one
'-'	don't care

Alias:

- Alias is an alternative name assigned to part of an object simplifying its access.
- Syntax :

alias alias_name : subtype is name;

- Examples:

signal inst : std_logic_vector(7 downto 0);

alias opcode : std_logic_vector(3 downto 0) is inst (7 downto 4);

alias srce : std_logic_vector(1 downto 0) is inst(3 downto 2);

alias dest : std_logic_vector(1 downto 0) is inst (1 downto 0);

Signal Array:

- A set of signals may also be declared as a signal array which is a concatenated set of signals.
- This is done by defining the signal of type bit_vector or std_logic_vector.
- bit_vector and std_logic_vector are types defined in the ieee.std_logic_1164 package.

- Signal array is declared as : < Type > < range >.
- Example:

```
signal data1:bit_vector(1 downto 0);
signal data2: std_logic_vector(7 down to 0);
signal address : std_logic_vector(0 to 15);
```

Subtype

- It is a type with a constraint
- Useful for range checking and for imposing additional constraints on types.

Syntax:

```
subtype subtype_name is base_type range range_constraint;
```

- For example: subtype DIGITS is integer range 0 to 9;

Operators

Predefined VHDL operators can be grouped into seven classes:

1. binary logical operators: **and or nand nor xor xnor**
 - **and** logical and result is boolean,
 - **nand** logical complement of and result is boolean,
 - **nor** logical complement of or result is boolean,
 - **xor** logical exclusive or result is boolean,
 - **xnor** logical complement of exclusive or result is boolean,
2. relational operators:
 - = test for equality, result is boolean
 - /= test for inequality, result is Boolean
 - < test for less than, result is boolean
 - <= test for less than or equal, result is boolean
 - > test for greater than, result is Boolean

- `>=` test for greater than or equal, result is Boolean
3. shift operators:
- `sll` shift left logical,
 - `srl` shift right logical,
 - `sla` shift left arithmetic,
 - `sra` shift right arithmetic,
 - `rol` rotate left,
 - `ror` rotate right,
4. adding operators:
- `+` addition, numeric + numeric, result numeric
 - `-` subtraction, numeric - numeric, result numeric
 - `&` concatenation, array or element & array or element, result array
5. unary sign operators:
- `+` unary plus, + numeric, result numeric
 - `-` unary minus, - numeric, result numeric
6. multiplying operators:
- `*` multiplication, numeric * numeric, result numeric
 - `/` division, numeric / numeric, result numeric
 - `mod` modulo, integer mod integer, result integer
 - `rem` remainder, integer rem integer, result integer
7. miscellaneous operators:
- `abs` absolute value, abs numeric, result numeric
 - `not` complement, not logic or boolean, result same
 - `**` exponentiation, numeric ** integer, result numeric

Multi-Dimensional Arrays

Syntax

type array_name is array (index_range , index_range) of element_type;

For example:

type memory is array (3 downto 0, 7 downto 0);

For synthesizers which do not accept multidimensional arrays, one can declare two unidimensional arrays.

For example:

type byte is array (7 downto 0) of std_logic;

type mem is array (3 downto 0) of byte;

Data Flow Modeling:

Dataflow Level

8. A Dataflow model specifies the functionality of the entity without explicitly specifying its structure.
9. This functionality shows the flow of information through the entity, which is expressed primarily using concurrent signal assignment statements and block statements.
10. The primary mechanism for modeling the dataflow behavior of an entity is using the concurrent signal assignment statement.

Entity

11. Entity describes the design interface.
12. The interconnections of the design unit with the external world are enumerated.
13. The properties of these interconnections are defined.

Entity declaration:

```
Entity<entity_name> is
    Port (port_name: <mode> <type>
    .....
);
```

End <entity_name>;

14. There are four modes for the ports in VHDL

in, out, inout, buffer

15. These modes describe the different kinds of interconnections that the port can have with the external circuitry.

16. Sample program:

```
Entity andgate is
Port ( c : out bit;
      a: in bit;
      b : in bit
      );
End andgate;
```

Architecture:

17. Architecture defines the functionality of the entity.

18. It forms the body of the VHDL code.

19. An architecture belongs to a specific entity.

20. Various constructs are used in the description of the architecture.

21. architecture declaration:

```
architecture <architecture_name >of
<entity _ name >is
<declerations>
Begin
<vhdl statements>
end <architecture_name>;
```

22. Example of a VHDL architecture is

```
architecture arc_andgate of andgate is
begin
    c <= a and b;
end arc_andgate;
```

Write the VHDL code for AND gate.

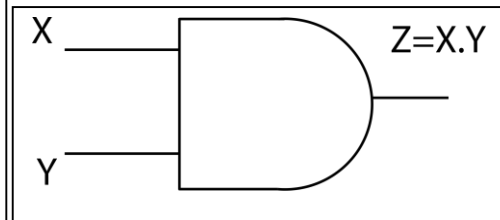
```
Library ieee;
use ieee.std_logic_1164.all;

Entity andgate is
Port(
    x, y: in std_logic;
    z : out std_logic;
);

End andgate;

architecture arc_andgate of andgate is
begin
    z <= x and y;
end arc_andgate;
```

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1



Write the VHDL code for full adder circuit

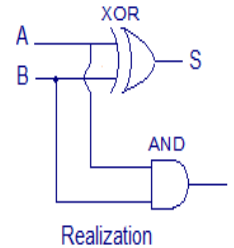
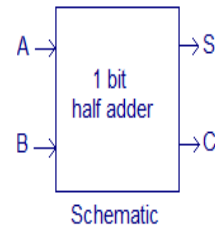
```
Library ieee;
use ieee.std_logic_1164.all;

Entity half_adder is
Port(
    a, b: in std_logic;
    c, s : out std_logic;
);
End half_adder;

architecture arc_half_adder of half_adder is
begin
    c<= x and y;
    s<= x xor y;
end arc_half_adder;
```

Inputs		Outputs	
A	B	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Truth table



Signals

23. Syntax: signal signal_name <list of signals> : type := initial_value;
24. Equivalent to wires.
25. Connect design entities together and communicate changes in values within a design.
26. Computed value is assigned to signal after a specified delay called as Delta Delay.
27. Signals can be declared in an entity (it can be seen by all the architectures), in an architecture (local to the architecture), in a package (globally available to the user of the package) or as a parameter of a subprogram (I.e. function or procedure).
28. Signals have three properties attached to it.
Type and Type attributes, value, Time (It has a history).
29. Signal assignment be done by using assignment operator: '<=>'.
29. Signal assignment is concurrent outside a process & sequential within a process.


```

Library ieee;
use ieee.std_logic_1164.all;

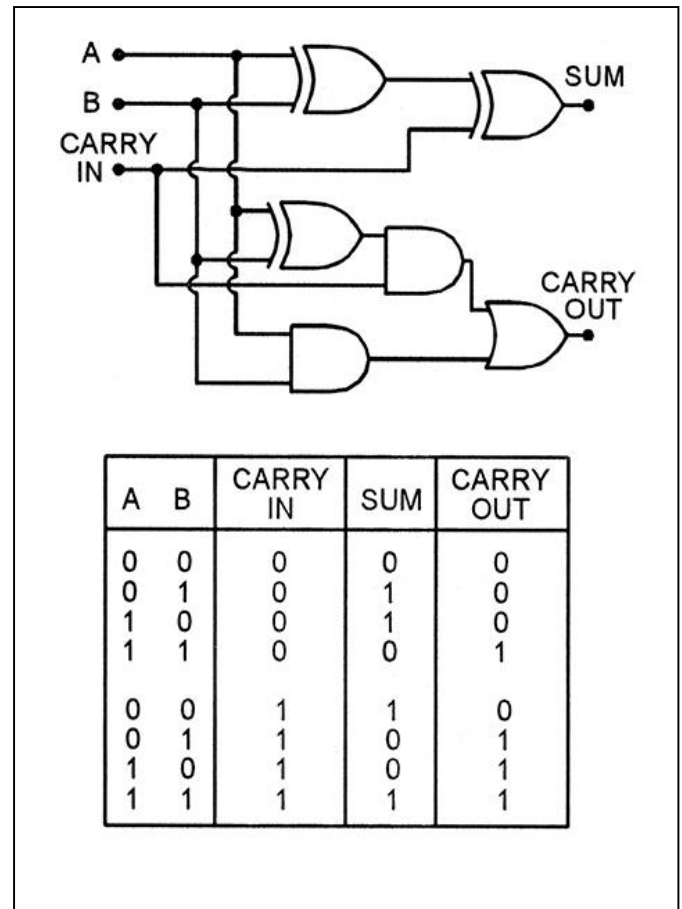
Entity full_adder is
Port(
    a, b, c: in std_logic;
    carry, sum : out std_logic;
);
End full_adder;

architecture arc_full_adder of full_adder is
signal x, y, z : std_logic;
begin
x<= a xor b;
sum<= x xor c;

y<= x and c;
z<= a and b;
carry <= y or z;
end arc_full_adder;

```

Write the VHDL code for full adder circuit.



Structural Modeling:

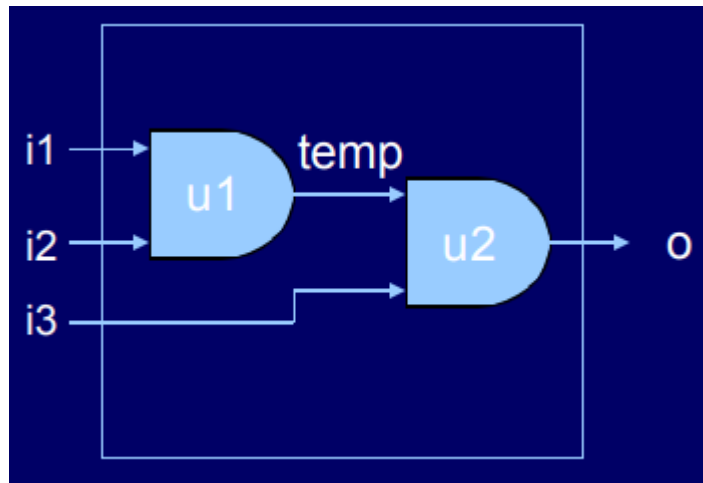
31. An entity is modeled as a set of components connected by signals, that is, as a netlist.
32. The behavior of the entity is not explicitly apparent from its model.
33. The component instantiation statement is the primary mechanism used for describing such a model of an entity.
34. A component instantiated in a structural description must first be declared using a component declaration.
35. A larger design entity can call a smaller design unit in it.
36. This forms a hierarchical structure.
37. This is allowed by a feature of VHDL called component instantiation.
38. A component is a design entity in itself which is instantiated in the larger entity.
39. A component is a design entity in itself which is instantiated in the larger entity.

40. Syntax:

```
component <component_name >
port (
    <port_name>: <mode> <type>;
    .....
);
end component;
```

41. The instance (calling) of a component in the entity is described as:
 <instant_name>: <component_name>port map
 (<association_list>);

42. For example: Fig is given as below:

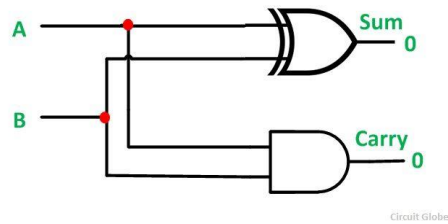


```

Library ieee;
use ieee.std_logic_1164.all;
entity and3gate is
port
(
    o : out std_logic;
    i1 : in std_logic;
    i2 : in std_logic;
    i3 : in std_logic
);
end and3gate;
architecture arc_and3gate of and3gate is
    component andgate is
    port
    (
        c : out std_logic;
        a : in std_logic;
        b : in std_logic
    );
    end component;
    signal temp1 : std_logic;
    begin
        u1:  andgate
            port map(temp1, i1, i2);
        u2:  andgate
            port map(o, temp1, i3);
    end arc_and3gate;

```

#Write a VHDL code to implement the half adder using structural modeling or component.



```
Library ieee;
use ieee.std_logic_1164.all;

entity andgate is
port
(
    c : out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end andgate;

architecture arch_andgate of andgate is
begin
    c<=a and b;
end arch_andgate;

entity xorgate is
port
(
    c : out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end xorgate;

architecture arch_xorgate of xorgate is
begin
```

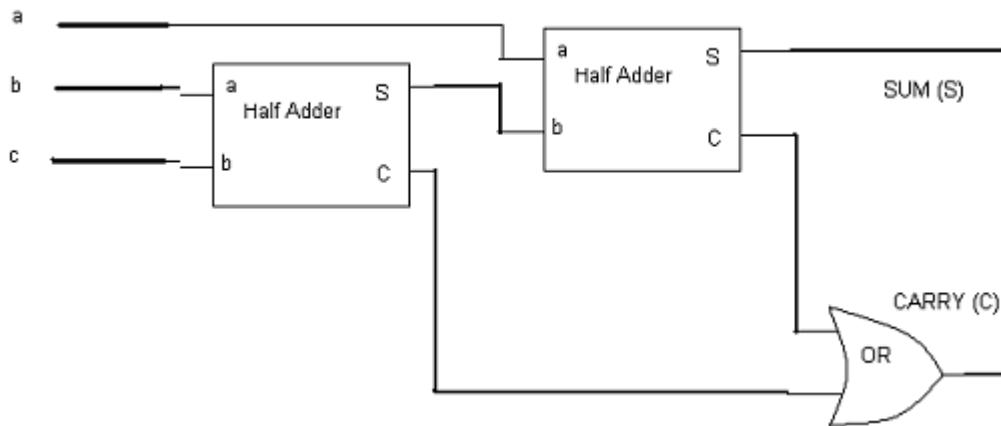
```

c<=a xor b;
end arch_xorgate;
entity halfadder is
port
(
    carry : out std_logic;
    Sum: out std_logic;
    a : in std_logic;
    b: in std_logic;
);
end halfadder;
architecture arch_halfadder of halfadder is
component andgate
port
(
    a, b : in std_logic;
    c: out std_logic;
);
End component;
Component xorgate
(
    a, b : in std_logic;
    c : out std_logic;
);
End component;
Begin
    U0: andgate portmap(carry, a, b);
    U1: xorgate portmap (sum, a, b);
End arch_halfadder;

```

Write the VHDL code to design the full adder circuit using gates as the component.

#Write a VHDL code to implement the full adder using two half adder.



```
Library ieee;
use ieee.std_logic_1164.all;
entity halfadder is
port
(
    s, c : out std_logic;
    x : in std_logic;
    y: in std_logic;
);
end halfadder;
architecture arch_halfadder of halfadder is
begin
s<=x xor y;
c<= x and y;
end arch_halfadder;
entity fulladder is
port
(
    a,b,c : in std_logic;
    Sum, carry : out std_logic;
```

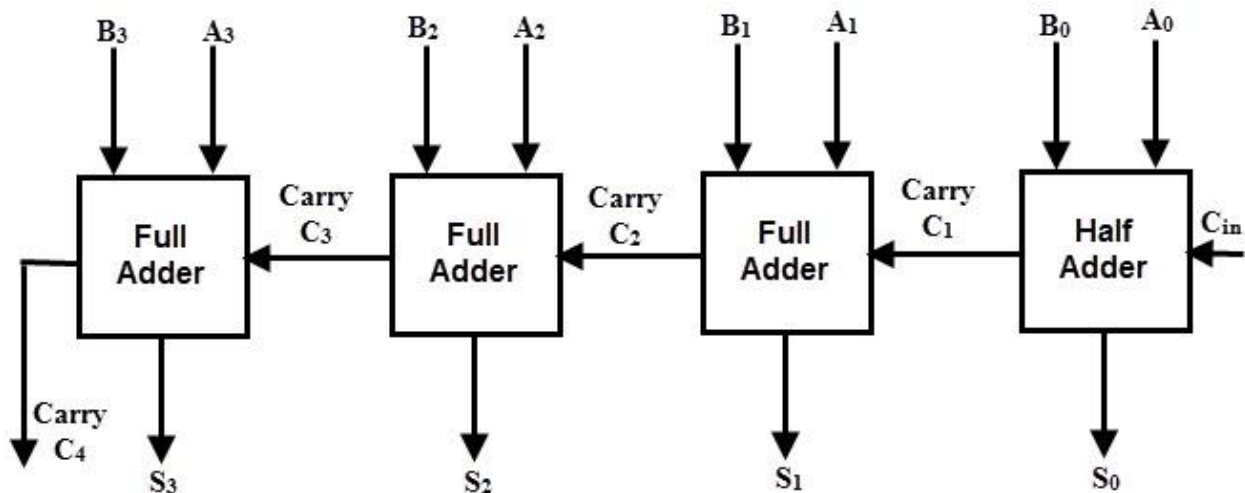
```

);
end fulladder;

architecture arch_fulladder of fulladder is
component halfadder
port
(
    x,y : in std_logic;
    s,c : out std_logic
);
End component;
Signal b1, b2, b3 : std_logic;
begin
U1: halfadder portmap(b,c,b2,b1);
U2: halfadder portmap (b2,a,sum,b3)
carry<= b1 or b2;
end arch_fulladder;

```

Write the VHDL code for 4-bit binary parallel adder.



```

Library ieee;
use ieee.std_logic_1164.all;

entity fulladder is

```

```

port
(
    sum, carry : out std_logic;
    a,b c. : in std_logic;
);
end halfadder;

architecture arc_full_adder of full_adder is
signal x, y, z : std_logic;
begin
x<= a xor b;
sum<= x xor c;
y<= x and c;
z<= a and b;
carry<= y or z;
end arc_full_adder;

entity BPA4 is
port
(
    A,B: in std_logic_vector(3 down to 0);
    Sum_bpa: out std_logic_vector(3 down to 0);
    Cin : in std_logic;
    Cout: out std_logic;
);
end BPA4;

architecture arc_BPA4 of BPA4 is
component fulladder
port
(
    sum, carry : out std_logic;
    a,b c. : in std_logic;
);

```



```

End component;

signal c1, c2, c3 : std_logic;

begin

U0: fulladder portmap(A(0),B(0), Cin, sum(0),c1);

U1: fulladder portmap(A(1),B(1), C1, sum(1),c2);

U2: fulladder portmap(A(2),B(2), C2, sum(2),c2);

U0: fulladder portmap(A(3),B(3), C3, sum(3),Cout);

end arc_BPA4;

```

Concatenation

43. This is the process of combining two signals into a single set which can be individually addressed.
44. The concatenation operator is '&'.
45. A concatenated signal's value is written in double quotes whereas the value of a single bit signal is written in single quotes.

Decision-making statements:

If statements:

```

If (expression) then
    S1
Elseif (expression) then
    S2
Elseif (expression) then
    S3
Elseif (expression) then
    S4
.....
.....
.....
Elseif (expression) then
    Sn
Else
    Sn+1

```

End if;

With-Select

46. The with-select statement is used for selective signal assignment.

47. It is a concurrent statement.

48. Syntax:

```
with expression select:
    target <= expression 1 when choice1
    expression 2 when choice2 . . .
    .....
    expression N when others;
```

49. Example:

```
entity mux2 is
port
(
    i0, i1 : in bit_vector(1 downto 0);
    y : out bit_vector(1 downto 0);
    sel : in bit
);
end mux2;
architecture behaviour of mux2 is
begin
    with sel select y <= i0 when '0',
    i1 when '1';
end behaviour;
```

When-Else

50. syntax :

```
Signal_name<= expression1 when condition1
else expression2 when condition2
else expression3;
```

51. Example:

```
entity tri_state is
port
```

```

        (      a, enable : in std_logic;
              b : out std_logic
        ); end tri_state;

architecture beh of tri_state is

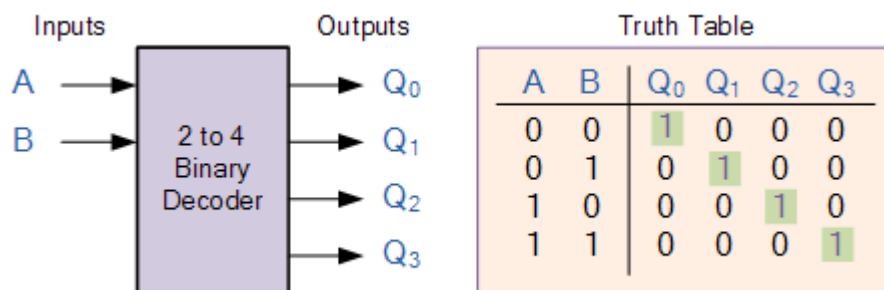
begin

        b <= a when enable = '1'
        else 'Z';

end beh;

```

Write the VHDL code to implement the 2 to 4 decoder.



```

Library ieee;

use ieee.std_logic_1164.all;

entity decoder is

port

(      SW : in std_logic_vector (1 down to 0);
      Q : out std_logic_vector(3 down to 0);
);

end decoder;

architecture arc_decoder of decoder is

begin

        if (SW = "00") then
            Q<= "0001" ;
        elseif (SW = "01") then

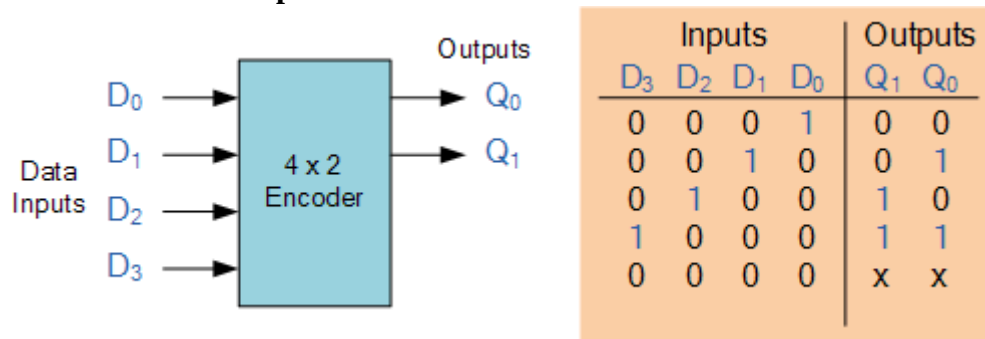
```

```

        Q<= "0010" ;
    elseif (SW = "10") then
        Q<= "0100";
    else
        Q<= "1000";
    Endif;
end arc_decoder;

```

#Write the VHDL code to implement the 4 to 2 encoder.



```

Library ieee;
use ieee.std_logic_1164.all;

entity enecoder is
port
(
    Q : out std_logic_vector (1 down to 0);
    D: in std_logic_vector(3 down to 0);
);
end encoder;

architecture arc_encoder of encoder is
begin
    if (D = "0001") then
        Q<= "00" ;
    elseif (Q = "0001") then
        Q<= "01" ;
    end if;
end arc_encoder;

```

```

elseif (D= "10") then
Q<= "10";
else
Q<= "11";
Endif;
end arc_encoder;

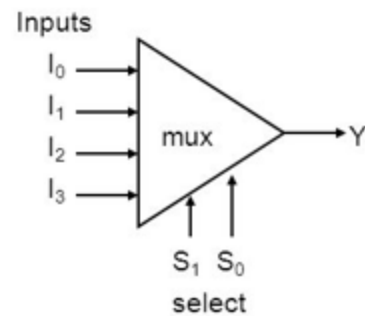
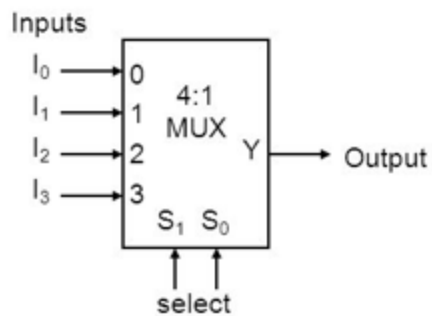
```

Write the VHDL code for implementation of 4 to 1 MUX.

Truth table for a 4-to-1 multiplexer:

I ₀	I ₁	I ₂	I ₃	S ₁	S ₀	Y
d ₀	d ₁	d ₂	d ₃	0	0	d ₀
d ₀	d ₁	d ₂	d ₃	0	1	d ₁
d ₀	d ₁	d ₂	d ₃	1	0	d ₂
d ₀	d ₁	d ₂	d ₃	1	1	d ₃

S ₁	S ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃



```

Library ieee;
use ieee.std_logic_1164.all;
entity MUX is
port
(
    Q : out std_logic;
    I0,I1,I2,I3 : in std_logic;
    SL: in std_logic_vector (1 down to 0);
);
end MUX;

```

architecture arc_MUX of MUX is

begin

if (SL= "00") then

Q<= I0;

elseif (SL= "01") then

Q<= I1 ;

elseif (SL= "10") then

Q<= I2;

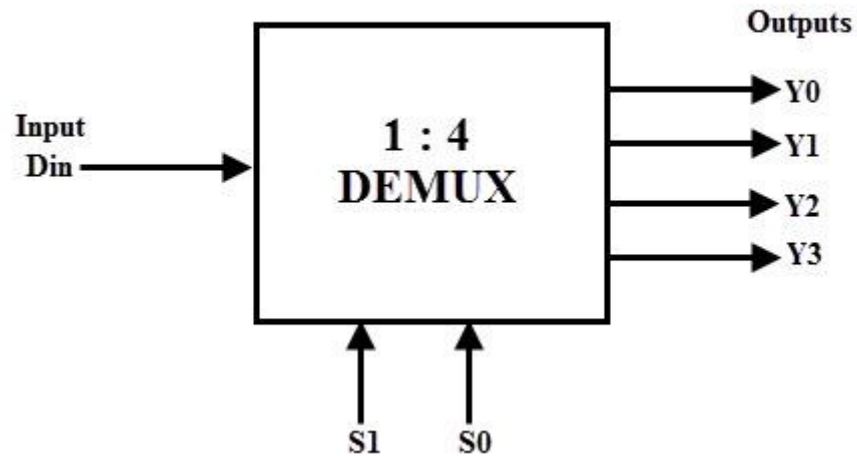
else

Q<= I4;

Endif;

end arc_MUX;

Write the VHDL code for implementation of 4 to 1 MUX.



Library ieee;

use ieee.std_logic_1164.all;

entity DMUX is

port

(Din : in std_logic;

Y0,Y1,Y2,Y3 : out std_logic;

```

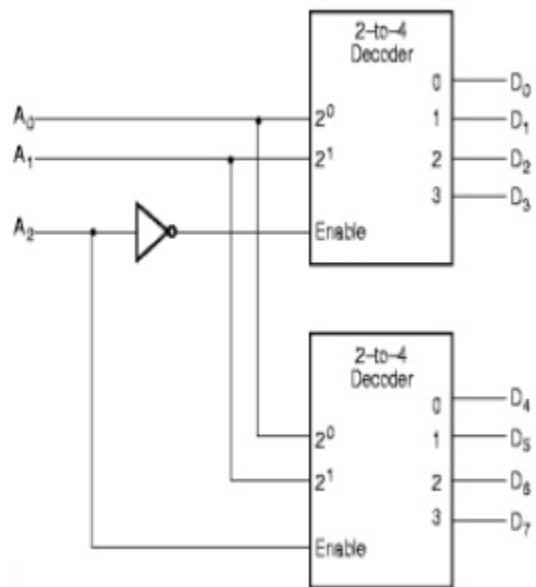
        SL: in std_logic_vector (1 down to 0);

    );
end DMUX;

architecture arc_DMUX of DMUX is
begin
    if (SL= "00") then
        Q0<= Din;
    elseif (SL= "01") then
        Q1<= Din ;
    elseif (SL= "10") then
        Q2<= Din;
    else
        Q3<= Din;
    Endif;
end arc_MUX;

```

Write the VHDL code to implement 3 to 8 decoder using two 2 to 4 decoder.



Do your self

Process Statement:

- A process statement defines an independent sequential process representing the behavior of some portion of the design.
- Simplified Syntax

[process_label:] process [(sensitivity_list)] [is]

process_declarations

begin

sequential_statements

end process [process_label] ;

- The process statement represents the behavior of some portion of the design. It consists of the sequential statements whose execution is made in order defined by the user.
- Each process can be assigned an optional label.
- The process declarative part defines local items for the process and may contain declarations of: subprograms, types, subtypes, constants, variables, files, aliases, attributes, use clauses and group declarations. It is not allowed to declare signals or shared variables inside processes.
- The statements, which describe the behavior in a process, are executed sequentially, in the order in which the designer specifies them. The execution of statements, however, does not terminate with the last statement in the process, but is repeated in an infinite loop. The loop can be *suspended* and *resumed* with *wait* statements. When the next statement to be executed is a *wait* statement, the process suspends its execution until a condition supporting the wait statement is met. See respective topics for details.
- A process declaration may contain optional sensitivity list. The list contains identifiers of signals to which the process is sensitive. A change of a value of any of those signals causes the suspended process to resume. A sensitivity list is a full equivalent of a *wait on sensitivity_list* statement at the end of the process. It is not allowed, however, to use wait statements and sensitivity list in the same process. In addition, if a process with a sensitivity list calls a procedure, then the procedure cannot contain any wait statements.

Sequential Circuits-Gated D Latch

```

LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop IS
    PORT ( D, Clock : IN    STD_LOGIC ;
          Q          : OUT  STD_LOGIC) ;
END flipflop ;

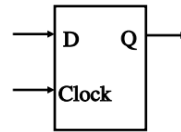
```

```

ARCHITECTURE behavioral OF flipflop IS
BEGIN
    PROCESS ( Clock )
    BEGIN
        IF Clock'EVENT AND Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END behavioral ;

```

Graphical symbol

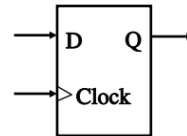


Truth table

Clock	D	Q(t+1)
0	–	Q(t)
1	0	0
1	1	1

Positive-edge-triggered D Flip-Flop

Graphical symbol



Truth table

Clk	D	Q(t+1)
↑	0	0
↑	1	1
0	–	Q(t)
1	–	Q(t)

```

LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY latch IS
    PORT ( D, Clock : IN      STD_LOGIC ;
          Q          : OUT    STD_LOGIC) ;
END latch ;

ARCHITECTURE behavioral OF latch IS
BEGIN
    PROCESS ( D, Clock )
    BEGIN
        IF Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END behavioral;

```

VHDL Code for a D Flip-Flop with Asynchronous Reset

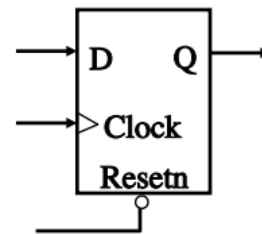
```

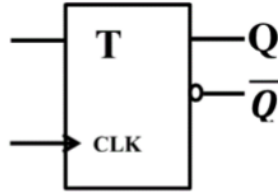
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop_ar IS
    PORT ( D, Resetn, Clock : IN      STD_LOGIC ;
          Q                  : OUT    STD_LOGIC) ;
END flipflop_ar ;

ARCHITECTURE behavioral OF flipflop_ar IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= '0' ;
        ELSIF rising_edge(Clock) THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END behavioral ;

```





The truth table of a T-flip-flop

T	Q_{t+1}
0	Q_t
1	$\overline{Q_t}$

VHDL Code for a T Flip Flop

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

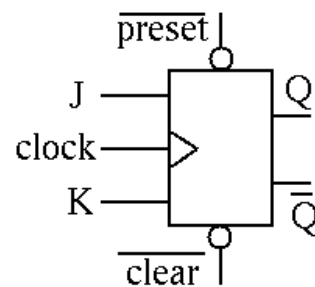
entity Toggle_flip_flop is
    port(
        t : in STD_LOGIC;
        clk : in STD_LOGIC;
        reset : in STD_LOGIC;
        dout : out STD_LOGIC
    );
end Toggle_flip_flop;

architecture virat of Toggle_flip_flop is
begin
    tff : process (t,clk,reset) is
        variable m : std_logic := '0';

    begin
        if (reset = '1') then
            m := '0';
        elsif (rising_edge (clk)) then
            if (t = '1') then
                m := not m;
            end if;
        end if;
        dout <= m;
    end process tff;
end virat;
```

VHDL code for a

JK Flip Flop



J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\overline{Q_t}$

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity jk is
  port(
    j : in STD_LOGIC;
    k : in STD_LOGIC;
    clk : in STD_LOGIC;
    reset : in STD_LOGIC;
    q : out STD_LOGIC;
    qb : out STD_LOGIC
  );
end jk;

architecture virat of jk is
begin
  jkff : process (j,k,clk,reset) is
    variable m : std_logic := '0';

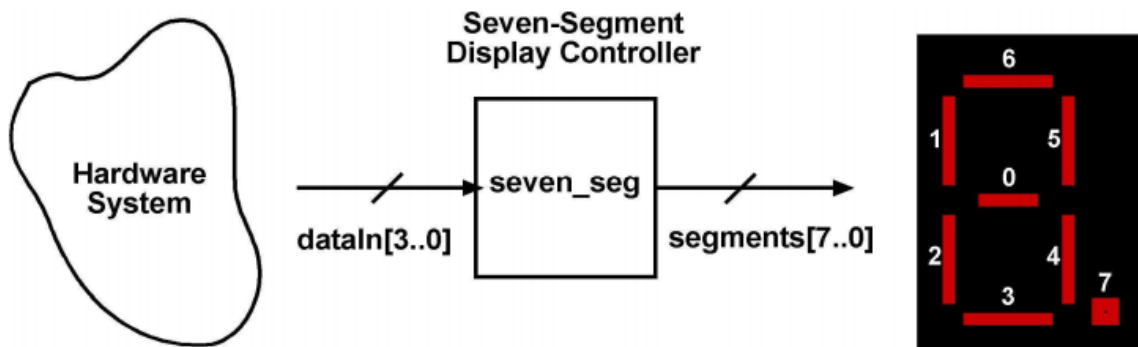
  begin
    if (reset = '1') then
      m := '0';
    elsif (rising_edge (clk)) then
      if (j /= k) then
        m := j;
      elsif (j = '1' and k = '1') then
        m := not m;
      end if;
    end if;

    q <= m;
    qb <= not m;
  end process jkff;
end virat;

```

Design synthesizable VHDL specification of seven segment display controller.

The seven segment display controller is shown below.



```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY seven_seg IS
    PORT( dataIn      : IN      std_logic_vector(3 DOWNTO 0);
          segments    : OUT     std_logic_vector(7 DOWNTO 0) );
END seven_seg;

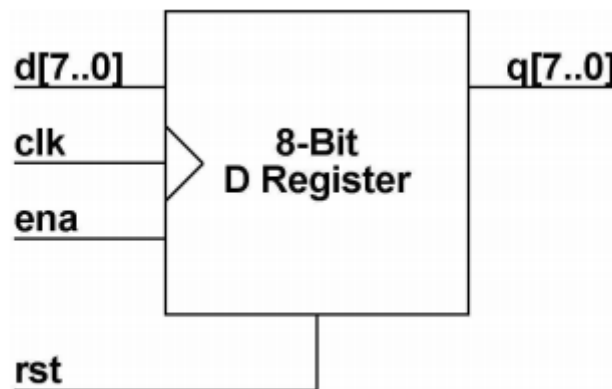
ARCHITECTURE synthesis1 OF seven_seg IS
BEGIN
    WITH dataIn SELECT
        segments <=
            "10000001" WHEN "0000",  -- 0
            "11001111" WHEN "0001",  -- 1
            "10010010" WHEN "0010",  -- 2
            "10000110" WHEN "0011",  -- 3
            "11001100" WHEN "0100",  -- 4
            "10100100" WHEN "0101",  -- 5
            "10100000" WHEN "0110",  -- 6
            "10001111" WHEN "0111",  -- 7
            "10000000" WHEN "1000",  -- 8
            "10000100" WHEN "1001",  -- 9
            "11111111" WHEN OTHERS;

END synthesis1;

```

Design the synthesizable VHDL specification of a 8-bit register with enable and asynchronous reset signal.

The block diagram of 8 –bit register shown below.



```

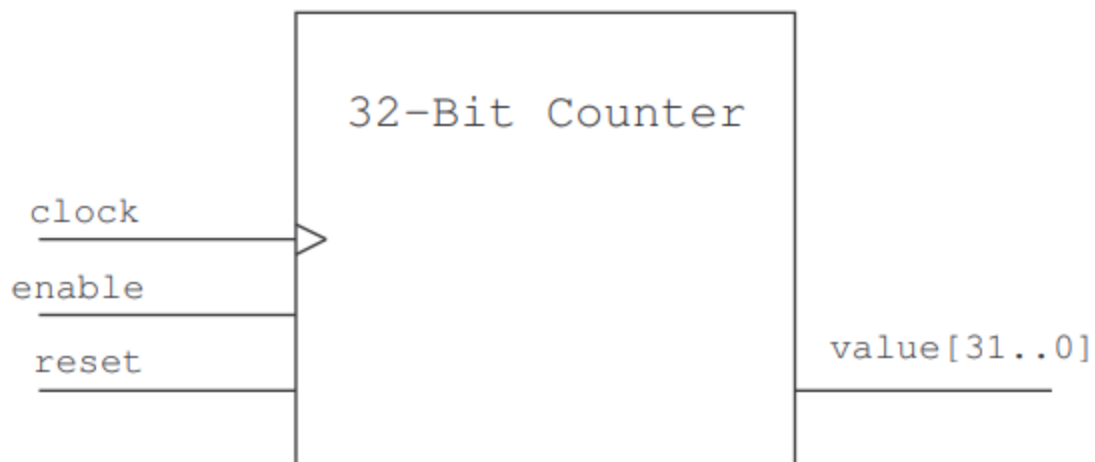
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dregister IS
    PORT( rst, clk, ena    : IN    std_logic;
          d                : IN    std_logic_vector(7 DOWNT0 0);
          q                : OUT   std_logic_vector(7 DOWNT0 0) );
END dregister;

ARCHITECTURE synthesis1 OF dregister IS
BEGIN
    PROCESS (rst, clk)
    BEGIN
        IF (rst = '1') THEN
            q <= X"00";
        ELSIF (clk'EVENT) AND (clk = '1') THEN
            IF (ena = '1') THEN
                q <= d;
            END IF;
        END IF;
    END PROCESS;
END synthesis1;

```

Design the 32-bit sequence counter using VHDL.



```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY counter IS
    PORT(
        reset          : IN      std_logic;
        clock           : IN      std_logic;
        enable          : IN      std_logic;
        value            : OUT     std_logic_vector(31 DOWNT0 0)
    );
END counter;

```

```

ARCHITECTURE synthesis1 OF counter IS

```

```

    SIGNAL count : unsigned(31 DOWNT0 0); -- The unsigned type is used
                                           -- so that unsigned arithmetic
                                           -- will be synthesized

BEGIN
    PROCESS (reset, clock)
    BEGIN
        IF (reset = '1') THEN
            count <= X"00000000";
        ELSIF (clock'EVENT) AND (clock = '1') THEN
            IF (enable = '1') THEN
                count <= count + 1;
            END IF;
        END IF;
    END PROCESS;

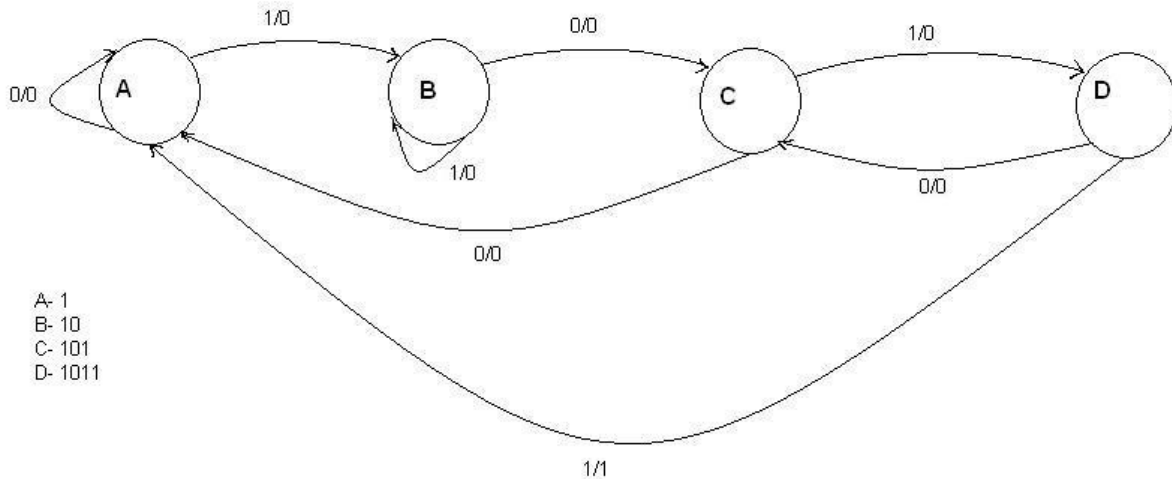
    value <= std_logic_vector(count); -- Here, the count value is
                                      -- converted to std_logic_vector
                                      -- using a conversion function

END synthesis1;

```

VHDL for Sequence Detector

State machine for detecting the (non-overlapping) sequence "1011".



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
--Sequence detector for detecting the sequence "1011".
```

```
--Non overlapping type.
```

```
entity seq_det is
```

```
port( clk : in std_logic; --clock signal
```

```
reset : in std_logic; --reset signal
```

```
seq : in std_logic; --serial bit sequence
```

```
det_vld : out std_logic --A '1' indicates the pattern "1011" is detected in the sequence.
```

```
);
```

```
end seq_det;
```

```
architecture Behavioral of seq_det is
```

```
type state_type is (A,B,C,D); --Defines the type for states in the state machine
```

```
signal state : state_type := A; --Declare the signal with the corresponding state type.
```

```
begin
```

```
process(clk)
```

```
begin
```

```
if( reset = '1' ) then --resets state and output signal when reset is asserted.
```

```
det_vld <= '0';
```

```
state <= A;
```

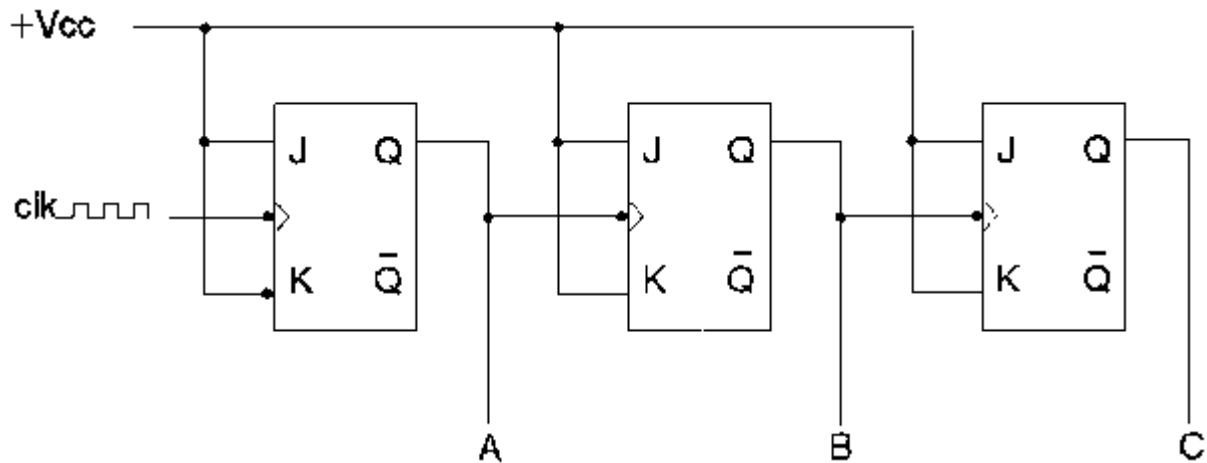
```
elsif ( rising_edge(clk) ) then --calculates the next state based on current state and input
```


bit.

```
case state is
  when A => --when the current state is A.
    det_vld <= '0';
    if ( seq = '0' ) then
      state <= A;
    else
      state <= B;
    end if;
  when B => --when the current state is B.
    if ( seq = '0' ) then
      state <= C;
    else
      state <= B;
    end if;
  when C => --when the current state is C.
    if ( seq = '0' ) then
      state <= A;
    else
      state <= D;
    end if;
  when D => --when the current state is D.
    if ( seq = '0' ) then
      state <= C;
    else
      state <= A;
      det_vld <= '1'; --Output is asserted when the pattern "1011" is found in the
sequence.
    end if;
  when others =>
    NULL;
end case;
end if;
end process;

end Behavioral;
```

VHDL code for Asynchronous counter using JK Flip Flop



```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.STD_LOGIC_ARITH.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity jkc is
```

```
Port ( clock : in std_logic;
```

```
      reset : in std_logic;
```

```
      count : out std_logic_vector(3 downto 0)
```

```
);
```

```
end jkc;
```

```
architecture rtl of jkc is
```

```
COMPONENT jkff
```

```
PORT(
```

```
  clock : in std_logic;
```

```
  reset : in std_logic;
```

```
  j   : in std_logic;
```

```
  k   : in std_logic;
```

```
  q   : out std_logic
```

```

);
END COMPONENT;

signal temp : std_logic_vector(3 downto 0) := "0000";
begin
d0 : jkff
    port map (
        reset => reset,
        clock => clock,
        j    => '1',
        k    => '1',
        q    => temp(3)
    );

d1 : jkff
    port map (
        reset => reset,
        clock => temp(3),
        j    => '1',
        k    => '1',
        q    => temp(2)
    );

d2 : jkff
    port map (
        reset => reset,
        clock => temp(2),
        j    => '1',

```

```
k    => '1',  
q    => temp(1)  
);
```

d3 : jkff

```
port map (  
    reset => reset,  
    clock => temp(1),  
    j    => '1',  
    k    => '1',  
    q    => temp(0)  
);
```

```
count(3) <= temp(0);  
count(2) <= temp(1);  
count(1) <= temp(2);  
count(0) <= temp(3);  
end rtl;
```